

Red Hat OpenShift Data Foundation 4.22

Deploying and managing OpenShift Data Foundation using Red Hat OpenStack Platform

Instructions on deploying and managing OpenShift Data Foundation on Red Hat OpenStack Platform

Last Updated: 2026-10-23

Red Hat OpenShift Data Foundation 4.22 Deploying and managing OpenShift Data Foundation using Red Hat OpenStack Platform

Instructions on deploying and managing OpenShift Data Foundation on Red Hat OpenStack Platform

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Read this document for instructions about how to install and manage Red Hat OpenShift Data Foundation using Red Hat OpenShift Container Platform on Red Hat OpenStack Platform (RHOSP). Deploying and managing OpenShift Data Foundation on Red Hat OpenStack Platform is a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

Table of Contents

PREFACE ... 6

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION ... 7

PREFACE ... 8

CHAPTER 1. PREPARING TO DEPLOY OPENSHIFT DATA FOUNDATION ... 9

CHAPTER 2. DEPLOYING OPENSHIFT DATA FOUNDATION ON RED HAT OPENSTACK PLATFORM IN INTERNAL MODE ... 12
2.1. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR 12
2.2. ENABLING CLUSTER-WIDE ENCRYPTION WITH KMS USING THE TOKEN AUTHENTICATION METHOD 13
2.3. ENABLING CLUSTER-WIDE ENCRYPTION WITH KMS USING THE KUBERNETES AUTHENTICATION METHOD 14
2.3.1. Enabling and disabling key rotation when using KMS 17
2.3.1.1. Enforcing precedence for key rotation 17
2.3.1.2. Enabling key rotation 18
2.3.1.2.1. Annotating a namespace for key rotation 18
2.3.1.2.2. Annotating a storage class for key rotation 19
2.3.1.2.3. Annotating a PersistentVolumeClaim for key rotation 19
2.3.1.3. Disabling key rotation 20
2.4. CREATING OPENSHIFT DATA FOUNDATION CLUSTER 21

CHAPTER 3. VERIFYING OPENSHIFT DATA FOUNDATION DEPLOYMENT ... 27
3.1. VERIFYING THE STATE OF THE PODS 27
3.2. VERIFYING THE OPENSHIFT DATA FOUNDATION CLUSTER IS HEALTHY 30
3.3. VERIFYING THE MULTICLOUD OBJECT GATEWAY IS HEALTHY 30
3.4. VERIFYING THAT THE SPECIFIC STORAGE CLASSES EXIST 31

CHAPTER 4. UNINSTALLING OPENSHIFT DATA FOUNDATION IN INTERNAL MODE ... 32
4.1. UNINSTALLING OPENSHIFT DATA FOUNDATION IN INTERNAL MODE 32

CHAPTER 5. DEPLOYING OPENSHIFT DATA FOUNDATION ON RED HAT OPENSTACK PLATFORM IN EXTERNAL MODE ... 33
5.1. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR 33
5.2. CREATING AN OPENSHIFT DATA FOUNDATION CLUSTER FOR EXTERNAL CEPH STORAGE SYSTEM 34
5.3. CONFIGURING TOPOLOGY AWARENESS FOR EXTERNAL MODE 40
5.4. VERIFYING YOUR OPENSHIFT DATA FOUNDATION INSTALLATION FOR EXTERNAL MODE 44
5.4.1. Verifying the state of the pods 44
5.4.2. Verifying that the OpenShift Data Foundation cluster is healthy 46
5.4.3. Verifying that the Multicloud Object Gateway is healthy 47
5.4.4. Verifying that the storage classes are created and listed 47
5.4.5. Verifying that Ceph cluster is connected 47
5.4.6. Verifying that storage cluster is ready 48
5.4.7. Verifying the state of the pods 48
5.4.8. Verifying that the OpenShift Data Foundation cluster is healthy 50
5.4.9. Verifying that the Multicloud Object Gateway is healthy 51
5.4.10. Verifying that the storage classes are created and listed 51
5.4.11. Verifying that Ceph cluster is connected 52
5.4.12. Verifying that storage cluster is ready 52

CHAPTER 6. UNINSTALLING OPENSHIFT DATA FOUNDATION IN EXTERNAL MODE ... 54

6.1. UNINSTALLING OPENSHIFT DATA FOUNDATION FROM EXTERNAL STORAGE SYSTEM 54
6.2. REMOVING MONITORING STACK FROM OPENSHIFT DATA FOUNDATION 57
6.3. REMOVING OPENSHIFT CONTAINER PLATFORM REGISTRY FROM OPENSHIFT DATA FOUNDATION 61
6.4. REMOVING THE CLUSTER LOGGING OPERATOR FROM OPENSHIFT DATA FOUNDATION 62

CHAPTER 7. DEPLOY STANDALONE MULTICLOUD OBJECT GATEWAY IN INTERNAL MODE 64
7.1. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR 64
7.2. CREATING A STANDALONE MULTICLOUD OBJECT GATEWAY 65

CHAPTER 8. VIEW OPENSHIFT DATA FOUNDATION TOPOLOGY 70

CHAPTER 9. STORAGE CLASSES AND STORAGE POOLS 71
9.1. CREATING STORAGE CLASSES AND POOLS 71
9.2. CREATING A STORAGE CLASS FOR PERSISTENT VOLUME ENCRYPTION 72

CHAPTER 10. CONFIGURE STORAGE FOR OPENSHIFT CONTAINER PLATFORM SERVICES 77
10.1. CONFIGURING IMAGE REGISTRY TO USE OPENSHIFT DATA FOUNDATION 77
10.2. CONFIGURING MONITORING TO USE OPENSHIFT DATA FOUNDATION 79

CHAPTER 11. CLUSTER LOGGING FOR OPENSHIFT DATA FOUNDATION 82
11.1. CONFIGURING PERSISTENT STORAGE 82
11.2. CONFIGURING CLUSTER LOGGING TO USE OPENSHIFT DATA FOUNDATION 83

CHAPTER 12. BACKING OPENSHIFT CONTAINER PLATFORM APPLICATIONS WITH OPENSHIFT DATA FOUNDATION 86

CHAPTER 13. HOW TO USE DEDICATED WORKER NODES FOR RED HAT OPENSHIFT DATA FOUNDATION . 88
13.1. ANATOMY OF AN INFRASTRUCTURE NODE 88
13.2. MACHINE SETS FOR CREATING INFRASTRUCTURE NODES 88
13.3. MANUAL CREATION OF INFRASTRUCTURE NODES 89

CHAPTER 14. SCALING STORAGE NODES 91
14.1. REQUIREMENTS FOR SCALING STORAGE NODES 91
14.2. SCALING UP STORAGE BY ADDING CAPACITY TO YOUR OPENSHIFT DATA FOUNDATION NODES ON RED HAT OPENSTACK PLATFORM INFRASTRUCTURE 91
14.3. SCALING OUT STORAGE CAPACITY BY ADDING NEW NODES 94
14.3.1. Adding a node to an installer-provisioned infrastructure 94
14.4. SCALING UP STORAGE CAPACITY 95

CHAPTER 15. MULTICLOUD OBJECT GATEWAY 96
15.1. ABOUT THE MULTICLOUD OBJECT GATEWAY 96
15.2. ACCESSING THE MULTICLOUD OBJECT GATEWAY WITH YOUR APPLICATIONS 96
15.2.1. Accessing the RADOS Object Gateway S3 endpoint 96
15.3. ADDING STORAGE RESOURCES FOR HYBRID OR MULTICLOUD 96
15.3.1. Creating a new backing store 96
15.3.2. Adding storage resources for hybrid or Multicloud using the command-line interface 97
15.3.2.1. Creating an AWS-backed backingstore 98
15.3.2.2. Creating an IBM COS-backed backingstore 99
15.3.2.3. Creating an Azure-backed backingstore 101
15.3.2.4. Creating a GCP-backed backingstore 102
15.3.2.5. Creating a local Persistent Volume-backed backingstore 104
15.3.3. Creating an s3 compatible Multicloud Object Gateway backingstore 105
15.3.4. Adding storage resources for hybrid and Multicloud using the user interface 107
15.3.5. Creating a new bucket class 109

15.3.6. Editing a bucket class 110
15.3.7. Editing backing stores for bucket class 110
15.4. MANAGING NAMESPACE BUCKETS 111
15.4.1. Amazon S3 API endpoints for objects in namespace buckets 111
15.4.2. Adding a namespace bucket using the Multicloud Object Gateway CLI and YAML 112
15.4.2.1. Adding an AWS S3 namespace bucket using YAML 113
15.4.2.2. Adding an IBM COS namespace bucket using YAML 115
15.4.2.3. Adding an AWS S3 namespace bucket using the Multicloud Object Gateway CLI 118
15.4.2.4. Adding an IBM COS namespace bucket using the Multicloud Object Gateway CLI 120
15.4.3. Adding a namespace bucket using the OpenShift Container Platform user interface 121
15.5. MIRRORING DATA FOR HYBRID AND MULTICLOUD BUCKETS 123
15.5.1. Creating bucket classes to mirror data using the command-line interface 123
15.5.2. Creating bucket classes to mirror data using a YAML 123
15.6. BUCKET POLICIES IN THE MULTICLOUD OBJECT GATEWAY 124
15.6.1. Introduction to bucket policies 124
15.6.2. Using bucket policies in Multicloud Object Gateway 124
15.6.3. Creating a user in the Multicloud Object Gateway 125
15.7. OBJECT BUCKET CLAIM 126
15.7.1. Object Bucket Claim 126
15.7.2. Dynamic Object Bucket Claim 127
15.7.3. Creating an Object Bucket Claim using the command-line interface 129
15.7.4. Creating an Object Bucket Claim using the OpenShift Web Console 132
15.7.5. Attaching an Object Bucket Claim to a deployment 133
15.7.6. Viewing object buckets using the OpenShift Web Console 133
15.7.7. Deleting Object Bucket Claims 134
15.8. CACHING POLICY FOR OBJECT BUCKETS 134
15.8.1. Creating an AWS cache bucket 134
15.8.2. Creating an IBM COS cache bucket 136
15.9. SCALING MULTICLOUD OBJECT GATEWAY PERFORMANCE BY ADDING ENDPOINTS 138
15.9.1. S3 endpoints in the Multicloud Object Gateway 138
15.9.2. Scaling the Multicloud Object Gateway with storage nodes 138
15.10. AUTOMATIC SCALING OF MULTICLOUD OBJECT GATEWAY ENDPOINTS 139

CHAPTER 16. MANAGING PERSISTENT VOLUME CLAIMS 141
16.1. CONFIGURING APPLICATION PODS TO USE OPENSHIFT DATA FOUNDATION 141
16.2. VIEWING PERSISTENT VOLUME CLAIM REQUEST STATUS 143
16.3. REVIEWING PERSISTENT VOLUME CLAIM REQUEST EVENTS 143
16.4. DYNAMIC PROVISIONING 143
16.4.1. About dynamic provisioning 144
16.4.2. Dynamic provisioning in OpenShift Data Foundation 144
16.4.3. Available dynamic provisioning plug-ins 144

CHAPTER 17. VOLUME SNAPSHOTS 146
17.1. CREATING VOLUME SNAPSHOTS 146
17.2. RESTORING VOLUME SNAPSHOTS 147
17.3. DELETING VOLUME SNAPSHOTS 148

CHAPTER 18. VOLUME CLONING 150
18.1. CREATING A CLONE 150

CHAPTER 19. REPLACING STORAGE NODES 151

CHAPTER 20. REPLACING OPERATIONAL NODES ON RED HAT OPENSTACK PLATFORM INSTALLER-PROVISIONED INFRASTRUCTURE 152

CHAPTER 21. REPLACING FAILED NODES ON RED HAT OPENSTACK PLATFORM INSTALLER-PROVISIONED INFRASTRUCTURE 154

CHAPTER 22. REPLACING STORAGE DEVICES 156

CHAPTER 23. REPLACING OPERATIONAL OR FAILED STORAGE DEVICES ON RED HAT OPENSTACK PLATFORM INSTALLER-PROVISIONED INFRASTRUCTURE 157

CHAPTER 24. UPGRADING TO OPENSHIFT DATA FOUNDATION 163
24.1. OVERVIEW OF THE OPENSHIFT DATA FOUNDATION UPDATE PROCESS 163
24.2. UPDATING RED HAT OPENSHIFT DATA FOUNDATION 4.21 TO 4.22 165
24.3. UPDATING RED HAT OPENSHIFT DATA FOUNDATION 4.22.X TO 4.22.Y 168
24.4. CHANGING THE UPDATE APPROVAL STRATEGY 170

PREFACE

Deploy and manage OpenShift Data Foundation on OpenShift Container Platform running on Red Hat OpenStack Platform (RHOSP) in internal or external mode.

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION

We appreciate your input on our documentation. Do let us know how we can make it better.

To give feedback, create a Jira ticket:

Procedure

1. Log in to the Jira.
2. Click **Create** in the top navigation bar
3. Enter a descriptive title in the Summary field.
4. Enter your suggestion for improvement in the Description field. Include links to the relevant parts of the documentation.
5. Select **Documentation** in the Components field.
6. Click Create at the bottom of the dialogue.

PREFACE

Red Hat OpenShift Data Foundation supports deployment on existing Red Hat OpenShift Container Platform (RHOCP) using Red Hat OpenStack Platform clusters.



NOTE

Both internal and external OpenShift Data Foundation clusters are supported on Red Hat OpenStack Platform. See the Planning your deployment guide for more information about deployment requirements.

To deploy OpenShift Data Foundation, start with the requirements in the Preparing to deploy OpenShift Data Foundation chapter and then follow the appropriate deployment process based on your requirement:

- Internal mode
  - Deploying OpenShift Data Foundation on Red Hat OpenStack Platform in internal mode
  - Deploy standalone Multicloud Object Gateway component
- External mode
  - Deploying OpenShift Data Foundation on Red Hat OpenStack Platform in external mode

Additional resources

- Planning your deployment
- Preparing to deploy OpenShift Data Foundation
- Deploying OpenShift Data Foundation on Red Hat OpenStack Platform in internal mode
- Deploy standalone Multicloud Object Gateway component
- Deploying OpenShift Data Foundation on Red Hat OpenStack Platform in external mode

CHAPTER 1. PREPARING TO DEPLOY OPENSHIFT DATA FOUNDATION

Deploying OpenShift Data Foundation on OpenShift Container Platform using dynamic storage devices provides you with the option to create internal cluster resources.

Before you begin the deployment of Red Hat OpenShift Data Foundation, follow these steps:

1. Optional: If you want to enable cluster-wide encryption using the external Key Management System (KMS) HashiCorp Vault, follow these steps:
   - Ensure that you have a valid Red Hat OpenShift Data Foundation Advanced subscription. To know how subscriptions for OpenShift Data Foundation work, see *knowledgebase article on OpenShift Data Foundation subscriptions*.
   - When the Token authentication method is selected for encryption then refer to *Enabling cluster-wide encryption with the Token authentication using KMS*.
   - When the Kubernetes authentication method is selected for encryption then refer to *Enabling cluster-wide encryption with the Kubernetes authentication using KMS* .
   - Ensure that you are using signed certificates on your Vault servers.
2. Optional: If you want to enable cluster-wide encryption using the external Key Management System (KMS) Thales CipherTrust Manager, you must first enable the Key Management Interoperability Protocol (KMIP) and use signed certificates on your server. Follow these steps:
   a. Create a KMIP client if one does not exist. From the user interface, select **KMIP → Client Profile → Add Profile**.
      i. Add the **CipherTrust** username to the **Common Name** field during profile creation.
   b. Create a token by navigating to **KMIP → Registration Token → New Registration Token**. Copy the token for the next step.
   c. To register the client, navigate to **KMIP → Registered Clients → Add Client**. Specify the **Name**. Paste the **Registration Token** from the previous step, then click **Save**.
   d. Download the Private Key and Client Certificate by clicking **Save Private Key** and **Save Certificate** respectively.
   e. To create a new KMIP interface, navigate to **Admin Settings → Interfaces → Add Interface**.
      i. Select **KMIP Key Management Interoperability Protocol** and click **Next**.
      ii. Select a free **Port**.
      iii. Select **Network Interface** as **all**.
      iv. Select **Interface Mode** as **TLS, verify client cert, user name taken from client cert, auth request is optional**.
      v. (Optional) You can enable hard delete to delete both metadata and material when the key is deleted. It is disabled by default.
      vi. Select the CA to be used, and click **Save**.

f. To get the server CA certificate, click on the Action menu **(⋮)** on the right of the newly created interface, and click **Download Certificate**.

g. Optional: If StorageClass encryption is to be enabled during deployment, create a key to act as the Key Encryption Key (KEK):

   i. Navigate to **Keys → Add Key**.

   ii. Enter **Key Name**.

   iii. Set the **Algorithm** and **Size** to **AES** and **256** respectively.

   iv. Enable **Create a key in Pre-Active state**and set the date and time for activation.

   v. Ensure that **Encrypt** and **Decrypt** are enabled under **Key Usage**.

   vi. Copy the ID of the newly created Key to be used as the Unique Identifier during deployment.

3. Minimum starting node requirements:
   An OpenShift Data Foundation cluster will be deployed with minimum configuration when the standard deployment resource requirement is not met. See *Resource requirements* in the *Planning guide*.

4. Disaster recovery requirements:
   Disaster Recovery features supported by Red Hat OpenShift Data Foundation require all of the following prerequisites to successfully implement a disaster recovery solution:

   - A valid Red Hat OpenShift Data Foundation Advanced subscription
   - A valid Red Hat Advanced Cluster Management for Kubernetes subscription
     To know how subscriptions for OpenShift Data Foundation work, see *knowledgebase article on OpenShift Data Foundation subscriptions*.

     For detailed requirements, see *Configuring OpenShift Data Foundation Disaster Recovery for OpenShift Workloads* guide, and the *Requirements and recommendations* section of the *Install guide* in Red Hat Advanced Cluster Management for Kubernetes documentation.

5. Disconnected environment requirements:
   For the list of packages to be mirrored for installing OpenShift Data Foundation in disconnected environments, see the Disconnected environment section in the *Planning your deployment* guide.

Additional resources

- knowledgebase article on OpenShift Data Foundation subscriptions
- Enabling cluster-wide encryption with the Token authentication using KMS
- Enabling cluster-wide encryption with the Kubernetes authentication using KMS
- Resource requirements
- Disconnected environment
- Configuring OpenShift Data Foundation Disaster Recovery for OpenShift Workloads

- Requirements and recommendations (Red Hat Advanced Cluster Management for Kubernetes Install guide)

CHAPTER 2. DEPLOYING OPENSHIFT DATA FOUNDATION ON RED HAT OPENSTACK PLATFORM IN INTERNAL MODE

Deploying OpenShift Data Foundation on OpenShift Container Platform in internal mode using dynamic storage devices provided by Red Hat OpenStack Platform installer-provisioned infrastructure (IPI) enables you to create internal cluster resources. This results in internal provisioning of the base services, which helps to make additional storage classes available to applications.

Ensure that you have addressed the requirements in *Preparing to deploy OpenShift Data Foundation* chapter before proceeding with the below steps for deploying using dynamic storage devices:

1. Install the Red Hat OpenShift Data Foundation Operator.
2. Create the OpenShift Data Foundation Cluster.

2.1. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR

You can install Red Hat OpenShift Data Foundation Operator using the Red Hat OpenShift Container Platform Software Catalog.

Prerequisites

- Access to an OpenShift Container Platform cluster using an account with **cluster-admin** and operator installation permissions.
- You must have at least three worker or infrastructure nodes in the Red Hat OpenShift Container Platform cluster.
- For additional resource requirements, see the Planning your deployment guide.



IMPORTANT

- When you need to override the cluster-wide default node selector for OpenShift Data Foundation, you can use the following command to specify a blank node selector for the **openshift-storage** namespace (create **openshift-storage** namespace in this case):

```
$ oc annotate namespace openshift-storage openshift.io/node-selector=
```

- Taint a node as **infra** to ensure only Red Hat OpenShift Data Foundation resources are scheduled on that node. This helps you save on subscription costs. For more information, see the *How to use dedicated worker nodes for Red Hat OpenShift Data Foundation* section in the Managing and Allocating Storage Resources guide.

Procedure

1. Log in to the OpenShift Web Console.
2. Click **Ecosystem → Software Catalog**

3. Scroll or type **OpenShift Data Foundation** into the **Filter by keyword** box to find the **OpenShift Data Foundation** Operator.
4. Click **Install**.
5. Set the following options on the **Install Operator** page:
    a. Update Channel as **stable-4.22**.
    b. Installation Mode as **A specific namespace on the cluster**.
    c. Installed Namespace as **Operator recommended namespace openshift-storage**. If Namespace **openshift-storage** does not exist, it is created during the operator installation.
    d. Select Approval Strategy as **Automatic** or **Manual**.
    If you select **Automatic** updates, then the Operator Lifecycle Manager (OLM) automatically upgrades the running instance of your Operator without any intervention.

    If you select **Manual** updates, then the OLM creates an update request. As a cluster administrator, you must then manually approve that update request to update the Operator to a newer version.
    e. Ensure that the **Enable** option is selected for the **Console plugin**.
    f. Click **Install**.

Verification

- After the operator is successfully installed, the web console automatically reloads to apply the changes. During this process, a temporary error message might appear on the page and this is expected and disappears after the refresh completes.
- In the Web Console:
    - Navigate to Installed Operators and verify that the **OpenShift Data Foundation** Operator shows a green tick indicating successful installation.
    - Navigate to **Storage** and verify if the **Data Foundation** dashboard is available.

2.2. ENABLING CLUSTER-WIDE ENCRYPTION WITH KMS USING THE TOKEN AUTHENTICATION METHOD

You can enable the key value backend path and policy in the vault for token authentication.

Prerequisites

- Administrator access to the vault.
- A valid Red Hat OpenShift Data Foundation Advanced subscription. For more information, see the knowledgebase article on OpenShift Data Foundation subscriptions.
- Carefully, select a unique path name as the backend **path** that follows the naming convention since you cannot change it later.

Procedure

1. Enable the Key/Value (KV) backend path in the vault.
   For vault KV secret engine API, version 1:

   ```
   $ vault secrets enable -path=odf kv
   ```

   For vault KV secret engine API, version 2:

   ```
   $ vault secrets enable -path=odf kv-v2
   ```

2. Create a policy to restrict the users to perform a write or delete operation on the secret:

   ```
   echo '
   path "odf/*" {
     capabilities = ["create", "read", "update", "delete", "list"]
   }
   path "sys/mounts" {
   capabilities = ["read"]
   }'| vault policy write odf -
   ```

3. Create a token that matches the above policy:

   ```
   $ vault token create -policy=odf -format json
   ```

2.3. ENABLING CLUSTER-WIDE ENCRYPTION WITH KMS USING THE KUBERNETES AUTHENTICATION METHOD

You can enable the Kubernetes authentication method for cluster-wide encryption using the Key Management System (KMS).

Prerequisites

- Administrator access to Vault.
- A valid Red Hat OpenShift Data Foundation Advanced subscription. For more information, see the knowledgebase article on OpenShift Data Foundation subscriptions.
- The OpenShift Data Foundation operator must be installed from the Software Catalog.
- Select a unique path name as the backend **path** that follows the naming convention carefully. You cannot change this path name later.

Procedure

1. Create a service account:

   ```
   $ oc -n openshift-storage create serviceaccount <serviceaccount_name>
   ```

   where, ***<serviceaccount_name>*** specifies the name of the service account.

   For example:

   ```
   $ oc -n openshift-storage create serviceaccount odf-vault-auth
   ```

2. Create **clusterrolebindings** and **clusterroles**:

```
$ oc -n openshift-storage create clusterrolebinding vault-tokenreview-binding --
clusterrole=system:auth-delegator --serviceaccount=openshift-
storage:_<serviceaccount_name>_
```

For example:

```
$ oc -n openshift-storage create clusterrolebinding vault-tokenreview-binding --
clusterrole=system:auth-delegator --serviceaccount=openshift-storage:odf-vault-auth
```

3. Create a secret for the **serviceaccount** token and CA certificate.

```
$ cat <<EOF | oc create -f -
apiVersion: v1
kind: Secret
metadata:
  name: odf-vault-auth-token
  namespace: openshift-storage
  annotations:
    kubernetes.io/service-account.name: <serviceaccount_name>
type: kubernetes.io/service-account-token
data: {}
EOF
```

where, ***<serviceaccount_name>*** is the service account created in the earlier step.

4. Get the token and the CA certificate from the secret.

```
$ SA_JWT_TOKEN=$(oc -n openshift-storage get secret odf-vault-auth-token -o jsonpath="
{.data['token']}" | base64 --decode; echo)
$ SA_CA_CRT=$(oc -n openshift-storage get secret odf-vault-auth-token -o jsonpath="
{.data['ca\.crt']}" | base64 --decode; echo)
```

5. Retrieve the OCP cluster endpoint.

```
$ OCP_HOST=$(oc config view --minify --flatten -o jsonpath="{.clusters[0].cluster.server}")
```

6. Fetch the service account issuer:

```
$ oc proxy &
$ proxy_pid=$!
$ issuer="$( curl --silent http://127.0.0.1:8001/.well-known/openid-configuration | jq -r
.issuer)"
$ AUDIENCE=$(echo '{"apiVersion": "authentication.k8s.io/v1", "kind": "TokenRequest"}' |
kubectl create f --raw /api/v1/namespaces/default/serviceaccounts/default/token | jq -r
'.spec.audiences' | jq -r .[0])
$ kill $proxy_pid
```

Or, for Vault version 1.21.0 and higher:

```
$ oc proxy &
$ proxy_pid=$!
$ ISSUER=$(echo '{"apiVersion": "authentication.k8s.io/v1", "kind": "TokenRequest"}' \   |
```

```
kubectl create f --raw /api/v1/namespaces/default/serviceaccounts/default/token \   | jq -r
'.status.token' \   | cut -d . -f2 \   | base64 -d \   | jq '.iss')
$ AUDIENCE=$(echo '{"apiVersion": "authentication.k8s.io/v1", "kind": "TokenRequest"}' |
kubectl create f --raw /api/v1/namespaces/default/serviceaccounts/default/token | jq -r
'.spec.audiences' | jq -r .[0])
$ kill $proxy_pid
```

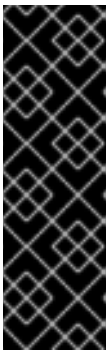
7. Use the information collected in the previous step to setup the Kubernetes authentication method in Vault:

```
$ vault auth enable kubernetes
```

```
$ vault write auth/kubernetes/config \
        token_reviewer_jwt="$SA_JWT_TOKEN" \
        kubernetes_host="$OCP_HOST" \
        kubernetes_ca_cert="$SA_CA_CRT" \
        issuer="$issuer"
```

Or, for Vault version 1.21.0 and higher:

```
$ oc exec -n vault -ti vault-0 – \
   vault write auth/kubernetes/config \
   token_reviewer_jwt="$SA_JWT_TOKEN" \
   kubernetes_host="$OCP_HOST" \
   kubernetes_ca_cert="$SA_CA_CRT" \
   issuer="$ISSUER"\
   token_audience="$AUDIENCE"
```



IMPORTANT

To configure the Kubernetes authentication method in Vault when the issuer is empty:

```
$ vault write auth/kubernetes/config \
        token_reviewer_jwt="$SA_JWT_TOKEN" \
        kubernetes_host="$OCP_HOST" \
        kubernetes_ca_cert="$SA_CA_CRT"
```

8. Enable the Key/Value (KV) backend path in Vault.
For Vault KV secret engine API, version 1:

```
$ vault secrets enable -path=odf kv
```

For Vault KV secret engine API, version 2:

```
$ vault secrets enable -path=odf kv-v2
```

9. Create a policy to restrict the users to perform a **write** or **delete** operation on the secret:

```
echo '
path "odf/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
```

```
}
path "sys/mounts" {
capabilities = ["read"]
}'| vault policy write odf -
```

10. Generate the roles:

```
$ vault write auth/kubernetes/role/odf-rook-ceph-op \
      bound_service_account_names=rook-ceph-system,rook-ceph-osd,noobaa \
      bound_service_account_namespaces=openshift-storage \
      policies=odf \
      ttl=1440h
```

The role **odf-rook-ceph-op** is later used while you configure the KMS connection details during the creation of the storage system.

```
$ vault write auth/kubernetes/role/odf-rook-ceph-osd \
      bound_service_account_names=rook-ceph-osd \
      bound_service_account_namespaces=openshift-storage \
      policies=odf \
      ttl=1440h
```

2.3.1. Enabling and disabling key rotation when using KMS

Security common practices require periodic encryption of key rotation. You can enable or disable key rotation when using KMS.

2.3.1.1. Enforcing precedence for key rotation

Configure key rotation precedence to control whether the rotation schedule is read from the PVC annotation or the storage class in OpenShift Data Foundation.

In key rotation, precedence refers to the order in which the system checks for scheduled annotations. In OpenShift Data Foundation, the default precedence is set to storage class (recommended). This means the system reads annotations only from the storage class.

However, if you want the system to check the persistent volume claim (PVC) first and then fall back to the storage class, you can configure this behavior by setting the **schedule-precedence** to PVC in the CSI-addons ConfigMap.

Procedure

1. Define the **ConfigMap** for csi-addons to set PVC precedence:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: csi-addons-config
  namespace: openshift-storage
data:
  "schedule-precedence": "pvc"
```

2. Restart the **csi-addons** operator pod for the changes to take effect:

```
oc delete po -n openshift-storage -l "app.kubernetes.io/name=csi-addons"
```

2.3.1.2. Enabling key rotation

Enable key rotation to automatically re-encrypt persistent volume data at a scheduled interval using the CSI Addons framework in OpenShift Data Foundation.

To enable key rotation, add the annotation **keyrotation.csiaddons.openshift.io/schedule: <value>** to a **PersistentVolumeClaim**, **Namespace**, or **StorageClass** (in decreasing order of precedence).



IMPORTANT

Key rotation is only supported for RBD backed volumes.

2.3.1.2.1. Annotating a namespace for key rotation

Add the key rotation annotation to a namespace so that all RBD-backed persistent volumes in that namespace are re-encrypted on the configured schedule.

Prerequisites

- Key rotation is only supported for RBD backed volumes.

Procedure

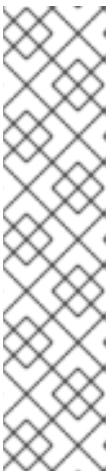
1. Verify the namespace exists:

```
$ oc get namespace default
NAME      STATUS   AGE
default   Active   5d2h
```

2. Annotate the namespace with the desired rotation schedule:

```
$ oc annotate namespace default "keyrotation.csiaddons.openshift.io/schedule=@weekly"
namespace/default annotated
```

<value> can be **@hourly**, **@daily**, **@weekly**, **@monthly**, or **@yearly**. If **<value>** is empty, the default is **@weekly**.



NOTE

- When any schedule for key rotation is used, including **@daily**, **@weekly**, **@monthly**, and cron expressions such as **0 * * * ***, multiple CronJobs are created. To prevent all Jobs from running simultaneously and overloading high-scale systems, executions are staggered within a 2-hour window from the scheduled start time (e.g., **@weekly** would be 00:00, 00:05, 00:10, and so on).
    - If the configured schedule interval is less than 2 hours, the stagger window is reduced to match the interval. For example, a job running every 30 minutes will be staggered within a 00:00–00:30 window rather than 00:00–02:00.

2.3.1.2.2. Annotating a storage class for key rotation

Add the key rotation annotation to a storage class so that all RBD-backed persistent volumes provisioned from that storage class are re-encrypted on the configured schedule.

Prerequisites

- Key rotation is only supported for RBD backed volumes.

Procedure

1. Verify the storage class exists:

```
$ oc get storageclass rbd-sc
NAME      PROVISIONER        RECLAIMPOLICY   VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION   AGE
rbd-sc    rbd.csi.ceph.com   Delete          Immediate          true                   5d2h
```

2. Annotate the storage class with the desired rotation schedule:

```
$ oc annotate storageclass rbd-sc "keyrotation.csiaddons.openshift.io/schedule=@weekly"
storageclass.storage.k8s.io/rbd-sc annotated
```

<value> can be **@hourly**, **@daily**, **@weekly**, **@monthly**, or **@yearly**. If **<value>** is empty, the default is **@weekly**.

2.3.1.2.3. Annotating a PersistentVolumeClaim for key rotation

Add the key rotation annotation directly to a PersistentVolumeClaim to re-encrypt that volume on the configured schedule, overriding any namespace or storage class annotation.

Prerequisites

- Key rotation is only supported for RBD backed volumes.

Procedure

1. Verify the PersistentVolumeClaim exists:

```
$ oc get pvc data-pvc
NAME      STATUS   VOLUME                                     CAPACITY   ACCESS MODES
STORAGECLASS      AGE
data-pvc  Bound    pvc-f37b8582-4b04-4676-88dd-e1b95c6abf74   1Gi        RWO
default           20h
```

2. Annotate the PVC with the desired rotation schedule:

```
$ oc annotate pvc data-pvc "keyrotation.csiaddons.openshift.io/schedule=@weekly"
persistentvolumeclaim/data-pvc annotated
```

<value> can be **@hourly**, **@daily**, **@weekly**, **@monthly**, or **@yearly**. If **<value>** is empty, the default is **@weekly**.

3. Verify the encryption key rotation cron job was created:

```
$ oc get encryptionkeyrotationcronjobs.csiaddons.openshift.io
NAME                SCHEDULE    SUSPEND   ACTIVE   LASTSCHEDULE   AGE
data-pvc-1642663516   @weekly                                3s
```

4. Optional: Update the schedule by overwriting the annotation:

```
$ oc annotate pvc data-pvc "keyrotation.csiaddons.openshift.io/schedule=*/1 * * * *" --
overwrite=true
persistentvolumeclaim/data-pvc annotated
```

5. Verify the updated cron job schedule:

```
$ oc get encryptionkeyrotationcronjobs.csiaddons.openshift.io
NAME                SCHEDULE    SUSPEND   ACTIVE   LASTSCHEDULE   AGE
data-pvc-1642664617   */1 * * * *                            3s
```

2.3.1.3. Disabling key rotation

Disable automatic key rotation for all PVCs in a storage class or for a specific persistent volume claim in OpenShift Data Foundation.

You can disable key rotation for the following:

- All the persistent volume claims (PVCs) of storage class
- A specific PVC

Disabling key rotation for all PVCs of a storage class

To disable key rotation for all PVCs, update the annotation of the storage class:

```
$ oc get storageclass rbd-sc
NAME      PROVISIONER        RECLAIMPOLICY   VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION   AGE
rbd-sc    rbd.csi.ceph.com   Delete          Immediate           true                   5d2h
```

```
$ oc annotate storageclass rbd-sc "keyrotation.csiaddons.openshift.io/enable: false"
storageclass.storage.k8s.io/rbd-sc annotated
```

Disabling key rotation for a specific persistent volume claim

Procedure

1. Identify the **EncryptionKeyRotationCronJob** CR for the PVC you want to disable key rotation on:

```
$ oc get encryptionkeyrotationcronjob -o jsonpath='{range .items[?
(@.spec.jobTemplate.spec.target.persistentVolumeClaim=="<PVC_NAME>")]}
{.metadata.name}{"\n"}{end}'
```

   Where **<PVC_NAME>** is the name of the PVC that you want to disable.

2. Apply the following to the **EncryptionKeyRotationCronJob** CR from the previous step to disable the key rotation:

   a. Update the **csiaddons.openshift.io/state** annotation from **managed** to **unmanaged**:

   ```
   $ oc annotate encryptionkeyrotationcronjob <encryptionkeyrotationcronjob_name>
   "csiaddons.openshift.io/state=unmanaged" --overwrite=true
   ```

   Where **<encryptionkeyrotationcronjob_name>** is the name of the **EncryptionKeyRotationCronJob** CR.

   b. Add **suspend: true** under the **spec** field:

   ```
   $ oc patch encryptionkeyrotationcronjob <encryptionkeyrotationcronjob_name> -p
   '{"spec": {"suspend": true}}' --type=merge.
   ```

   The key rotation will be disabled for the PVC.

2.4. CREATING OPENSHIFT DATA FOUNDATION CLUSTER

Create an OpenShift Data Foundation cluster after you install the OpenShift Data Foundation operator.

Prerequisites

- The OpenShift Data Foundation operator must be installed from the Software Catalog. For more information, see Installing OpenShift Data Foundation Operator using the Software Catalog.

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation → Storage Systems → Create StorageSystem**.

2. In the **Backing storage** page, select the following:

   a. Select the **Use an existing StorageClass** option.

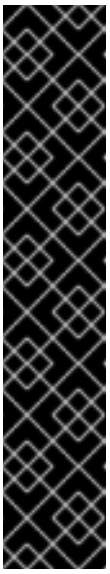
   b. Select the **Storage Class**.
   By default, it is set to **standard**.

   c. Click **Next**.

3. In the **Advanced Settings** page, you can configure the following optional items:

   - **Enable Network Files System**
     Automatically enable NFS for the cluster
   - **Use Ceph RBD as the default StorageClass**
     Sets Ceph RBD as the default StorageClass, eliminating the need to manually annotate a StorageClass.
   - **Set default StorageClass for virtualization**
     Marks the RBD virtualization storage class as the default for KubeVirt VM disks (PVs) after installation.

- **Use external PostgreSQL** [Technology preview]
  Enables the use of an external PostgreSQL instance. This provides a high-availability solution for the Multicloud Object Gateway, where the PostgreSQL pod is a single point of failure.



IMPORTANT

OpenShift Data Foundation ships PostgreSQL images maintained by Red Hat, which are used to store metadata for the Multicloud Object Gateway. This PostgreSQL usage is at the application level.

As a result, OpenShift Data Foundation does not perform database-level optimizations or in-depth insights.

If you have well-maintained and optimized PostgreSQL instance, then it is recommended to use it. OpenShift Data Foundation supports external PostgreSQL instances.

Any PostgreSQL-related issues requiring code changes or deep technical analysis may need to be addressed upstream. This could result in longer resolution times.

  a. Provide the following connection details:
    - **Username**
    - **Password**
    - **Server name** and **Port**
    - **Database name**
  b. Select **Enable TLS/SSL** checkbox to enable encryption for the Postgres server.

- **Enable automatic backup**
  Allows automatic backup for the Multicloud Object Gateway metadata database.
  a. Select the backup frequency: Daily, Weekly, or Monthly.
  b. Specify the number of backups to retain.



NOTE

This option is disabled if the **Use external PostgreSQL** option is selected.

a. Click **Next**.

1. In the **Capacity and nodes** page, provide the necessary information:

b. Select a value for **Requested Capacity** from the dropdown list. It is set to **2 TiB** by default.



NOTE

Once you select the initial storage capacity, cluster expansion is performed only using the selected usable capacity (three times of raw storage).

c. In the **Select Nodes** section, select at least three available nodes.

d. In the **Configure performance** section, select one of the following performance profiles:

- Lean
  Use this in a resource constrained environment with minimum resources that are lower than the recommended. This profile minimizes resource consumption by allocating fewer CPUs and less memory.

- Balanced (default)
  Use this when recommended resources are available. This profile provides a balance between resource consumption and performance for diverse workloads.

- Performance
  Use this in an environment with sufficient resources to get the best performance. This profile is tailored for high performance by allocating ample memory and CPUs to ensure optimal execution of demanding workloads.



NOTE

You have the option to configure the performance profile even after the deployment using the **Configure performance** option from the options menu of the **StorageSystems** tab.



IMPORTANT

Before selecting a resource profile, make sure to check the current availability of resources within the cluster. Opting for a higher resource profile in a cluster with insufficient resources might lead to installation failures.

For more information about resource requirements, see Resource requirement for performance profiles.

e. Optional: Select the **Taint nodes** checkbox to dedicate the selected nodes for OpenShift Data Foundation.
For cloud platforms with multiple availability zones, ensure that the Nodes are spread across different Locations/availability zones.

If the nodes selected do not match the OpenShift Data Foundation cluster requirements of an aggregated 30 CPUs and 72 GiB of RAM, a minimal cluster is deployed. For minimum starting node requirements, see the Resource requirements section in the *Planning* guide.

f. Click **Next**.

1. Optional: In the **Security and network** page, configure the following based on your requirements:

g. To enable encryption, select **Enable data encryption for block and file storage**

i. Select either one or both the encryption levels:

- **Cluster-wide encryption**
  Encrypts the entire cluster (block and file).

- **StorageClass encryption**
  Creates encrypted persistent volume (block only) using encryption enabled storage class.

ii. Optional: Select the **Connect to an external key management service** checkbox. This is optional for cluster-wide encryption.

A. From the **Key Management Service Provider** drop-down list, select one of the following providers and provide the necessary details:

- **Vault**

I. Select an **Authentication Method**.

- **Using Token authentication method**

  - Enter a unique **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Token**.

  - Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:

    - Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.

  - Optional: Enter **TLS Server Name**, **Authentication Path**, and **Vault Enterprise Namespace**.

  - Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key**.

  - Click **Save**.

- **Using Kubernetes authentication method**

  - Enter a unique Vault **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Role** name.

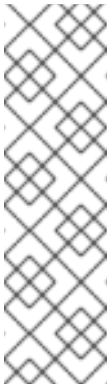
  - Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:

    - Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.

    - Optional: Enter **TLS Server Name**, **Authentication Path**, and **Vault Enterprise Namespace** if applicable.

    - Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key** .

  - Click **Save**.



NOTE

In case you need to enable key rotation for Vault KMS, run the following command in the OpenShift web console after the storage cluster is created:

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type=json -p '[{"op": "add", "path":"/spec/encryption/keyRotation/enable", "value": true}]'
```

- **Thales CipherTrust Manager (using KMIP)**

  I. Enter a unique **Connection Name** for the Key Management service within the project.

  II. In the **Address** and **Port** sections, enter the IP of Thales CipherTrust Manager and the port where the KMIP interface is enabled. For example:

  - **Address**: 123.34.3.2
  - **Port**: 5696

  III. Upload the **Client Certificate**, **CA certificate**, and **Client Private Key**.

  IV. If StorageClass encryption is enabled, enter the Unique Identifier to be used for encryption and decryption generated above.

  V. The **TLS Server** field is optional and used when there is no DNS entry for the KMIP endpoint. For example, **kmip_all_<port>.ciphertrustmanager.local**.

h. To enable in-transit encryption, select **In-transit encryption**.

  i. Select a **Network**.

  ii. Click **Next**.

1. In the **Review and create** page, review the configuration details.
   To modify any configuration settings, click **Back**.
2. Click **Create StorageSystem**.



NOTE

When your deployment has five or more nodes, racks, or rooms, and when there are five or more number of failure domains present in the deployment, you can configure Ceph monitor counts based on the number of racks or zones. An alert is displayed in the notification panel or Alert Center of the OpenShift Web Console to indicate the option to increase the number of Ceph monitor counts. You can use the **Configure** option in the alert to configure the Ceph monitor counts. For more information, see Resolving low Ceph monitor count alert .

Verification

- To verify the final Status of the installed storage cluster:

a. In the OpenShift Web Console, navigate to **Storage → Data Foundation → Storage System → ocs-storagecluster**.

b. Verify that **Status** of **StorageCluster** is **Ready** and has a green tick mark next to it.

- To verify that OpenShift Data Foundation is successfully installed, see Verifying your OpenShift Data Foundation deployment.

Additional resources

- Alerts

CHAPTER 3. VERIFYING OPENSHIFT DATA FOUNDATION DEPLOYMENT

Use this section to verify that OpenShift Data Foundation is deployed correctly.

3.1. VERIFYING THE STATE OF THE PODS

Verify that all required OpenShift Data Foundation pods are in a **Running** state after deployment.

Procedure

1. Click **Workloads → Pods** from the OpenShift Web Console.
2. Select **openshift-storage** from the **Project** drop-down list.

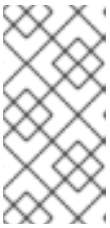


NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

For more information on the expected number of pods for each component and how it varies depending on the number of nodes, see the following table:

1. Set filter for Running and Completed pods to verify that the following pods are in **Running** and **Completed** state:



NOTE

The available pods depend on the cluster configuration. When the cluster is deployed as a standalone Multicloud Object Gateway, the **rook-ceph-operator-*** pods are not available. Similarly, when the cluster is deployed without the Multicloud Object Gateway, **noobaa-*** pods are not available.

Pods corresponding to OpenShift Data Foundation cluster

Component	Corresponding pods

OpenShift Data Foundation Operator	• **ocs-operator-*** (1 pod on any storage node) • **ocs-metrics-exporter-*** (1 pod on any storage node) • **odf-operator-controller-manager-*** (1 pod on any storage node) • **odf-console-*** (1 pod on any storage node) • **csi-addons-controller-manager-*** (1 pod on any storage node) • **ux-backend-server-*** (1 pod on any storage node) • * **ocs-client-operator**-* (1 pod on any storage node) • **ocs-client-operator-console**-* (1 pod on any storage node) • **ocs-provider-server**-* (1 pod on any storage node)
Rook-ceph Operator	**rook-ceph-operator-*** (1 pod on any storage node)
Multicloud Object Gateway	• **noobaa-operator-*** (1 pod on any storage node) • **noobaa-core-*** (1 pod on any storage node) • **noobaa-db-pg-cluster-1** and noobaa-db-pg-cluster-2 (2 instances of MCG DB pod on any storage node) • **noobaa-endpoint-*** (1 pod on any storage node) • **cnpg-controller-manager-*** (1 pod on any storage node)
MON	**rook-ceph-mon-*** (3 pods distributed across storage nodes)

MGR	**rook-ceph-mgr-*** (2 pods on different storage nodes, one active, one standby)
MDS	**rook-ceph-mds-ocs-storagecluster-cephfilesystem-*** (2 pods distributed across storage nodes)
CSI	• **cephfs** ◦ **openshift-storage.cephfs.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.cephfs.csi.ceph.com-nodeplugin-*** (1 pod on any storage nodes) • **nfs** ◦ **openshift-storage.nfs.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.nfs.csi.ceph.com-nodeplugin-*** (1 pod on any storage node) • **rbd** ◦ **openshift-storage.rbd.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.rbd.csi.ceph.com-nodeplugin-*** (1 pod on any storage node)
rook-ceph-crashcollector	**rook-ceph-crashcollector-*** (1 pod on each storage node)
OSD	• **rook-ceph-osd-*** (1 pod for each device) • **rook-ceph-osd-prepare-ocs-*** (1 pod for each device)

ceph-csi-operator	**ceph-csi-controller-manager-*** (1 pod for each device)
odf-blackbox-exporter	**odf-blackbox-exporter-*** (1 pod on any storage node) **NOTE:** This does not apply to clusters without storage, for example, Multicloud Object Gateway-only deployments or external mode deployments.

3.2. VERIFYING THE OPENSHIFT DATA FOUNDATION CLUSTER IS HEALTHY

Verify that the OpenShift Data Foundation cluster is healthy after installation.

Procedure

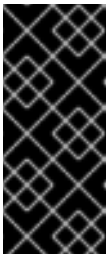
1. In the OpenShift Web Console, click **Storage → Data Foundation**.
2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.
3. In the **Status** card of the **Block and File** tab, verify that the *Storage Cluster* has a green tick.
4. In the **Details** card, verify that the cluster information is displayed.
   For more information on the health of the OpenShift Data Foundation cluster using the **Block and File** dashboard, see Monitoring OpenShift Data Foundation.

3.3. VERIFYING THE MULTICLOUD OBJECT GATEWAY IS HEALTHY

Verify that the Multicloud Object Gateway is healthy after deployment to confirm that the object service and data resiliency are operational and that the NooBaa database has a recent backup.

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation**.
2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.
   a. In the **Status card** of the **Object** tab, verify that both *Object Service* and *Data Resiliency* have a green tick.
   b. In the **Details** card, verify that the MCG information is displayed.
   For more information on the health of the OpenShift Data Foundation cluster using the object service dashboard, see Monitoring OpenShift Data Foundation.



IMPORTANT

To avoid data loss, it is recommended to take a backup of NooBaa DB PVC regularly. If NooBaa DB fails and cannot be recovered, then you can revert to the latest backed-up version. For instructions on backing up your NooBaa DB, follow the steps in the knowledgebase article, Perform a One-Time Backup of the Database for the Multicloud Object Gateway.

3.4. VERIFYING THAT THE SPECIFIC STORAGE CLASSES EXIST

Verify that the expected OpenShift Data Foundation storage classes are present after deployment.

Procedure

1. Click **Storage → Storage Classes**from the left pane of the OpenShift Web Console.
2. Verify that the following storage classes are created with the OpenShift Data Foundation cluster creation:
   - **ocs-storagecluster-ceph-rbd**
   - **ocs-storagecluster-cephfs**
   - **openshift-storage.noobaa.io**

CHAPTER 4. UNINSTALLING OPENSHIFT DATA FOUNDATION IN INTERNAL MODE

4.1. UNINSTALLING OPENSHIFT DATA FOUNDATION IN INTERNAL MODE

Uninstalling OpenShift Data Foundation in internal mode permanently removes the storage cluster, all associated PVCs, and the operator.

Procedure

1. To uninstall OpenShift Data Foundation in internal mode, refer to the knowledgebase article on Uninstalling OpenShift Data Foundation.

CHAPTER 5. DEPLOYING OPENSHIFT DATA FOUNDATION ON RED HAT OPENSTACK PLATFORM IN EXTERNAL MODE

Red Hat OpenShift Data Foundation can use an externally hosted Red Hat Ceph Storage (RHCS) cluster as the storage provider on Red Hat OpenStack Platform. See *Planning your deployment* for more information.

For instructions regarding how to install a RHCS cluster, see the *installation guide*.

Follow these steps to deploy OpenShift Data Foundation in external mode:

1. Install the OpenShift Data Foundation Operator.
2. Create the OpenShift Data Foundation Cluster.

5.1. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR

You can install Red Hat OpenShift Data Foundation Operator using the Red Hat OpenShift Container Platform Software Catalog.

Prerequisites

- Access to an OpenShift Container Platform cluster using an account with **cluster-admin** and operator installation permissions.
- You must have at least three worker or infrastructure nodes in the Red Hat OpenShift Container Platform cluster.
- For additional resource requirements, see the Planning your deployment guide.



IMPORTANT

- When you need to override the cluster-wide default node selector for OpenShift Data Foundation, you can use the following command to specify a blank node selector for the **openshift-storage** namespace (create **openshift-storage** namespace in this case):

```
$ oc annotate namespace openshift-storage openshift.io/node-selector=
```

- Taint a node as **infra** to ensure only Red Hat OpenShift Data Foundation resources are scheduled on that node. This helps you save on subscription costs. For more information, see the *How to use dedicated worker nodes for Red Hat OpenShift Data Foundation* section in the Managing and Allocating Storage Resources guide.

Procedure

1. Log in to the OpenShift Web Console.
2. Click **Ecosystem → Software Catalog**

3. Scroll or type **OpenShift Data Foundation** into the **Filter by keyword** box to find the **OpenShift Data Foundation** Operator.
4. Click **Install**.
5. Set the following options on the **Install Operator** page:
    a. Update Channel as **stable-4.22**.
    b. Installation Mode as **A specific namespace on the cluster**.
    c. Installed Namespace as **Operator recommended namespace openshift-storage**. If Namespace **openshift-storage** does not exist, it is created during the operator installation.
    d. Select Approval Strategy as **Automatic** or **Manual**.
    If you select **Automatic** updates, then the Operator Lifecycle Manager (OLM) automatically upgrades the running instance of your Operator without any intervention.

    If you select **Manual** updates, then the OLM creates an update request. As a cluster administrator, you must then manually approve that update request to update the Operator to a newer version.
    e. Ensure that the **Enable** option is selected for the **Console plugin**.
    f. Click **Install**.

Verification

- After the operator is successfully installed, the web console automatically reloads to apply the changes. During this process, a temporary error message might appear on the page and this is expected and disappears after the refresh completes.
- In the Web Console:
    - Navigate to Installed Operators and verify that the **OpenShift Data Foundation** Operator shows a green tick indicating successful installation.
    - Navigate to **Storage** and verify if the **Data Foundation** dashboard is available.

5.2. CREATING AN OPENSHIFT DATA FOUNDATION CLUSTER FOR EXTERNAL CEPH STORAGE SYSTEM

Create an OpenShift Data Foundation cluster service in external mode by importing the external Ceph cluster credentials, enabling OpenShift workloads to consume storage from the existing Ceph deployment.

You need to create a new OpenShift Data Foundation cluster after you install OpenShift Data Foundation operator on OpenShift Container Platform deployed on Red Hat OpenStack platform.

Prerequisites

- A valid Red Hat OpenShift Data Foundation Advanced subscription. To know how subscriptions for OpenShift Data Foundation work, see knowledgebase article on OpenShift Data Foundation subscriptions.

- Ensure the OpenShift Container Platform version is 4.22 or above before deploying OpenShift Data Foundation 4.22.
- OpenShift Data Foundation operator must be installed. For more information, see Installing OpenShift Data Foundation Operator using the Software Catalog.
- To check the supportability and interoperability of Red Hat Ceph Storage (RHCS) with Red Hat OpenShift Data Foundation in external mode, go to the lab Red Hat OpenShift Data Foundation Supportability and Interoperability Checker.
  - Select **Service Type** as **ODF as Self-Managed Service**.
  - Select appropriate **Version** from the drop down.
  - On the Versions tab, click the **Supported RHCS versions in the External Mode**tab.
- Red Hat Ceph Storage must have Ceph Dashboard installed and configured. For more information, see Ceph Dashboard installation and access .
- It is recommended that the external Red Hat Ceph Storage cluster has the PG Autoscaler enabled. For more information, see The placement group autoscaler section in the Red Hat Ceph Storage documentation.

Procedure

1. In the OpenShift Web Console, click **Storage → External Systems**.
2. Click **Connect external systems**.
3. Select **Red Hat/IBM Ceph Cluster** and click **Next**.
4. In the Connect Red Hat Ceph storage page, provide the necessary information:
   a. Click on the **Download Script** link to download the python script for extracting Ceph cluster details.
   b. For extracting the Red Hat Ceph Storage (RHCS) cluster details, contact the RHCS administrator to run the downloaded python script on a Red Hat Ceph Storage node with the **admin key**.
      i. Run the following command on the RHCS node to view the list of available arguments:

```
# python3 ceph-external-cluster-details-exporter.py --help
```



IMPORTANT

Use **python** instead of **python3** if the Red Hat Ceph Storage 4.x cluster is deployed on Red Hat Enterprise Linux 7.x (RHEL 7.x) cluster.

You can also run the script from inside a MON container (containerized deployment) or from a MON node (RPM deployment).



NOTE

Use the **yum install cephadm** command and then the **cephadm** command to deploy your RHCS cluster using containers. You must pull the RHCS container images using the **cephadm** command, rather than using **yum** for installing the Ceph packages onto nodes. For more information, see RHCS product documentation.

ii. To retrieve the external cluster details from the RHCS cluster, choose one of the following two options, either a configuration file or command-line flags.

A. **Configuration file**
Use **config-file** flag. This stores the parameters used during deployment.

In new deployments, you can save the parameters used during deployment in a configuration file. This file can then be used during upgrade to preserve the parameters as well as adding any additional parameters. Use **config-file** to set the path to the configuration file.

Example configuration file saved in /**config.ini**:

```
[Configurations]
format = bash
cephfs-filesystem-name = <filesystem-name>
rbd-data-pool-name = <pool_name>
...
```

Set the path to the **config.ini** file using **config-file**:

```
# python3 ceph-external-cluster-details-exporter.py --config-file /config.ini
```

B. **Command-line flags**
Retrieve the external cluster details from the RHCS cluster, and pass the parameters for your deployment.

```
# python3 ceph-external-cluster-details-exporter.py \
--rbd-data-pool-name <rbd block pool name>  [optional arguments]
```

For example:

```
# python3 ceph-external-cluster-details-exporter.py --rbd-data-pool-name ceph-rbd --monitoring-endpoint xxx.xxx.xxx.xxx --monitoring-endpoint-port xxxx --rgw-endpoint xxx.xxx.xxx.xxx:xxxx --run-as-user client.ocs
```

RBD parameters

rbd-data-pool-name

A mandatory parameter that is used for providing block storage in OpenShift Data Foundation.

rados-namespace

Divides an RBD data pool into separate logical namespaces, used for creating RBD PVC in a **radosNamespace**. Flags required with **rados-namespace** are **restricted-auth-permission** and **k8s-cluster-name**.

rbd-metadata-ec-pool-name

(Optional) The name of the erasure coded RBD metadata pool.

RGW parameters

rgw-endpoint

(Optional) This parameter is required only if the object storage is to be provisioned through Ceph Rados Gateway for OpenShift Data Foundation. Provide the endpoint in the following format: **<ip_address>:<port>**



NOTE

A fully-qualified domain name (FQDN) is also supported in the format **<FQDN>:<PORT>**.

rgw-pool-prefix

(Optional) The prefix of the RGW pools. If not specified, the default prefix is **default**.

rgw-tls-cert-path

(Optional) The file path of the RADOS Gateway endpoint TLS certificate.

- To provide the TLS certificate and RGW endpoint details to the helper script, **ceph-external-cluster-details-exporter.py**, run the following command:

```
# python3 ceph-external-clustergw-endpoint r-details-exporter.py --rbd-data-pool-name <rbd block pool name> --rgw-endpoint <ip_address>:<port> --rgw-tls-cert-path <file path containing cert>
```

This creates a resource to create a Ceph Object Store CR such as Kubernetes Secret containing the TLS certificate. All the intermediate certificates including private keys need to be stored in the certificate file.

rgw-skip-tls

(Optional) This parameter ignores the TLS certification validation when a self-signed certificate is provided (not recommended).

Monitoring parameters

monitoring-endpoint

(Optional) This parameter accepts comma-separated list of IP addresses of active and standby **mgrs** reachable from the OpenShift Container Platform cluster. If not provided, the value is automatically populated.

monitoring-endpoint-port

(Optional) It is the port associated with the **ceph-mgr** Prometheus exporter specified by **--monitoring-endpoint**. If not provided, the value is automatically populated.

Ceph parameters

ceph-conf

(Optional) The name of the Ceph configuration file.

run-as-user

(Optional) This parameter is used for providing name for the Ceph user which is created by the script. If this parameter is not specified, a default user name **client.healthchecker** is created. The permissions for the new user is set as:

- caps: [mgr] allow command config
- caps: [mon] allow r, allow command quorum_status, allow command version
- caps: [osd] allow rwx pool=**RGW_POOL_PREFIX.rgw.meta**, allow r pool=**.rgw.root**, allow rw pool=**RGW_POOL_PREFIX.rgw.control**, allow rx pool=**RGW_POOL_PREFIX.rgw.log**, allow x pool=**RGW_POOL_PREFIX.rgw.buckets.index**

CephFS parameters

cephfs-metadata-pool-name

(Optional) The name of the CephFS metadata pool.

cephfs-data-pool-name

(Optional) The name of the CephFS data pool.

cephfs-filesystem-name

(Optional) The name of the CephFS filesystem.

Output parameters

dry-run

(Optional) This parameter helps to print the executed commands without running them.

output

(Optional) The file where the output is required to be stored.

Multicluster parameters

k8s-cluster-name

(Optional) Kubernetes cluster name.

cluster-name

(Optional) The Ceph cluster name.

restricted-auth-permission

(Optional) This parameter restricts **cephCSIKeyrings** auth permissions to specific pools and clusters. Mandatory flags that need to be set with this are **rbd-data-pool-name** and **cluster-name**. You can also pass the **cephfs-filesystem-name** flag if there is CephFS user restriction so that permission is restricted to a particular CephFS filesystem.



NOTE

This parameter must be applied only for the new deployments. To restrict **csi-users** per pool and per cluster, you need to create new **csi-users** and new secrets for those **csi-users**.

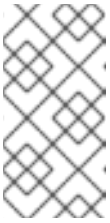
Example with restricted auth permission:

```
# python3 /etc/ceph/create-external-cluster-resources.py --cephfs-filesystem-name myfs --rbd-data-pool-name replicapool --cluster-name rookStorage --restricted-auth-permission true
```

Example of JSON output generated using the python script:

```
[{"name": "rook-ceph-mon-endpoints", "kind": "ConfigMap", "data": {"data": "xxx.xxx.xxx.xxx:xxxx", "maxMonId": "0", "mapping": "{}"}}, {"name": "rook-ceph-mon", "kind": "Secret", "data": {"admin-secret": "admin-secret", "fsid": "<fs-id>", "mon-secret": "mon-secret"}}, {"name": "rook-ceph-operator-creds", "kind": "Secret", "data": {"userID": "<user-id>", "userKey": "<user-key>"}}, {"name": "rook-csi-rbd-node", "kind": "Secret", "data": {"userID": "csi-rbd-node", "userKey": "<user-key>"}}, {"name": "ceph-rbd", "kind": "StorageClass", "data": {"pool": "<pool>"}}, {"name": "monitoring-endpoint", "kind": "CephCluster", "data": {"MonitoringEndpoint": "xxx.xxx.xxx.xxx", "MonitoringPort": "xxxx"}}, {"name": "rook-ceph-dashboard-link", "kind": "Secret", "data": {"userID": "ceph-dashboard-link", "userKey": "<user-key>"}}, {"name": "rook-csi-rbd-provisioner", "kind": "Secret", "data": {"userID": "csi-rbd-provisioner", "userKey": "<user-key>"}}, {"name": "rook-csi-cephfs-provisioner", "kind": "Secret", "data": {"adminID": "csi-cephfs-provisioner", "adminKey": "<admin-key>"}}, {"name": "rook-csi-cephfs-node", "kind": "Secret", "data": {"adminID": "csi-cephfs-node", "adminKey": "<admin-key>"}}, {"name": "cephfs", "kind": "StorageClass", "data": {"fsName": "cephfs", "pool": "cephfs_data"}}, {"name": "ceph-rgw", "kind": "StorageClass", "data": {"endpoint": "xxx.xxx.xxx.xxx:xxxx", "poolPrefix": "default"}}, {"name": "rgw-admin-ops-user", "kind": "Secret", "data": {"accessKey": "<access-key>", "secretKey": "<secret-key>"}}]
```

i. Save the JSON output to a file with **.json** extension



NOTE

For OpenShift Data Foundation to work seamlessly, ensure that the parameters (RGW endpoint, CephFS details, RBD pool, and so on) to be uploaded using the JSON file remain unchanged on the RHCS external cluster after the storage cluster creation.

ii. Run the command when there is a multi-tenant deployment in which the RHCS cluster is already connected to OpenShift Data Foundation deployment with a lower version.

```
# python3 ceph-external-cluster-details-exporter.py --upgrade
```

a. Click **Browse** to select and upload the JSON file.
The content of the JSON file is populated and displayed in the text box.

b. (Optional) Select **In-transit encryption** to enable encryption for entire Ceph traffic including data using Ceh **msgrv2**.



NOTE

Verify that the external Red Hat Ceph Storage (RHCS) cluster is configured for in-transit encryption if required. Follow the documentation for detailed configuration steps before enabling in-transit encryption.

c. (Optional) Select the advanced configuration options that apply to your environment:

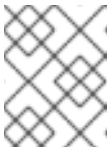
iii. **Use Ceph RBD as the default StorageClass**– Sets the RBD StorageClass as the cluster default.

iv. **Use external PostgreSQL** *(Technology preview)* – Uses an external PostgreSQL instance instead of the in-cluster database.
Additional advanced options may be available in the UI. Select only those that are required for the deployment policy and environment.

v. **Enable automatic backup**
Allows automatic backup for the Multicloud Object Gateway metadata database.

A. Select the backup frequency: Daily, Weekly, or Monthly.

B. Specify the number of backups to retain.



NOTE

This option is disabled if the **Use external PostgreSQL** option is selected.

5. Review the selections and click **Connect**.

Verification

To verify the final Status of the installed storage cluster:

1. In the OpenShift Web Console, navigate to **Storage → Data Foundation → Storage System → ocs-external-storagecluster**.
2. Verify that **Status** of **StorageCluster** is **Ready** and has a green tick.
3. To verify that OpenShift Data Foundation, pods and StorageClass are successfully installed, see Verifying your external mode OpenShift Data Foundation installation for external Ceph storage system.

5.3. CONFIGURING TOPOLOGY AWARENESS FOR EXTERNAL MODE

Topology awareness ensures that data for persistent volumes (PVs) is provisioned from Ceph OSDs that belong to the same failure domain as the OpenShift node where the application pod runs.

Prerequisites

- You have installed OpenShift Data Foundation in external mode.
- You have access to the external Red Hat Ceph Storage cluster with administrative privileges.
- You know the Ceph failure domain that you want to use for topology-aware provisioning.
- The OpenShift worker nodes that host the application workloads have labels that map to the Ceph failure domain values.

Procedure

1. Identify the Ceph failure domain and the corresponding OpenShift nodes.

a. Check the Ceph failure domain:

```
$ ceph osd tree
```

Example output:

```
ID  CLASS  WEIGHT   TYPE NAME       STATUS  REWEIGHT  PRI-AFF
...
-9         0.50000      host osd-0
 3    hdd  0.25000          osd.3       up   1.00000  1.00000
...
```

In this example, the failure domain is **host**, and the device class is **hdd**.

b. Identify the OpenShift worker nodes and map them to the Ceph failure domains:

```
$ oc get nodes
```

Example output:

```
NAME           STATUS   ROLES    AGE    VERSION
compute-0       Ready    worker   2d23h   v1.34.1
compute-1       Ready    worker   2d23h   v1.34.1
compute-2       Ready    worker   2d23h   v1.34.1
```

c. Verify the node labels that you want to use for topology awareness:

```
$ oc get nodes --show-labels
```

In this example, the label **topology.kubernetes.io/hostname** and its values are chosen, for example, **compute-0**, **compute-1**, and **compute-2**, when configuring topology-aware external cluster resources.

2. Create separate block pools for each failure domain.
In this example, a replica-3 pool is created for each failure domain.

```
$ ceph osd pool create pool-1-data 32 32 replicated
$ ceph osd pool set pool-1-data size 3
$ ceph osd pool create pool-2-data 32 32 replicated
$ ceph osd pool set pool-2-data size 3
$ ceph osd pool create pool-3-data 32 32 replicated
$ ceph osd pool set pool-3-data size 3
```

Verify the pools:

```
$ ceph osd pool ls detail
```

3. Generate the external cluster resources with topology-aware settings.
Run the following command on the Ceph client:

```
$ python3 /etc/rook-external/create-external-cluster-resources.py \
  --rbd-data-pool-name rbd \
  --topology-pools pool-1-data,pool-2-data,pool-3-data \
```

```
--topology-failure-domain-label host \
--topology-failure-domain-values compute-0,compute-1,compute-2
```

where

--rbd-data-pool-name

Specifies an existing RBD pool.

--topology-pools

Specifies the topology-constrained pools created for each failure domain.

--topology-failure-domain-label

Specifies the Ceph failure domain label, for example, **host**.

--topology-failure-domain-values

Specifies the corresponding OpenShift failure domain values, for example, **compute-0,compute-1,compute-2**.

4. Upload the generated JSON file when creating the external cluster in the OpenShift web console.

5. Verify that topology awareness is configured.

   a. Verify that the operator configuration contains the topology domain label:

   ```
   $ oc get operatorconfigs.csi.ceph.io -o yaml | grep topology -A3
   ```

   Expected output:

   ```
   topology:
     domainLabels:
     - kubernetes.io/hostname
   ```

   b. Verify that the topology-aware storage class contains the topology parameters:

   ```
   $ oc get sc ocs-external-storagecluster-ceph-non-resilient-rbd -o yaml
   ```

   Example output:

   ```
   parameters:
     topologyConstrainedPools: |-
       [
         {
           "poolName": "pool-1-data",
           "domainSegments": [
             {
               "domainLabel": "hostname",
               "value": "compute-0"
             }
           ]
         },
         {
           "poolName": "pool-2-data",
           "domainSegments": [
             {
   ```

```
            "domainLabel": "hostname",
            "value": "compute-1"
          }
        ]
      },
      {
        "poolName": "pool-3-data",
        "domainSegments": [
          {
            "domainLabel": "hostname",
            "value": "compute-2"
          }
        ]
      }
    ]
  topologyFailureDomainLabel: host
  topologyFailureDomainValues: compute-0,compute-1,compute-2
  topologyPools: pool-1-data,pool-2-data,pool-3-data
```

6. Test topology-aware provisioning by creating a persistent volume claim (PVC) and a pod. Apply the following YAML:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: topo-verify-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: ocs-external-storagecluster-ceph-non-resilient-rbd
---
apiVersion: v1
kind: Pod
metadata:
  name: topo-verify-pod
spec:
  containers:
  - name: test-container
    image: busybox
    command: [ "sh", "-c", "sleep 3600" ]
    volumeMounts:
    - mountPath: /data
      name: my-volume
  volumes:
  - name: my-volume
    persistentVolumeClaim:
      claimName: topo-verify-pvc
```

The **WaitForFirstConsumer** volume binding mode ensures that the PV is provisioned only after the pod is scheduled, so topology constraints can be applied.

7. Verify data placement in the expected failure domain.

a. Get the image name for the provisioned PV:

```
$ kubectl get pv $(kubectl get pvc topo-verify-pvc -o jsonpath='{.spec.volumeName}') -o
jsonpath='{.spec.csi.volumeAttributes.imageName}'
```

b. Check usage from the Ceph side:

```
$ rbd du pool-1-data/<image_name>
```

c. Identify the OSDs that store the image data:

```
$ ceph osd map pool-1-data rbd_header.<image_name>
```

Example output:

```
... -> pg 14.1ed52b0 (14.10) -> up ([4,5,6], p4) acting ([4,5,6], p4)
```

d. Verify that the OSDs belong to the expected failure domains:

```
$ ceph osd tree | grep -E "osd.4|osd.5|osd.6"
```

Example output:

```
5   hdd  0.25000        osd.5      up   1.00000  1.00000
6   hdd  0.25000        osd.6      up   1.00000  1.00000
4   hdd  0.25000        osd.4      up   1.00000  1.00000
```

5.4. VERIFYING YOUR OPENSHIFT DATA FOUNDATION INSTALLATION FOR EXTERNAL MODE

Verify the OpenShift Data Foundation external mode installation is deployed correctly.

5.4.1. Verifying the state of the pods

1. Click **Workloads → Pods** from the left pane of the OpenShift Web Console.
2. Select **openshift-storage** from the **Project** drop-down list.



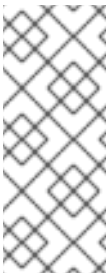
NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

3. Verify that the following pods are in running state:
   Pods corresponding to OpenShift Data Foundation components

Component	Corresponding pods

OpenShift Data Foundation Operator	• **ocs-operator-*** (1 pod on any worker node) • **ocs-metrics-exporter-*** (1 pod on any worker node) • **odf-operator-controller-manager-*** (1 pod on any worker node) • **odf-console-*** (1 pod on any worker node) • **csi-addons-controller-manager-*** (1 pod on any worker node)
Rook-ceph Operator	**rook-ceph-operator-*** (1 pod on any worker node)
Multicloud Object Gateway	• **noobaa-operator-*** (1 pod on any worker node) • **noobaa-core-*** (1 pod on any worker node) • **noobaa-db-pg-cluster-1** and **noobaa-db-pg-cluster-2** (2 instances of MCG DB pod on any storage node) • **noobaa-endpoint-*** (1 pod on any storage node) • **cnpg-controller-manager-*** (1 pod on any storage node)

CSI	• **cephfs** ◦ **openshift-storage.cephfs.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.cephfs.csi.ceph.com-nodeplugin-*** (1 pod on each storage node)  NOTE If an MDS is not deployed in the external cluster, the **openshift-storage.cephfs.csi.ceph.com-ctrlplugin-*** and **openshift-storage.cephfs.csi.ceph.com-nodeplugin-*** pods are not created. • **nfs** ◦ **openshift-storage.nfs.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.nfs.csi.ceph.com-nodeplugin-*** (1 pod on each storage node) • **rbd** ◦ **openshift-storage.rbd.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.rbd.csi.ceph.com-nodeplugin-*** (1 pod on each storage node)

5.4.2. Verifying that the OpenShift Data Foundation cluster is healthy

1. In the OpenShift Web Console, click **Storage → Data Foundation**.
2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.
3. In the **Status** card of the **Block and File** tab, verify that both *Storage Cluster* and *Data Resiliency* have a green tick.
4. In the **Details** card, verify that the cluster information is displayed as follows.

+ Service Name:: OpenShift Data Foundation Cluster Name:: ocs-external-storagecluster Provider:: OpenStack Mode:: External Version:: ocs-operator-4.22.0

5.4.3. Verifying that the Multicloud Object Gateway is healthy

1. In the OpenShift Web Console, click **Storage → Data Foundation**.
2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.
   a. In the **Status** card of the **Object** tab, verify that both *Object Service* and *Data Resiliency* have a green tick.
   b. In the **Details** card, verify that the Multicloud Object Gateway (MCG) information is displayed.

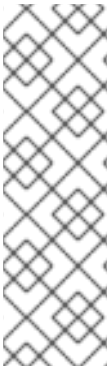


NOTE

The RADOS Object Gateway is only listed in case RADOS Object Gateway endpoint details were included while deploying OpenShift Data Foundation in external mode.

5.4.4. Verifying that the storage classes are created and listed

- Click **Storage → Storage Classes**from the left pane of the OpenShift Web Console.
- Verify that the following storage classes are created with the OpenShift Data Foundation cluster creation:
  - **ocs-external-storagecluster-ceph-rbd**
  - **ocs-external-storagecluster-ceph-rgw**
  - **ocs-external-storagecluster-cephfs**
  - **openshift-storage.noobaa.io**



NOTE

- If MDS is not deployed in the external cluster, **ocs-external-storagecluster-cephfs** storage class will not be created.
- If RGW is not deployed in the external cluster, the **ocs-external-storagecluster-ceph-rgw** storage class will not be created.

For more information regarding MDS and RGW, see the Red Hat Ceph Storage documentation.

Additional resources

- Monitoring OpenShift Data Foundation
- Red Hat Ceph Storage documentation

5.4.5. Verifying that Ceph cluster is connected

Run the following command to verify if the OpenShift Data Foundation cluster is connected to the external Red Hat Ceph Storage cluster.

```
$ oc get cephcluster -n openshift-storage
```

```
NAME                                DATADIRHOSTPATH     MONCOUNT    AGE      PHASE
MESSAGE                  HEALTH
ocs-external-storagecluster-cephcluster                          31m15s   Connected   Cluster
connected successfully   HEALTH_OK
```

5.4.6. Verifying that storage cluster is ready

Run the following command to verify if the storage cluster is ready and the **External** option is set to true.

```
$ oc get storagecluster -n openshift-storage
```

```
NAME                          AGE      PHASE EXTERNAL  CREATED AT            VERSION
ocs-external-storagecluster 31m15s    Ready true      2021-02-29T20:43:04Z    4.22.0
```

5.4.7. Verifying the state of the pods

Verify that all expected OpenShift Data Foundation pods are running in the **openshift-storage** namespace on OpenShift Platform on OpenStack.

Procedure

1. Click **Workloads → Pods** from the left pane of the OpenShift Web Console.
2. Select **openshift-storage** from the **Project** drop-down list.



NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

3. Verify that the following pods are in running state:
   Pods corresponding to OpenShift Data Foundation components

Component	Corresponding pods

OpenShift Data Foundation Operator	• **ocs-operator-*** (1 pod on any worker node) • **ocs-metrics-exporter-*** (1 pod on any worker node) • **odf-operator-controller-manager-*** (1 pod on any worker node) • **odf-console-*** (1 pod on any worker node) • **csi-addons-controller-manager-*** (1 pod on any worker node)
Rook-ceph Operator	**rook-ceph-operator-*** (1 pod on any worker node)
Multicloud Object Gateway	• **noobaa-operator-*** (1 pod on any worker node) • **noobaa-core-*** (1 pod on any worker node) • **noobaa-db-pg-cluster-1** and **noobaa-db-pg-cluster-2** (2 instances of MCG DB pod on any storage node) • **noobaa-endpoint-*** (1 pod on any storage node) • **cnpg-controller-manager-*** (1 pod on any storage node)

CSI	• **cephfs** ◦ **openshift-storage.cephfs.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.cephfs.csi.ceph.com-nodeplugin-*** (1 pod on each storage node) NOTE If an MDS is not deployed in the external cluster, the **openshift-storage.cephfs.csi.ceph.com-ctrlplugin-*** and **openshift-storage.cephfs.csi.ceph.com-nodeplugin-*** pods are not created. • **nfs** ◦ **openshift-storage.nfs.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.nfs.csi.ceph.com-nodeplugin-*** (1 pod on each storage node) • **rbd** ◦ **openshift-storage.rbd.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.rbd.csi.ceph.com-nodeplugin-*** (1 pod on each storage node)

5.4.8. Verifying that the OpenShift Data Foundation cluster is healthy

Verify the health status of the OpenShift Data Foundation storage cluster after deploying in external mode on OpenShift Platform on OpenStack.

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation**.

2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.

3. In the **Status** card of the **Block and File** tab, verify that both *Storage Cluster* and *Data Resiliency* have a green tick.

4. In the **Details** card, verify that the cluster information is displayed as follows.
+ Service Name:: OpenShift Data Foundation Cluster Name:: ocs-external-storagecluster Provider:: OpenStack Mode:: External Version:: ocs-operator-4.22.0

   + For more information on the health of OpenShift Data Foundation cluster using the Block and File dashboard, see the Monitoring OpenShift Data Foundation documentation.

Additional resources

- Monitoring OpenShift Data Foundation

5.4.9. Verifying that the Multicloud Object Gateway is healthy

Verify that the Multicloud Object Gateway is healthy and both Object Service and Data Resiliency show green status after deploying on OpenShift Platform on OpenStack.

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation**.

2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.

   a. In the **Status** card of the **Object** tab, verify that both *Object Service* and *Data Resiliency* have a green tick.

   b. In the **Details** card, verify that the Multicloud Object Gateway (MCG) information is displayed.



NOTE

The RADOS Object Gateway is only listed in case RADOS Object Gateway endpoint details were included while deploying OpenShift Data Foundation in external mode.

For more information on the health of OpenShift Data Foundation cluster using the object dashboard, see the Monitoring OpenShift Data Foundation documentation.

Additional resources

- Monitoring OpenShift Data Foundation

5.4.10. Verifying that the storage classes are created and listed

Verify that the expected OpenShift Data Foundation storage classes are created and listed after deploying in external mode on OpenShift Platform on OpenStack.

Procedure

1. Click **Storage → Storage Classes**from the left pane of the OpenShift Web Console.
2. Verify that the following storage classes are created with the OpenShift Data Foundation cluster creation:
   - **ocs-external-storagecluster-ceph-rbd**
   - **ocs-external-storagecluster-ceph-rgw**
   - **ocs-external-storagecluster-cephfs**
   - **openshift-storage.noobaa.io**



NOTE

- If MDS is not deployed in the external cluster, **ocs-external-storagecluster-cephfs** storage class will not be created.
- If RGW is not deployed in the external cluster, the **ocs-external-storagecluster-ceph-rgw** storage class will not be created.

For more information regarding MDS and RGW, see the Red Hat Ceph Storage documentation.

Additional resources

- Red Hat Ceph Storage documentation

5.4.11. Verifying that Ceph cluster is connected

Verify that the OpenShift Data Foundation cluster is connected to the external Red Hat Ceph Storage cluster in an OpenShift Platform on OpenStack deployment.

Procedure

1. Run the following command to verify if the OpenShift Data Foundation cluster is connected to the external Red Hat Ceph Storage cluster:

```
$ oc get cephcluster -n openshift-storage
```

```
NAME                                 DATADIRHOSTPATH     MONCOUNT   AGE     PHASE
MESSAGE                 HEALTH
ocs-external-storagecluster-cephcluster                              31m15s   Connected   Cluster
connected successfully   HEALTH_OK
```

5.4.12. Verifying that storage cluster is ready

Verify that the external OpenShift Data Foundation storage cluster is ready and the **External** flag is set to true after deployment on OpenShift Platform on OpenStack.

Procedure

1. Run the following command to verify if the storage cluster is ready and the **External** option is set to true:

```
$ oc get storagecluster -n openshift-storage
```

```
NAME                          AGE     PHASE EXTERNAL  CREATED AT              VERSION
ocs-external-storagecluster 31m15s  Ready true      2021-02-29T20:43:04Z  4.22.0
```

CHAPTER 6. UNINSTALLING OPENSHIFT DATA FOUNDATION IN EXTERNAL MODE

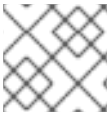
6.1. UNINSTALLING OPENSHIFT DATA FOUNDATION FROM EXTERNAL STORAGE SYSTEM

Use the steps in this section to uninstall OpenShift Data Foundation. Uninstalling OpenShift Data Foundation does not remove the RBD pool from the external cluster, or uninstall the external Red Hat Ceph Storage cluster.

Uninstall Annotations

Annotations on the Storage Cluster are used to change the behavior of the uninstall process. To define the uninstall behavior, the following two annotations have been introduced in the storage cluster:

- **uninstall.ocs.openshift.io/cleanup-policy: delete**
- **uninstall.ocs.openshift.io/mode: graceful**



NOTE

The **uninstall.ocs.openshift.io/cleanup-policy** is not applicable for external mode.

The below table provides information on the different values that can used with these annotations:

uninstall.ocs.openshift.io uninstall annotations descriptions

Annotation	Value	Default	Behavior
cleanup-policy	delete	Yes	Rook cleans up the physical drives and the **DataDirHostPath**
cleanup-policy	retain	No	Rook does **not** clean up the physical drives and the **DataDirHostPath**
mode	graceful	Yes	Rook and NooBaa **pauses** the uninstall process until the PVCs and the OBCs are removed by the administrator/user
mode	forced	No	Rook and NooBaa proceeds with uninstall even if PVCs/OBCs provisioned using Rook and NooBaa exist respectively

You can change the uninstall mode by editing the value of the annotation by using the following commands:

```
$ oc annotate storagecluster ocs-external-storagecluster -n openshift-storage
uninstall.ocs.openshift.io/mode="forced" --overwrite
storagecluster.ocs.openshift.io/ocs-external-storagecluster annotated
```

Prerequisites

- Ensure that the OpenShift Data Foundation cluster is in a healthy state. The uninstall process can fail when some of the pods are not terminated successfully due to insufficient resources or nodes. In case the cluster is in an unhealthy state, contact Red Hat Customer Support before uninstalling OpenShift Data Foundation.
- Ensure that applications are not consuming persistent volume claims (PVCs) or object bucket claims (OBCs) using the storage classes provided by OpenShift Data Foundation.

Procedure

1. Delete the volume snapshots that are using OpenShift Data Foundation.
   a. List the volume snapshots from all the namespaces

      ```
      $ oc get volumesnapshot --all-namespaces
      ```

   b. From the output of the previous command, identify and delete the volume snapshots that are using OpenShift Data Foundation.

      ```
      $ oc delete volumesnapshot <VOLUME-SNAPSHOT-NAME> -n <NAMESPACE>
      ```

2. Delete PVCs and OBCs that are using OpenShift Data Foundation.
   In the default uninstall mode (graceful), the uninstaller waits till all the PVCs and OBCs that use OpenShift Data Foundation are deleted.

   If you wish to delete the Storage Cluster without deleting the PVCs beforehand, you may set the uninstall mode annotation to "forced" and skip this step. Doing so will result in orphan PVCs and OBCs in the system.

   a. Delete OpenShift Container Platform monitoring stack PVCs using OpenShift Data Foundation.
      See Removing monitoring stack from OpenShift Data Foundation

   b. Delete OpenShift Container Platform Registry PVCs using OpenShift Data Foundation.
      Removing OpenShift Container Platform registry from OpenShift Data Foundation

   c. Delete OpenShift Container Platform logging PVCs using OpenShift Data Foundation.
      Removing the cluster logging operator from OpenShift Data Foundation

   d. Delete other PVCs and OBCs provisioned using OpenShift Data Foundation.

      - Given below is a sample script to identify the PVCs and OBCs provisioned using OpenShift Data Foundation. The script ignores the PVCs and OBCs that are used internally by OpenShift Data Foundation.

        ```
        #!/bin/bash
        ```

```
RBD_PROVISIONER="openshift-storage.rbd.csi.ceph.com"
CEPHFS_PROVISIONER="openshift-storage.cephfs.csi.ceph.com"
NOOBAA_PROVISIONER="openshift-storage.noobaa.io/obc"
RGW_PROVISIONER="openshift-storage.ceph.rook.io/bucket"

NOOBAA_DB_PG_1_PVC="noobaa-db-pg-cluster-1"
NOOBAA_DB_PG_2_PVC="noobaa-db-pg-cluster-2"
NOOBAA_BACKINGSTORE_PVC="noobaa-default-backing-store-noobaa-pvc"

# Find all the OCS StorageClasses
OCS_STORAGECLASSES=$(oc get storageclasses | grep -e
"$RBD_PROVISIONER" -e "$CEPHFS_PROVISIONER" -e
"$NOOBAA_PROVISIONER" -e "$RGW_PROVISIONER" | awk '{print $1}')

# List PVCs in each of the StorageClasses
for SC in $OCS_STORAGECLASSES
do
      echo
"=============================================================================
=="
      echo "$SC StorageClass PVCs and OBCs"
      echo
"=============================================================================
=="
      oc get pvc  --all-namespaces --no-headers 2>/dev/null | grep $SC | grep -v -e
"$NOOBAA_DB_PVC" -e "$NOOBAA_BACKINGSTORE_PVC"
      oc get obc  --all-namespaces --no-headers 2>/dev/null | grep $SC
      echo
done
```

- Delete the OBCs.

  ```
  $ oc delete obc <obc name> -n <project name>
  ```

- Delete the PVCs.

  ```
  $ oc delete pvc <pvc name> -n <project-name>
  ```

  Ensure that you have removed any custom backing stores, bucket classes, and so on that are created in the cluster.

3. Delete the Storage Cluster object and wait for the removal of the associated resources.

   ```
   $ oc delete -n openshift-storage storagecluster --all --wait=true
   ```

4. Delete the namespace and wait until the deletion is complete. You will need to switch to another project if **openshift-storage** is the active project.
   For example:

   ```
   $ oc project default
   $ oc delete project openshift-storage --wait=true --timeout=5m
   ```

   The project is deleted if the following command returns a **NotFound** error.

```
$ oc get project openshift-storage
```



NOTE

While uninstalling OpenShift Data Foundation, if the namespace is not deleted completely and remains in **Terminating** state, perform the steps in Troubleshooting and deleting remaining resources during Uninstall to identify objects that are blocking the namespace from being terminated.

5. Confirm all PVs provisioned using OpenShift Data Foundation are deleted. If there is any PV left in the **Released** state, delete it.

```
$ oc get pv
$ oc delete pv <pv name>
```

6. Remove **CustomResourceDefinitions**.

```
$ oc delete crd backingstores.noobaa.io bucketclasses.noobaa.io
cephblockpools.ceph.rook.io cephclusters.ceph.rook.io cephfilesystems.ceph.rook.io
cephnfses.ceph.rook.io cephobjectstores.ceph.rook.io cephobjectstoreusers.ceph.rook.io
noobaas.noobaa.io ocsinitializations.ocs.openshift.io storageclusters.ocs.openshift.io
cephclients.ceph.rook.io cephobjectrealms.ceph.rook.io cephobjectzonegroups.ceph.rook.io
cephobjectzones.ceph.rook.io cephrbdmirrors.ceph.rook.io storagesystems.odf.openshift.io --
wait=true --timeout=5m
```

7. To ensure that OpenShift Data Foundation is uninstalled completely:

   a. In the OpenShift Container Platform Web Console, click **Storage**.

   b. Verify that **OpenShift Data Foundation** no longer appears under Storage.

6.2. REMOVING MONITORING STACK FROM OPENSHIFT DATA FOUNDATION

Remove the OpenShift monitoring stack from OpenShift Data Foundation external mode storage before uninstalling the deployment to reclaim the PVCs used by Prometheus and Alertmanager.

The PVCs that are created as a part of configuring the monitoring stack are in the **openshift-monitoring** namespace.

Prerequisites

- PVCs are configured to use the OpenShift Container Platform monitoring stack. For information, see configuring monitoring stack.

Procedure

1. List the pods and PVCs that are currently running in the **openshift-monitoring** namespace.

```
$ oc get pod,pvc -n openshift-monitoring
NAME                        READY   STATUS    RESTARTS   AGE
pod/alertmanager-main-0     3/3     Running   0          8d
```

```
pod/alertmanager-main-1          3/3     Running   0          8d
pod/alertmanager-main-2          3/3     Running   0          8d
pod/cluster-monitoring-
operator-84457656d-pkrxm         1/1     Running   0          8d
pod/grafana-79ccf6689f-2ll28     2/2     Running   0          8d
pod/kube-state-metrics-
7d86fb966-rvd9w                  3/3     Running   0          8d
pod/node-exporter-25894          2/2     Running   0          8d
pod/node-exporter-4dsd7          2/2     Running   0          8d
pod/node-exporter-6p4zc          2/2     Running   0          8d
pod/node-exporter-jbjvg          2/2     Running   0          8d
pod/node-exporter-jj4t5          2/2     Running   0          6d18h
pod/node-exporter-k856s          2/2     Running   0          6d18h
pod/node-exporter-rf8gn          2/2     Running   0          8d
pod/node-exporter-rmb5m          2/2     Running   0          6d18h
pod/node-exporter-zj7kx          2/2     Running   0          8d
pod/openshift-state-metrics-
59dbd4f654-4clng                 3/3     Running   0          8d
pod/prometheus-adapter-
5df5865596-k8dzn                 1/1     Running   0          7d23h
pod/prometheus-adapter-
5df5865596-n2gj9                 1/1     Running   0          7d23h
pod/prometheus-k8s-0             6/6     Running   1          8d
pod/prometheus-k8s-1             6/6     Running   1          8d
pod/prometheus-operator-
55cfb858c9-c4zd9                 1/1     Running   0          6d21h
pod/telemeter-client-
78fc8fc97d-2rgfp                 3/3     Running   0          8d

NAME                                                STATUS   VOLUME
CAPACITY   ACCESS MODES   STORAGECLASS                AGE
persistentvolumeclaim/my-alertmanager-claim-alertmanager-main-0   Bound    pvc-0d519c4f-
15a5-11ea-baa0-026d231574aa   40Gi       RWO            ocs-external-storagecluster-ceph-rbd
8d
persistentvolumeclaim/my-alertmanager-claim-alertmanager-main-1   Bound    pvc-
0d5a9825-15a5-11ea-baa0-026d231574aa   40Gi       RWO            ocs-external-
storagecluster-ceph-rbd   8d
persistentvolumeclaim/my-alertmanager-claim-alertmanager-main-2   Bound    pvc-
0d6413dc-15a5-11ea-baa0-026d231574aa   40Gi       RWO            ocs-external-
storagecluster-ceph-rbd   8d
persistentvolumeclaim/my-prometheus-claim-prometheus-k8s-0        Bound    pvc-0b7c19b0-
15a5-11ea-baa0-026d231574aa   40Gi       RWO            ocs-external-storagecluster-ceph-rbd
8d
persistentvolumeclaim/my-prometheus-claim-prometheus-k8s-1        Bound    pvc-0b8aed3f-
15a5-11ea-baa0-026d231574aa   40Gi       RWO            ocs-external-storagecluster-ceph-rbd
8d
```

2. Edit the monitoring **configmap**.

```
$ oc -n openshift-monitoring edit configmap cluster-monitoring-config
```

Remove any **config** sections that reference the OpenShift Data Foundation storage classes as shown in the following example and save it.

Before editing

```
.
.
.
apiVersion: v1
data:
  config.yaml: |
    alertmanagerMain:
      volumeClaimTemplate:
        metadata:
          name: my-alertmanager-claim
        spec:
          resources:
            requests:
              storage: 40Gi
          storageClassName: ocs-external-storagecluster-ceph-rbd
    prometheusK8s:
      volumeClaimTemplate:
        metadata:
          name: my-prometheus-claim
        spec:
          resources:
            requests:
              storage: 40Gi
          storageClassName: ocs-external-storagecluster-ceph-rbd
kind: ConfigMap
metadata:
  creationTimestamp: "2019-12-02T07:47:29Z"
  name: cluster-monitoring-config
  namespace: openshift-monitoring
  resourceVersion: "22110"
  selfLink: /api/v1/namespaces/openshift-monitoring/configmaps/cluster-monitoring-config
  uid: fd6d988b-14d7-11ea-84ff-066035b9efa8


.
.
.
```

After editing

```
.
.
.
apiVersion: v1
data:
  config.yaml: |
kind: ConfigMap
metadata:
  creationTimestamp: "2019-11-21T13:07:05Z"
  name: cluster-monitoring-config
  namespace: openshift-monitoring
  resourceVersion: "404352"
  selfLink: /api/v1/namespaces/openshift-monitoring/configmaps/cluster-monitoring-config
  uid: d12c796a-0c5f-11ea-9832-063cd735b81c
.
.
.
```

In this example, **alertmanagerMain** and **prometheusK8s** monitoring components are using the OpenShift Data Foundation PVCs.

3. List the pods consuming the PVC.
   In this example, the **alertmanagerMain** and **prometheusK8s** pods that were consuming the PVCs are in the **Terminating** state. You can delete the PVCs once these pods are no longer using OpenShift Data Foundation PVC.

```
$ oc get pod,pvc -n openshift-monitoring
NAME                                          READY   STATUS      RESTARTS AGE
pod/alertmanager-main-0                       3/3     Terminating   0      10h
pod/alertmanager-main-1                       3/3     Terminating   0      10h
pod/alertmanager-main-2                       3/3     Terminating   0      10h
pod/cluster-monitoring-operator-84cd9df668-zhjfn   1/1     Running       0      18h
pod/grafana-5db6fd97f8-pmtbf                  2/2     Running       0      10h
pod/kube-state-metrics-895899678-z2r9q        3/3     Running       0      10h
pod/node-exporter-4njxv                       2/2     Running       0      18h
pod/node-exporter-b8ckz                       2/2     Running       0      11h
pod/node-exporter-c2vp5                       2/2     Running       0      18h
pod/node-exporter-cq65n                       2/2     Running       0      18h
pod/node-exporter-f5sm7                       2/2     Running       0      11h
pod/node-exporter-f852c                       2/2     Running       0      18h
pod/node-exporter-l9zn7                       2/2     Running       0      11h
pod/node-exporter-ngbs8                       2/2     Running       0      18h
pod/node-exporter-rv4v9                       2/2     Running       0      18h
pod/openshift-state-metrics-77d5f699d8-69q5x      3/3     Running       0      10h
pod/prometheus-adapter-765465b56-4tbxx        1/1     Running       0      10h
pod/prometheus-adapter-765465b56-s2qg2        1/1     Running       0      10h
pod/prometheus-k8s-0                          6/6     Terminating   1      9m47s
pod/prometheus-k8s-1                          6/6     Terminating   1      9m47s
pod/prometheus-operator-cbfd89f9-ldnwc        1/1     Running       0      43m
pod/telemeter-client-7b5ddb4489-2xfpz         3/3     Running       0      10h

NAME                                        STATUS  VOLUME
CAPACITY   ACCESS MODES   STORAGECLASS               AGE
```

```
persistentvolumeclaim/ocs-alertmanager-claim-alertmanager-main-0 Bound pvc-2eb79797-1fed-11ea-93e1-0a88476a6a64 40Gi RWO ocs-external-storagecluster-ceph-rbd 19h
persistentvolumeclaim/ocs-alertmanager-claim-alertmanager-main-1 Bound pvc-2ebeee54-1fed-11ea-93e1-0a88476a6a64 40Gi RWO ocs-external-storagecluster-ceph-rbd 19h
persistentvolumeclaim/ocs-alertmanager-claim-alertmanager-main-2 Bound pvc-2ec6a9cf-1fed-11ea-93e1-0a88476a6a64 40Gi RWO ocs-external-storagecluster-ceph-rbd 19h
persistentvolumeclaim/ocs-prometheus-claim-prometheus-k8s-0 Bound pvc-3162a80c-1fed-11ea-93e1-0a88476a6a64 40Gi RWO ocs-external-storagecluster-ceph-rbd 19h
persistentvolumeclaim/ocs-prometheus-claim-prometheus-k8s-1 Bound pvc-316e99e2-1fed-11ea-93e1-0a88476a6a64 40Gi RWO ocs-external-storagecluster-ceph-rbd 19h
```

4. Delete relevant PVCs. Make sure you delete all the PVCs that are consuming the storage classes.

```
$ oc delete -n openshift-monitoring pvc <pvc-name> --wait=true --timeout=5m
```

6.3. REMOVING OPENSHIFT CONTAINER PLATFORM REGISTRY FROM OPENSHIFT DATA FOUNDATION

Use this section to clean up the OpenShift Container Platform registry from OpenShift Data Foundation. If you want to configure an alternative storage, see image registry

The PVCs that are created as a part of configuring OpenShift Container Platform registry are in the **openshift-image-registry** namespace.

Prerequisites

- The image registry should have been configured to use an OpenShift Data Foundation PVC.

Procedure

1. Edit the **configs.imageregistry.operator.openshift.io** object and remove the content in the **storage** section.

```
$ oc edit configs.imageregistry.operator.openshift.io
```

Before editing

```
.
.
.
storage:
  pvc:
    claim: registry-cephfs-rwx-pvc
.
.
.
```

After editing

```
.
.
.
storage:
  emptyDir: {}
.
.
.
```

In this example, the PVC is called **registry-cephfs-rwx-pvc**, which is now safe to delete.

2. Delete the PVC.

```
$ oc delete pvc <pvc-name> -n openshift-image-registry --wait=true --timeout=5m
```

6.4. REMOVING THE CLUSTER LOGGING OPERATOR FROM OPENSHIFT DATA FOUNDATION

Use this section to clean up the cluster logging operator from OpenShift Data Foundation.

The Persistent Volume Claims (PVCs) that are created as a part of configuring the cluster logging operator are in the **openshift-logging** namespace.

Prerequisites

- The cluster logging instance should have been configured to use the OpenShift Data Foundation PVCs.

Procedure

1. Remove the **ClusterLogging** instance in the namespace.

```
$ oc delete clusterlogging instance -n openshift-logging --wait=true --timeout=5m
```

The PVCs in the **openshift-logging** namespace are now safe to delete.

2. Delete the PVCs.

```
$ oc delete pvc <pvc-name> -n openshift-logging --wait=true --timeout=5m
```

<pvc-name>

Is the name of the PVC

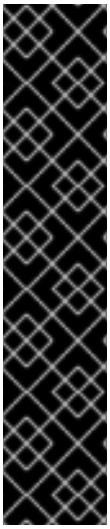
CHAPTER 7. DEPLOY STANDALONE MULTICLOUD OBJECT GATEWAY IN INTERNAL MODE

7.1. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR

You can install Red Hat OpenShift Data Foundation Operator using the Red Hat OpenShift Container Platform Software Catalog.

Prerequisites

- Access to an OpenShift Container Platform cluster using an account with **cluster-admin** and operator installation permissions.
- You must have at least three worker or infrastructure nodes in the Red Hat OpenShift Container Platform cluster.
- For additional resource requirements, see the Planning your deployment guide.



IMPORTANT

- When you need to override the cluster-wide default node selector for OpenShift Data Foundation, you can use the following command to specify a blank node selector for the **openshift-storage** namespace (create **openshift-storage** namespace in this case):

```
$ oc annotate namespace openshift-storage openshift.io/node-selector=
```

- Taint a node as **infra** to ensure only Red Hat OpenShift Data Foundation resources are scheduled on that node. This helps you save on subscription costs. For more information, see the *How to use dedicated worker nodes for Red Hat OpenShift Data Foundation* section in the Managing and Allocating Storage Resources guide.

Procedure

1. Log in to the OpenShift Web Console.
2. Click **Ecosystem → Software Catalog**.
3. Scroll or type **OpenShift Data Foundation** into the **Filter by keyword** box to find the **OpenShift Data Foundation** Operator.
4. Click **Install**.
5. Set the following options on the **Install Operator** page:
   a. Update Channel as **stable-4.22**.
   b. Installation Mode as **A specific namespace on the cluster**.
   c. Installed Namespace as **Operator recommended namespace openshift-storage**. If Namespace **openshift-storage** does not exist, it is created during the operator installation.

d. Select Approval Strategy as **Automatic** or **Manual**.
If you select **Automatic** updates, then the Operator Lifecycle Manager (OLM) automatically upgrades the running instance of your Operator without any intervention.

If you select **Manual** updates, then the OLM creates an update request. As a cluster administrator, you must then manually approve that update request to update the Operator to a newer version.

e. Ensure that the **Enable** option is selected for the **Console plugin**.

f. Click **Install**.

Verification

- After the operator is successfully installed, the web console automatically reloads to apply the changes. During this process, a temporary error message might appear on the page and this is expected and disappears after the refresh completes.
- In the Web Console:
  - Navigate to Installed Operators and verify that the **OpenShift Data Foundation** Operator shows a green tick indicating successful installation.
  - Navigate to **Storage** and verify if the **Data Foundation** dashboard is available.

7.2. CREATING A STANDALONE MULTICLOUD OBJECT GATEWAY

You can create only the standalone Multicloud Object Gateway (MCG) component while deploying OpenShift Data Foundation. After you create the MCG component, you can create and manage buckets using the MCG object browser. For more information, see Creating and managing buckets using MCG object browser.

Prerequisites

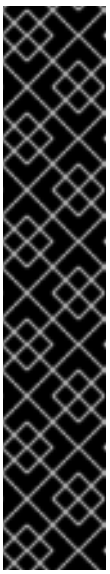
- Ensure that the OpenShift Data Foundation Operator is installed.
- The Multicloud Object Gateway database uses ReadWriteOncePod (RWOP) access mode to safeguard against node failures. If the underlying CSI driver does not support RWOP, the access mode must be manually changed to ReadWriteOnce (RWO) before deployment. For more information, see the knowledgebase article How to change NooBaa DB access mode from RWOP to RWO in standalone MCG deployments?.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → StorageCluster**.
2. On the Welcome page, click Configure Data Foundation.
3. Select Setup Multicloud Object Gateway.
4. In the **Backing storage** page, select the following:
   a. Select the **Use an existing StorageClass** option.
   b. Click **Next**.

5. In the **Advanced Settings** page, you can configure the following optional items:

- **Use external PostgreSQL** [Technology preview]
  Enables the use of an external PostgreSQL instance. This provides a high-availability solution for the Multicloud Object Gateway, where the PostgreSQL pod is a single point of failure.



IMPORTANT

OpenShift Data Foundation ships PostgreSQL images maintained by Red Hat, which are used to store metadata for the Multicloud Object Gateway. This PostgreSQL usage is at the application level.

As a result, OpenShift Data Foundation does not perform database-level optimizations or in-depth insights.

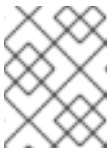
If you have well-maintained and optimized PostgreSQL instance, then it is recommended to use it. OpenShift Data Foundation supports external PostgreSQL instances.

Any PostgreSQL-related issues requiring code changes or deep technical analysis may need to be addressed upstream. This could result in longer resolution times.

  a. Provide the following connection details:
    - **Username**
    - **Password**
    - **Server name** and **Port**
    - **Database name**
  b. Select **Enable TLS/SSL** checkbox to enable encryption for the Postgres server.

- **Enable automatic backup**
  Allows automatic backup for the NooBaa database.

  a. Select the backup frequency: Daily, Weekly, or Monthly.
  b. Specify the number of backups to retain.
  c. Select a VolumeSnapshotClass to add a volume snapshot.



NOTE

This option is disabled if the **Use external PostgreSQL** option is selected.

  d. Click **Next**.

6. In the **Security** page, select any of the following optional items:

  a. Select the **Connect to an external key management service**checkbox. This is optional for cluster-wide encryption.

b. From the **Key Management Service Provider** drop-down list, either select **Vault** or **Thales CipherTrust Manager (using KMIP)**. If you selected **Vault**, go to the next step. If you selected **Thales CipherTrust Manager (using KMIP)**, go to step iii.

c. Select an **Authentication Method**.

Using Token authentication method

- Enter a unique **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Token**.
- Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:
  - Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.
  - Optional: Enter **TLS Server Name**, **Authentication Path**, and **Vault Enterprise Namespace**.
  - Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key** .
  - Click **Save** and skip to step iv.

Using Kubernetes authentication method

- Enter a unique Vault **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Role** name.
- Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:
  - Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.
  - Optional: Enter **TLS Server Name**, **Authentication Path**, and **Vault Enterprise Namespace** if applicable.
  - Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key** .
  - Click **Save** and skip to step iv.

d. To use **Thales CipherTrust Manager (using KMIP)** as the KMS provider, follow the steps below:

i. Enter a unique **Connection Name** for the Key Management service within the project.

ii. In the **Address** and **Port** sections, enter the IP of Thales CipherTrust Manager and the port where the KMIP interface is enabled. For example:

- **Address**: 123.34.3.2
- **Port**: 5696

iii. Upload the **Client Certificate**, **CA certificate**, and **Client Private Key**.

iv. If StorageClass encryption is enabled, enter the Unique Identifier to be used for encryption and decryption generated above.

v. The **TLS Server** field is optional and used when there is no DNS entry for the KMIP endpoint. For example, **kmip_all_<port>.ciphertrustmanager.local**.

e. Select a **Network**.

f. Click **Next**.

7. In the **Review and create** page, review the configuration details:
To modify any configuration settings, click **Back**.

8. Click **Create StorageSystem**.

Verification

Verifying that the OpenShift Data Foundation cluster is healthy

1. In the OpenShift Web Console, click **Storage → Data Foundation**.

2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.

   a. In the **Status card** of the **Object** tab, verify that both *Object Service* and *Data Resiliency* have a green tick.

   b. In the **Details** card, verify that the MCG information is displayed.

Verifying the state of the pods

1. Click **Workloads → Pods** from the OpenShift Web Console.

2. Select **openshift-storage** from the **Project** drop-down list and verify that the following pods are in **Running** state.



NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

Component	Corresponding pods

Component	Corresponding pods
OpenShift Data Foundation Operator	• **ocs-operator-*** (1 pod on any storage node) • **ocs-metrics-exporter-*** (1 pod on any storage node) • **odf-operator-controller-manager-*** (1 pod on any storage node) • **odf-console-*** (1 pod on any storage node) • **csi-addons-controller-manager-*** (1 pod on any storage node)
Rook-ceph Operator	**rook-ceph-operator-*** (1 pod on any storage node)
Multicloud Object Gateway	• **noobaa-operator-*** (1 pod on any storage node) • **noobaa-core-*** (1 pod on any storage node) • **noobaa-db-pg-cluster-1** and noobaa-db-pg-cluster-2 (2 instances of MCG DB pod on any storage node) • **noobaa-endpoint-*** (1 pod on any storage node) • **cnpg-controller-manager-*** (1 pod on any storage node)

CHAPTER 8. VIEW OPENSHIFT DATA FOUNDATION TOPOLOGY

The topology provides a visual representation of the OpenShift Data Foundation storage cluster at multiple levels of abstraction and enables interaction with each layer. It illustrates how the cluster components are organized and related, while offering visibility into Object Storage Devices (OSDs), including their count, type, and status.

Procedure

1. On the OpenShift Web Console, navigate to **Storage → Data Foundation → Topology**.
   The view shows the storage cluster and the topology groupings inside it. You can see the nodes depicted by circular entities within the topology groupings, which are indicated by dotted lines. The topology groupings are based on the cluster's topology configuration (for example, racks in bare metal or vSphere clusters, or zones in cloud-based clusters). The label of each item or resource contains basic information such as status and health or indication for alerts.

2. Choose a node to view node details on the right-hand panel. You can also access resources or deployments within a node by clicking on the search/preview decorator icon.
   The node details panel displays information about the OSDs running on that node, including:

   - Number of OSDs on the node
   - OSD disk type (for example, SSD, HDD)
   - OSD status (for example, Running, Error, CrashLoopBackOff)

3. To view deployment details

   a. Click the preview decorator on a node. A modal window appears above the node that displays all of the deployments associated with that node along with their statuses.

   b. Click the **Back to main view** button in the model's upper left corner to close and return to the previous view.

   c. Select a specific deployment to see more information about it. All relevant data is shown in the side panel.

4. Click the **Resources** tab to view the pods information. This tab provides a deeper understanding of the problems and offers granularity that aids in better troubleshooting.

5. Click the **OSD information** tab to view detailed information about the OSDs. This tab provides more comprehensive OSD-specific details, including:

   - OSD pod name and status
   - Disk device associated with each OSD
   - Device class (for example, SSD, HDD)
   - Resource utilization metrics

6. Click the pod links to view the pod information page on OpenShift Container Platform. The link opens in a new window.

CHAPTER 9. STORAGE CLASSES AND STORAGE POOLS

Storage classes and storage pools in OpenShift Data Foundation control how persistent volumes are provisioned, including replica count, compression, and encryption settings.

The OpenShift Data Foundation operator installs a default storage class depending on the platform in use. This default storage class is owned and controlled by the operator and it cannot be deleted or modified. However, you can create a custom storage class if you want the storage class to have a different behavior.

You can create multiple storage pools which map to storage classes that provide the following features:

- Enable applications with their own high availability to use persistent volumes with two replicas, potentially improving application performance.
- Save space for persistent volume claims using storage classes with compression enabled.



NOTE

Multiple storage classes and multiple pools are not supported for *external mode* OpenShift Data Foundation clusters.



NOTE

With a minimal cluster of a single device set, only two new storage classes can be created. Every storage cluster expansion allows two new additional storage classes.

9.1. CREATING STORAGE CLASSES AND POOLS

You can create a storage class using an existing pool or you can create a new pool for the storage class while creating it.

Prerequisites

- Ensure that you are logged into the OpenShift Container Platform web console and OpenShift Data Foundation cluster is in **Ready** state.

Procedure

1. Click **Storage → StorageClasses**.
2. Click **Create Storage Class**
3. Enter the storage class **Name** and **Description**.
4. **Reclaim Policy** is set to **Delete** as the default option. Use this setting.
   If you change the reclaim policy to **Retain** in the storage class, the persistent volume (PV) remains in **Released** state even after deleting the persistent volume claim (PVC).
5. **Volume binding mode** is set to **WaitForConsumer** as the default option.
   If you choose the **Immediate** option, then the PV gets created immediately when creating the PVC.
6. Select **RBD** or **CephFS Provisioner** as the plugin for provisioning the persistent volumes.

7. Choose a **Storage system** for your workloads.
8. Select an existing **Storage Pool** from the list or create a new pool.



NOTE

The 2-way replication data protection policy is only supported for the non-default RBD pool. 2-way replication can be used by creating an additional pool. To know about Data Availability and Integrity considerations for replica 2 pools, see Knowledgebase Customer Solution Article .

Create new pool

a. Click **Create New Pool**
b. Enter **Pool name**.
c. Select **Data protection policy** as either **Replication** or **Erasure coding**.
   i. Select **Erasure coding** to enable the use of erasure coding instead of replication to protect the cluster data. Supported only for Block (RBD) and File (CephFS).
   ii. Select the **Scheme** (k+m) for your workload.



NOTE

The minimum number of storage nodes required is equal to the number of data chunks (k) plus the number of parity chunks (m). The recommended minimum is k + m + 1 nodes.

d. Select **Data compression** if you need to compress the data.
Enabling compression can impact application performance and might prove ineffective when data to be written is already compressed or encrypted. Data written before enabling compression will not be compressed.
e. Click **Create** to create the new storage pool.
f. Click **Finish** after the pool is created.

9. Optional: Select **Enable Encryption** checkbox.
10. Click **Create** to create the storage class.

9.2. CREATING A STORAGE CLASS FOR PERSISTENT VOLUME ENCRYPTION

Prerequisites

Based on your use case, you must ensure to configure access to KMS for one of the following:

- Using **vaulttokens**: Ensure to configure access as described in Configuring access to KMS using **vaulttokens**

- Using **vaulttenantsa**: Ensure to configure access as described in Configuring access to KMS using **vaulttenantsa**
- Using Thales CipherTrust Manager (using KMIP): Ensure to configure access as described in Configuring access to KMS using Thales CipherTrust Manager
- (For users on Azure platform only) Using Azure Vault: Ensure to set up client authentication and fetch the client credentials from Azure using the following steps:
    1. Create Azure Vault. For more information, see Quickstart: Create a key vault using the Azure portal in Microsoft product documentation.
    2. Create Service Principal with certificate based authentication. For more information, see Create an Azure service principal with Azure CLI in Microsoft product documentation.
    3. Set Azure Key Vault role based access control (RBAC). For more information, see Enable Azure RBAC permissions on Key Vault.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → StorageClasses**.
2. Click **Create Storage Class**
3. Enter the storage class **Name** and **Description**.
4. Select either **Delete** or **Retain** for the **Reclaim Policy**. By default, **Delete** is selected.
5. Select either **Immediate** or **WaitForFirstConsumer** as the **Volume binding mode**. **WaitForConsumer** is set as the default option.
6. Select **RBD Provisioner openshift-storage.rbd.csi.ceph.com** which is the plugin used for provisioning the persistent volumes.
7. Select **Storage Pool** where the volume data is stored from the list or create a new pool.
8. Select the **Enable encryption** checkbox.
    - Choose one of the following options to set the KMS connection details:
        - **Choose existing KMS connection**: Select an existing KMS connection from the drop-down list. The list is populated from the the connection details available in the **csi-kms-connection-details** ConfigMap.
            a. Select the **Provider** from the drop down.
            b. Select the **Key service** for the given provider from the list.
        - **Create new KMS connection**: This is applicable for **vaulttokens** and **Thales CipherTrust Manager (using KMIP)** only.
            a. Select one of the following **Key Management Service Provider** and provide the required details.
                - **Vault**
                    i. Enter a unique **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), Port number and **Token**.

ii. Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:

A. Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.

B. Optional: Enter **TLS Server Name** and **Vault Enterprise Namespace**.

C. Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key** .

D. Click **Save**.

- **Thales CipherTrust Manager (using KMIP)**

i. Enter a unique **Connection Name**.

ii. In the **Address** and **Port** sections, enter the IP of Thales CipherTrust Manager and the port where the KMIP interface is enabled. For example, **Address**: 123.34.3.2, **Port**: 5696.

iii. Upload the **Client Certificate**, **CA certificate**, and **Client Private Key**.

iv. Enter the **Unique Identifier** for the key to be used for encryption and decryption, generated above.

v. The **TLS Server** field is optional and used when there is no DNS entry for the KMIP endpoint. For example, **kmip_all_<port>.ciphertrustmanager.local**.

- **Azure Key Vault** (Only for Azure users on Azure platform)
For information about setting up client authentication and fetching the client credentials, see the Prerequisites in Creating an OpenShift Data Foundation cluster section of the *Deploying OpenShift Data Foundation using Microsoft Azure* guide.

i. Enter a unique **Connection name** for the key management service within the project.

ii. Enter **Azure Vault URL**.

iii. Enter **Client ID**.

iv. Enter **Tenant ID**.

v. Upload **Certificate** file in **.PEM** format and the certificate file must include a client certificate and a private key.

b. Click **Save**.

c. Click **Create**.

9. Edit the ConfigMap to add the **vaultBackend** parameter if the HashiCorp Vault setup does not allow automatic detection of the Key/Value (KV) secret engine API version used by the backend path.



NOTE

vaultBackend is an optional parameters that is added to the **configmap** to specify the version of the KV secret engine API associated with the backend path. Ensure that the value matches the KV secret engine API version that is set for the backend path, otherwise it might result in a failure during persistent volume claim (PVC) creation.

a. Identify the encryptionKMSID being used by the newly created storage class.

   i. On the OpenShift Web Console, navigate to **Storage → Storage Classes**.

   ii. Click the **Storage class** name → **YAML** tab.

   iii. Capture the **encryptionKMSID** being used by the storage class.
   Example:

```
encryptionKMSID: 1-vault
```

b. On the OpenShift Web Console, navigate to **Workloads → ConfigMaps**.

c. To view the KMS connection details, click **csi-kms-connection-details**.

d. Edit the ConfigMap.

   i. Click Action menu **(⋮) → Edit ConfigMap**.

   ii. Add the **vaultBackend** parameter depending on the backend that is configured for the previously identified **encryptionKMSID**.
   You can assign **kv** for KV secret engine API, version 1 and **kv-v2** for KV secret engine API, version 2.

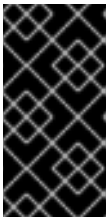
   Example:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: csi-kms-connection-details
[...]
data:
  1-vault: |-
    {
      "encryptionKMSType": "vaulttokens",
      "kmsServiceName": "1-vault",
      [...]
      "vaultBackend": "kv-v2"
    }
  2-vault: |-
    {
      "encryptionKMSType": "vaulttenantsa",
      [...]
      "vaultBackend": "kv"
    }
```

   iii. Click Save

Next steps

- The storage class can be used to create encrypted persistent volumes. For more information, see managing persistent volume claims.



IMPORTANT

Red Hat works with the technology partners to provide this documentation as a service to the customers. However, Red Hat does not provide support for the HashiCorp product. For technical assistance with this product, contact HashiCorp.

CHAPTER 10. CONFIGURE STORAGE FOR OPENSHIFT CONTAINER PLATFORM SERVICES

You can use OpenShift Data Foundation to provide storage for OpenShift Container Platform services such as image registry, monitoring, and logging.

The process for configuring storage for these services depends on the infrastructure used in your OpenShift Data Foundation deployment.



WARNING

Always ensure that you have plenty of storage capacity for these services. If the storage for these critical services runs out of space, the cluster becomes inoperable and very difficult to recover.

Red Hat recommends configuring shorter curation and retention intervals for these services. See *Configuring the Curator schedule* and the *Modifying retention time for Prometheus metrics data* sub section of *Configuring persistent storage* in the OpenShift Container Platform documentation for details.

If you do run out of storage space for these services, contact Red Hat Customer Support.

10.1. CONFIGURING IMAGE REGISTRY TO USE OPENSHIFT DATA FOUNDATION

OpenShift Container Platform provides a built in Container Image Registry which runs as a standard workload on the cluster. A registry is typically used as a publication target for images built on the cluster as well as a source of images for workloads running on the cluster.



WARNING

This process does not migrate data from an existing image registry to the new image registry. If you already have container images in your existing registry, back up your registry before you complete this process, and re-register your images when this process is complete.

Prerequisites

- Administrative access to OpenShift Web Console.
- OpenShift Data Foundation Operator is installed and running in the **openshift-storage** namespace. In OpenShift Web Console, click **Ecosystem → Installed Operators** to view installed operators.

- Image Registry Operator is installed and running in the **openshift-image-registry** namespace. In OpenShift Web Console, click **Administration → Cluster Settings → Cluster Operators** to view cluster operators.
- A storage class with provisioner **openshift-storage.cephfs.csi.ceph.com** is available. In OpenShift Web Console, click **Storage → StorageClasses** to view available storage classes.

Procedure

1. **Create a Persistent Volume Claim for the Image Registry to use.**
   a. In the OpenShift Web Console, click **Storage → Persistent Volume Claims**
   b. Set the **Project** to **openshift-image-registry**.
   c. Click **Create Persistent Volume Claim**
      i. From the list of available storage classes retrieved above, specify the **Storage Class** with the provisioner **openshift-storage.cephfs.csi.ceph.com**.
      ii. Specify the Persistent Volume Claim **Name**, for example, **ocs4registry**.
      iii. Specify an **Access Mode** of **Shared Access (RWX)**.
      iv. Specify a **Size** of at least 100 GB.
      v. Click **Create**.
      Wait until the status of the new Persistent Volume Claim is listed as **Bound**.
2. **Configure the cluster's Image Registry to use the new Persistent Volume Claim.**
   a. Click **Administration → Custom Resource Definitions**
   b. Click the **Config** custom resource definition associated with the **imageregistry.operator.openshift.io** group.
   c. Click the **Instances** tab.
   d. Beside the cluster instance, click the **Action Menu (⋮) → Edit Config**.
   e. Add the new Persistent Volume Claim as persistent storage for the Image Registry.
      i. Add the following under **spec:**, replacing the existing **storage:** section if necessary.

```
storage:
  pvc:
    claim: <new-pvc-name>
```

      For example:

```
storage:
  pvc:
    claim: ocs4registry
```

      ii. Click **Save**.

3. **Verify that the new configuration is being used.**

   a. Click **Workloads → Pods**.

   b. Set the **Project** to **openshift-image-registry**.

   c. Verify that the new **image-registry-*** pod appears with a status of **Running**, and that the previous **image-registry-*** pod terminates.

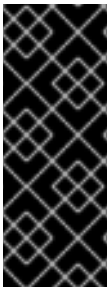
   d. Click the new **image-registry-*** pod to view pod details.

   e. Scroll down to **Volumes** and verify that the **registry-storage** volume has a **Type** that matches your new Persistent Volume Claim, for example, **ocs4registry**.

10.2. CONFIGURING MONITORING TO USE OPENSHIFT DATA FOUNDATION

OpenShift Data Foundation provides a monitoring stack that comprises of Prometheus and Alert Manager.

Follow the instructions in this section to configure OpenShift Data Foundation as storage for the monitoring stack.



IMPORTANT

Monitoring will not function if it runs out of storage space. Always ensure that you have plenty of storage capacity for monitoring.

Red Hat recommends configuring a short retention interval for this service. See the Modifying retention time for Prometheus metrics data of Monitoring guide in the OpenShift Container Platform documentation for details.

Prerequisites

- Administrative access to OpenShift Web Console.
- OpenShift Data Foundation Operator is installed and running in the **openshift-storage** namespace. In the OpenShift Web Console, click **Ecosystem → Installed Operators** to view installed operators.
- Monitoring Operator is installed and running in the **openshift-monitoring** namespace. In the OpenShift Web Console, click **Administration → Cluster Settings → Cluster Operators** to view cluster operators.
- A storage class with provisioner **openshift-storage.rbd.csi.ceph.com** is available. In the OpenShift Web Console, click **Storage → StorageClasses** to view available storage classes.

Procedure

1. In the OpenShift Web Console, go to **Workloads → Config Maps**.
2. Set the **Project** dropdown to **openshift-monitoring**.
3. Click **Create Config Map**.

4. Define a new **cluster-monitoring-config** Config Map using the following example. Replace the content in angle brackets (**<**, **>**) with your own values, for example, **retention: 24h** or **storage: 40Gi**.

   Replace the **storageClassName** with the **storageclass** that uses the provisioner **openshift-storage.rbd.csi.ceph.com**. In the example given below the name of the **storageclass** is **ocs-storagecluster-ceph-rbd**.

   Example cluster-monitoring-config Config Map

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
data:
  config.yaml: |
    prometheusK8s:
      retention: <time to retain monitoring files, for example 24h>
      volumeClaimTemplate:
        metadata:
          name: ocs-prometheus-claim
        spec:
          storageClassName: ocs-storagecluster-ceph-rbd
          resources:
            requests:
              storage: <size of claim, e.g. 40Gi>
    alertmanagerMain:
      volumeClaimTemplate:
        metadata:
          name: ocs-alertmanager-claim
        spec:
          storageClassName: ocs-storagecluster-ceph-rbd
          resources:
            requests:
              storage: <size of claim, e.g. 40Gi>
```

1. Click **Create** to save and create the Config Map.

Verification

1. Verify that the Persistent Volume Claims are bound to the pods.

   a. Go to **Storage → Persistent Volume Claims**

   b. Set the **Project** dropdown to **openshift-monitoring**.

   c. Verify that 5 Persistent Volume Claims are visible with a state of **Bound**, attached to three **alertmanager-main-*** pods, and two **prometheus-k8s-*** pods.
   Monitoring storage created and bound image::images/openshift-monitoring-pvcs-created.png["Screenshot of OpenShift Web Console showing five pods with persistent volume claims bound in the openshift-monitoring project"]

2. Verify that the new **alertmanager-main-*** pods appear with a state of **Running**.

a. Go to **Workloads → Pods**.

b. Click the new **alertmanager-main-*** pods to view the pod details.

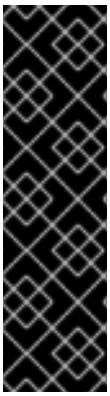
c. Scroll down to **Volumes** and verify that the volume has a **Type**, **ocs-alertmanager-claim** that matches one of your new Persistent Volume Claims, for example, **ocs-alertmanager-claim-alertmanager-main-0**.
Persistent Volume Claims attached to alertmanager-main-* pod* image::images/openshift-monitoring-alert-pods-attached.png["Screenshot of OpenShift Web Console showing persistent volume claim attached to the altermanager pod"]

3. Verify that the new **prometheus-k8s-*** pods appear with a state of **Running**.

a. Click the new **prometheus-k8s-*** pods to view the pod details.

b. Scroll down to **Volumes** and verify that the volume has a **Type**, **ocs-prometheus-claim** that matches one of your new Persistent Volume Claims, for example, **ocs-prometheus-claim-prometheus-k8s-0**.
Persistent Volume Claims attached to prometheus-k8s-* pod* image::images/openshift-monitoring-prometheus-pods-attached.png["Screenshot of OpenShift Web Console showing persistent volume claim attached to the prometheus pod"]

CHAPTER 11. CLUSTER LOGGING FOR OPENSHIFT DATA FOUNDATION



IMPORTANT

Always ensure that you have plenty of storage capacity for these services. If you run out of storage space for these critical services, the logging application becomes inoperable and very difficult to recover.

Red Hat recommends configuring shorter curation and retention intervals for these services. See *Cluster logging curator* in the OpenShift Container Platform documentation for details.

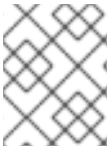
If you run out of storage space for these services, contact Red Hat Customer Support.

11.1. CONFIGURING PERSISTENT STORAGE

You can configure a persistent storage class and size for the Elasticsearch cluster using the storage class name and size parameters. The Cluster Logging Operator creates a Persistent Volume Claim for each data node in the Elasticsearch cluster based on these parameters. For example:

```
spec:
   logStore:
     type: "elasticsearch"
     elasticsearch:
       nodeCount: 3
       storage:
         storageClassName: "ocs-storagecluster-ceph-rbd”
         size: "200G"
```

This example specifies that each data node in the cluster will be bound to a Persistent Volume Claim that requests **200GiB** of **ocs-storagecluster-ceph-rbd** storage. Each primary shard will be backed by a single replica. A copy of the shard is replicated across all the nodes and are always available and the copy can be recovered if at least two nodes exist due to the single redundancy policy. For information about Elasticsearch replication policies, see *Elasticsearch replication policy* in About deploying and configuring cluster logging.



NOTE

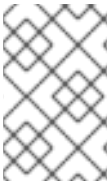
Omission of the storage block will result in a deployment backed by default storage. For example:

```
spec:
   logStore:
     type: "elasticsearch"
     elasticsearch:
       nodeCount: 3
       storage: {}
```

For more information, see Configuring cluster logging.

11.2. CONFIGURING CLUSTER LOGGING TO USE OPENSHIFT DATA FOUNDATION

Follow the instructions in this section to configure OpenShift Data Foundation as storage for the OpenShift cluster logging.



NOTE

You can obtain all the logs when you configure logging for the first time in OpenShift Data Foundation. However, after you uninstall and reinstall logging, the old logs are removed and only the new logs are processed.

Prerequisites

- Administrative access to OpenShift Web Console.
- OpenShift Data Foundation Operator is installed and running in the **openshift-storage** namespace.
- Cluster logging Operator is installed and running in the **openshift-logging** namespace.

Procedure

1. Click **Administration → Custom Resource Definitions**from the left pane of the OpenShift Web Console.
2. On the Custom Resource Definitions page, click **ClusterLogging**.
3. On the Custom Resource Definition Overview page, select **View Instances** from the Actions menu or click the **Instances** Tab.
4. On the Cluster Logging page, click **Create Cluster Logging**.
   You might have to refresh the page to load the data.
5. In the YAML, replace the **storageClassName** with the **storageclass** that uses the provisioner **openshift-storage.rbd.csi.ceph.com**. In the example given below the name of the **storageclass** is **ocs-storagecluster-ceph-rbd**:

```
apiVersion: "logging.openshift.io/v1"
kind: "ClusterLogging"
metadata:
  name: "instance"
  namespace: "openshift-logging"
spec:
  managementState: "Managed"
  logStore:
    type: "elasticsearch"
    elasticsearch:
      nodeCount: 3
      storage:
        storageClassName: ocs-storagecluster-ceph-rbd
        size: 200G # Change as per your requirement
      redundancyPolicy: "SingleRedundancy"
  visualization:
    type: "kibana"
```

```
    kibana:
      replicas: 1
  curation:
    type: "curator"
    curator:
      schedule: "30 3 * * *"
  collection:
    logs:
      type: "fluentd"
      fluentd: {}
```

If you have tainted the OpenShift Data Foundation nodes, you must add toleration to enable scheduling of the daemonset pods for logging.

```
spec:
[...]
  collection:
    logs:
      fluentd:
        tolerations:
        - effect: NoSchedule
          key: node.ocs.openshift.io/storage
          value: 'true'
      type: fluentd
```

6. Click **Save**.

Verification

1. Verify that the Persistent Volume Claims are bound to the **elasticsearch** pods.
   a. Go to **Storage → Persistent Volume Claims**.
   b. Set the **Project** dropdown to **openshift-logging**.
   c. Verify that Persistent Volume Claims are visible with a state of **Bound**, attached to **elasticsearch-*** pods.
   Cluster logging created and bound image::images/ocs-cluster-logging-verify.png[Screenshot of Persistent Volume Claims with a bound state attached to elasticsearch pods]
2. Verify that the new cluster logging is being used.
   a. Click **Workload → Pods**.
   b. Set the Project to **openshift-logging**.
   c. Verify that the new **elasticsearch-*** pods appear with a state of **Running**.
   d. Click the new **elasticsearch-*** pod to view pod details.
   e. Scroll down to **Volumes** and verify that the elasticsearch volume has a **Type** that matches your new Persistent Volume Claim, for example, **elasticsearch-elasticsearch-cdm-9r624biv-3**.

f. Click the Persistent Volume Claim name and verify the storage class name in the PersistentVolumeClaim Overview page.



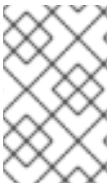
NOTE

Make sure to use a shorter curator time to avoid PV full scenario on PVs attached to Elasticsearch pods.

You can configure Curator to delete Elasticsearch data based on retention settings. It is recommended that you set the following default index data retention of 5 days as a default.

```
config.yaml: |
    openshift-storage:
      delete:
        days: 5
```

For more details, see Curation of Elasticsearch Data .



NOTE

To uninstall the cluster logging backed by Persistent Volume Claim, use the procedure removing the cluster logging operator from OpenShift Data Foundation in the uninstall chapter of the respective deployment guide.

CHAPTER 12. BACKING OPENSHIFT CONTAINER PLATFORM APPLICATIONS WITH OPENSHIFT DATA FOUNDATION

You cannot directly install OpenShift Data Foundation during the OpenShift Container Platform installation. However, you can install OpenShift Data Foundation on an existing OpenShift Container Platform by using the Software Catalog and then configure the OpenShift Container Platform applications to be backed by OpenShift Data Foundation.

Prerequisites

- OpenShift Container Platform is installed and you have administrative access to OpenShift Web Console.
- OpenShift Data Foundation is installed and running in the **openshift-storage** namespace.

Procedure

1. In the OpenShift Web Console, perform one of the following:
   - Click **Workloads → Deployments**.
     In the Deployments page, you can do one of the following:
     - Select any existing deployment and click **Add Storage** option from the **Action** menu (⋮).
     - Create a new deployment and then add storage.
       i. Click **Create Deployment** to create a new deployment.
       ii. Edit the **YAML** based on your requirement to create a deployment.
       iii. Click **Create**.
       iv. Select **Add Storage** from the **Actions** drop-down menu on the top right of the page.
   - Click **Workloads → Deployment Configs**
     In the Deployment Configs page, you can do one of the following:
     - Select any existing deployment and click **Add Storage** option from the **Action** menu (⋮).
     - Create a new deployment and then add storage.
       i. Click **Create Deployment Config** to create a new deployment.
       ii. Edit the **YAML** based on your requirement to create a deployment.
       iii. Click **Create**.
       iv. Select **Add Storage** from the **Actions** drop-down menu on the top right of the page.
2. In the Add Storage page, you can choose one of the following options:
   - Click the **Use existing claim** option and select a suitable PVC from the drop-down list.

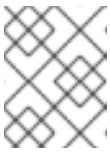
- Click the **Create new claim** option.
    a. Select the appropriate **CephFS** or **RBD** storage class from the **Storage Class** drop-down list.
    b. Provide a name for the Persistent Volume Claim.
    c. Select ReadWriteOnce (RWO) or ReadWriteMany (RWX) access mode.



NOTE

ReadOnlyMany (ROX) is deactivated as it is not supported.

    d. Select the size of the desired storage capacity.



NOTE

You can expand the block PVs but cannot reduce the storage capacity after the creation of Persistent Volume Claim.

3. Specify the mount path and subpath (if required) for the mount path volume inside the container.
4. Click **Save**.

Verification

1. Depending on your configuration, perform one of the following:
    - Click **Workloads → Deployments**.
    - Click **Workloads → Deployment Configs**
2. Set the Project as required.
3. Click the deployment for which you added storage to display the deployment details.
4. Scroll down to **Volumes** and verify that your deployment has a **Type** that matches the Persistent Volume Claim that you assigned.
5. Click the Persistent Volume Claim name and verify the storage class name in the Persistent Volume Claim Overview page.

CHAPTER 13. HOW TO USE DEDICATED WORKER NODES FOR RED HAT OPENSHIFT DATA FOUNDATION

Any Red Hat OpenShift Container Platform subscription requires an OpenShift Data Foundation subscription. However, you can save on the OpenShift Container Platform subscription costs if you are using infrastructure nodes to schedule OpenShift Data Foundation resources.

It is important to maintain consistency across environments with or without Machine API support. Because of this, it is highly recommended in all cases to have a special category of nodes labeled as either worker or infra or have both roles. See the *Manual creation of infrastructure nodes* section for more information.

13.1. ANATOMY OF AN INFRASTRUCTURE NODE

Infrastructure nodes for use with OpenShift Data Foundation have a few attributes. The **infra** node-role label is required to ensure the node does not consume RHOCP entitlements. The **infra** node-role label is responsible for ensuring only OpenShift Data Foundation entitlements are necessary for the nodes running OpenShift Data Foundation.

- Labeled with **node-role.kubernetes.io/infra**

Adding an OpenShift Data Foundation taint with a **NoSchedule** effect is also required so that the **infra** node will only schedule OpenShift Data Foundation resources.

- Tainted with **node.ocs.openshift.io/storage="true"**

The label identifies the RHOCP node as an **infra** node so that RHOCP subscription cost is not applied. The taint prevents non OpenShift Data Foundation resources to be scheduled on the tainted nodes.



NOTE

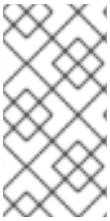
Adding storage taint on nodes might require toleration handling for the other **daemonset** pods such as **openshift-dns daemonset**. For information about how to manage the tolerations, see the *Openshift-dns daemonsets tolerations* knowledge base article.

Example of the taint and labels required on infrastructure node that will be used to run OpenShift Data Foundation services:

```
spec:
  taints:
  - effect: NoSchedule
    key: node.ocs.openshift.io/storage
    value: "true"
  metadata:
    creationTimestamp: null
    labels:
      node-role.kubernetes.io/worker: ""
      node-role.kubernetes.io/infra: ""
      cluster.ocs.openshift.io/openshift-storage: ""
```

13.2. MACHINE SETS FOR CREATING INFRASTRUCTURE NODES

If the Machine API is supported in the environment, then labels should be added to the templates for the Machine Sets that will be provisioning the infrastructure nodes. Avoid the anti-pattern of adding labels manually to nodes created by the machine API. Doing so is analogous to adding labels to pods created by a deployment. In both cases, when the pod/node fails, the replacement pod/node will not have the appropriate labels.



NOTE

In EC2 environments, you will need three machine sets, each configured to provision infrastructure nodes in a distinct availability zone (such as us-east-2a, us-east-2b, us-east-2c). Currently, OpenShift Data Foundation does not support deploying in more than three availability zones.

The following Machine Set template example creates nodes with the appropriate taint and labels required for infrastructure nodes. This will be used to run OpenShift Data Foundation services.

```
template:
  metadata:
    creationTimestamp: null
    labels:
      machine.openshift.io/cluster-api-cluster: kb-s25vf
      machine.openshift.io/cluster-api-machine-role: worker
      machine.openshift.io/cluster-api-machine-type: worker
      machine.openshift.io/cluster-api-machineset: kb-s25vf-infra-us-west-2a
  spec:
    taints:
    - effect: NoSchedule
      key: node.ocs.openshift.io/storage
      value: "true"
    metadata:
      creationTimestamp: null
      labels:
        node-role.kubernetes.io/infra: ""
        cluster.ocs.openshift.io/openshift-storage: ""
```



IMPORTANT

If you add a taint to the infrastructure nodes, you also need to add tolerations to the taint for other workloads, for example, the fluentd pods. For more information, see the *Infrastructure Nodes in OpenShift 4* knowledge base article.

13.3. MANUAL CREATION OF INFRASTRUCTURE NODES

Only when the Machine API is not supported in the environment should labels be directly applied to nodes. Manual creation requires that at least 3 RHOCP worker nodes are available to schedule OpenShift Data Foundation services, and that these nodes have sufficient CPU and memory resources. To avoid the RHOCP subscription cost, the following is required:

```
oc label node <node> node-role.kubernetes.io/infra=""
oc label node <node> cluster.ocs.openshift.io/openshift-storage=""
```

Adding a **NoSchedule** OpenShift Data Foundation taint is also required so that the **infra** node will only schedule OpenShift Data Foundation resources and repel any other non-OpenShift Data Foundation workloads.

```
oc adm taint node <node> node.ocs.openshift.io/storage="true":NoSchedule
```



WARNING

Do not remove the node-role node-role.kubernetes.io/worker=""

The removal of the **node-role.kubernetes.io/worker=""** can cause issues unless changes are made both to the OpenShift scheduler and to MachineConfig resources.

If already removed, it should be added again to each **infra** node. Adding node-role **node-role.kubernetes.io/infra=""** and OpenShift Data Foundation taint is sufficient to conform to entitlement exemption requirements.

CHAPTER 14. SCALING STORAGE NODES

Expand your OpenShift Data Foundation cluster capacity by scaling up existing nodes with additional storage devices or scaling out by adding new worker nodes.

To scale the storage capacity of OpenShift Data Foundation, you can do either of the following:

- **Scale up storage nodes** - Add storage capacity to the existing OpenShift Data Foundation worker nodes
- **Scale out storage nodes** - Add new worker nodes containing storage capacity

14.1. REQUIREMENTS FOR SCALING STORAGE NODES

Before you proceed to scale the storage nodes, refer to the following sections to understand the node requirements for your specific Red Hat OpenShift Data Foundation instance:

- Platform requirements
- Storage device requirements
  - Dynamic storage devices
  - Capacity planning



WARNING

Always ensure that you have plenty of storage capacity.

If storage ever fills completely, it is not possible to add capacity or delete or migrate content away from the storage to free up space. Completely full storage is very difficult to recover.

Capacity alerts are issued when cluster storage capacity reaches 75% (near-full) and 85% (full) of total capacity. Always address capacity warnings promptly, and review your storage regularly to ensure that you do not run out of storage space.

If you do run out of storage space completely, contact Red Hat Customer Support.

14.2. SCALING UP STORAGE BY ADDING CAPACITY TO YOUR OPENSHIFT DATA FOUNDATION NODES ON RED HAT OPENSTACK PLATFORM INFRASTRUCTURE

To increase the storage capacity in a dynamically created storage cluster on an user-provisioned infrastructure, you can add storage capacity and performance to your configured Red Hat OpenShift Data Foundation worker nodes.

Prerequisites

- You have administrative privilege to the OpenShift Container Platform Console.

- You have a running OpenShift Data Foundation Storage Cluster.
- The disk should be of the same size and type as used during initial deployment.

Procedure

1. Log in to the OpenShift Web Console.
2. Navigate to **Storage → Storage cluster**.
   a. Click **Actions** at the top right of the page to extend the options menu.
   b. Select **Add Capacity** from the options menu.
   c. Select the **Storage Class**. Choose the storage class which you wish to use to provision new storage devices.
   + The storage class should be set to **standard** if you are using the default storage class generated during deployment. If you have created other storage classes, select whichever is appropriate.

   + The **Raw Capacity** field shows the size set during storage class creation. The total amount of storage consumed is three times this amount, because OpenShift Data Foundation uses a replica count of 3.

   d. Click **Add**.
3. To check the status, navigate to **Storage → Storage cluster** and verify that the **Storage Cluster** in the Status card has a green tick.

Verification

- Verify the **Raw Capacity** card.
  a. In the OpenShift Web Console, click **Storage → Storage cluster**.
  b. In the **Block and File** tab, check the **Raw Capacity** card.
  Note that the capacity increases based on your selections.



NOTE

The raw capacity does not take replication into account and shows the full capacity.

- Verify that the new OSDs and their corresponding new Persistent Volume Claims (PVCs) are created.
  - To view the state of the newly created OSDs:
    a. Click **Workloads → Pods** from the OpenShift Web Console.
    b. Select **openshift-storage** from the **Project** drop-down list.



NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

- To view the state of the PVCs:

  a. Click **Storage → Persistent Volume Claims** from the OpenShift Web Console.

  b. Select **openshift-storage** from the **Project** drop-down list.



NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

- Optional: If cluster-wide encryption is enabled on the cluster, verify that the new OSD devices are encrypted.

  a. Identify the nodes where the new OSD pods are running.

```
$ oc get -n openshift-storage -o=custom-columns=NODE:.spec.nodeName pod/<OSD-pod-name>
```

<OSD-pod-name>

Is the name of the OSD pod.
For example:

```
$ oc get -n openshift-storage -o=custom-columns=NODE:.spec.nodeName pod/rook-ceph-osd-0-544db49d7f-qrgqm
```

Example output:

```
NODE
compute-1
```

  b. For each of the nodes identified in the previous step, do the following:

    i. Create a debug pod and open a chroot environment for the selected hosts.

```
$ oc debug node/<node-name>
```

<node-name>

Is the name of the node.

```
$ chroot /host
```

    ii. Check for the **crypt** keyword beside the **ocs-deviceset** names.

```
$ lsblk
```



IMPORTANT

Cluster reduction is supported only with the Red Hat Support Team's assistance.

14.3. SCALING OUT STORAGE CAPACITY BY ADDING NEW NODES

To scale out storage capacity, you need to perform the following:

- Add a new node to increase the storage capacity when existing worker nodes are already running at their maximum supported OSDs, which is the increment of 3 OSDs of the capacity selected during initial configuration.
- Verify that the new node is added successfully
- Scale up the storage capacity after the node is added

14.3.1. Adding a node to an installer-provisioned infrastructure

Use this procedure to add a new node to an installer-provisioned infrastructure cluster.

Prerequisites

- You have administrative privilege to the OpenShift Container Platform Console.
- You have a running OpenShift Data Foundation Storage Cluster.

Procedure

1. Navigate to **Compute → Machine Sets**.
2. On the machine set where you want to add nodes, select **Edit Machine Count**
   a. Add the amount of nodes, and click **Save**.
   b. Click **Compute → Nodes** and confirm if the new node is in **Ready** state.
3. Apply the OpenShift Data Foundation label to the new node.
   a. For the new node, click **Action menu (⋮) → Edit Labels**.
   b. Add **cluster.ocs.openshift.io/openshift-storage**, and click **Save**.



NOTE

It is recommended to add 3 nodes, one each in different zones. You must add 3 nodes and perform this procedure for all of them. In case of bare metal installer-provisioned infrastructure deployment, you must expand the cluster first. For instructions, see Expanding the cluster.

Verification

1. Execute the following command in the terminal and verify that the new node is present in the output:

```
$ oc get nodes --show-labels | grep cluster.ocs.openshift.io/openshift-storage= |cut -d' ' -f1
```

2. On the OpenShift web console, click **Workloads → Pods**, confirm that at least the following pods on the new node are in **Running** state:

- **openshift-storage.cephfs.csi.ceph.com-***
- **openshift-storage.rbd.csi.ceph.com-***

14.4. SCALING UP STORAGE CAPACITY

After you add a new node to OpenShift Data Foundation, you must scale up the storage capacity as described in Scaling up storage by adding capacity .

CHAPTER 15. MULTICLOUD OBJECT GATEWAY

The Multicloud Object Gateway (MCG) provides S3-compatible object storage that spans on-premise and cloud environments, with unified access through a single namespace.

15.1. ABOUT THE MULTICLOUD OBJECT GATEWAY

The Multicloud Object Gateway (MCG) is a lightweight object storage service for OpenShift, allowing users to start small and then scale as needed on-premise, in multiple clusters, and with cloud-native storage.

15.2. ACCESSING THE MULTICLOUD OBJECT GATEWAY WITH YOUR APPLICATIONS

You can access the object service with any application targeting AWS S3 or code that uses AWS S3 Software Development Kit (SDK). Applications need to specify the Multicloud Object Gateway (MCG) endpoint, an access key, and a secret access key. You can use your terminal or the MCG CLI to retrieve this information.

Prerequisites

- A running OpenShift Data Foundation Platform.

Additional resources

- Accessing the RADOS Object Gateway S3 endpoint

15.2.1. Accessing the RADOS Object Gateway S3 endpoint

You can access the RADOS Object Gateway (RGW) endpoint directly. The RGW route is created by default and is named **rook-ceph-rgw-ocs-storagecluster-cephobjectstore**.

For information about RGW S3 supported and unsupported verbs, see S3 supported and unsupported verbs in the *Red Hat Ceph Storage Developer Guide* .

15.3. ADDING STORAGE RESOURCES FOR HYBRID OR MULTICLOUD

Connect cloud and on-premises storage providers to the Multicloud Object Gateway by adding backing stores for AWS, Azure, GCP, IBM COS, or local persistent volumes.

15.3.1. Creating a new backing store

Use this procedure to create a new backing store in OpenShift Data Foundation.

Prerequisites

- Administrator access to OpenShift Data Foundation.

Procedure

1. In the OpenShift Web Console, click **Storage → Object Storage**.
2. Click the **Backing Store** tab.

3. Click **Create Backing Store**.

4. On the **Create New Backing Store** page, perform the following:

   a. Enter a **Backing Store Name**.

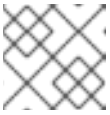
   b. Select a **Provider**.

   c. Select a **Region**.

   d. Optional: Enter an **Endpoint**.

   e. Select a **Secret** from the drop-down list, or create your own secret. Optionally, you can **Switch to Credentials** view which lets you fill in the required secrets.
   For more information on creating an OCP secret, see the section Creating the secret in the *Openshift Container Platform* documentation.

   Each backingstore requires a different secret. For more information on creating the secret for a particular backingstore, see the Section 15.3.2, "Adding storage resources for hybrid or Multicloud using the command-line interface" and follow the procedure for the addition of storage resources using a YAML.

   NOTE

   This menu is relevant for all providers except Google Cloud and local PVC.

   f. Enter the **Target bucket**. The target bucket is a container storage that is hosted on the remote cloud service. It allows you to create a connection that tells the MCG that it can use this bucket for the system.

5. Click **Create Backing Store**.

Verification

1. In the OpenShift Web Console, click **Storage → Object Storage**.

2. Click the **Backing Store** tab to view all the backing stores.

15.3.2. Adding storage resources for hybrid or Multicloud using the command-line interface

The Multicloud Object Gateway (MCG) simplifies the process of spanning data across cloud provider and clusters.

You must add a backing storage that can be used by the MCG.

Depending on the type of your deployment, you can choose one of the following procedures to create a backing storage:

- For creating an AWS-backed backingstore, see Section 15.3.2.1, "Creating an AWS-backed backingstore"
- For creating an IBM COS-backed backingstore, see Section 15.3.2.2, "Creating an IBM COS-backed backingstore"

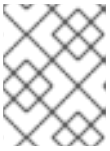
- For creating an Azure-backed backingstore, see Section 15.3.2.3, "Creating an Azure-backed backingstore"
- For creating a GCP-backed backingstore, see Section 15.3.2.4, "Creating a GCP-backed backingstore"
- For creating a local Persistent Volume-backed backingstore, see Section 15.3.2.5, "Creating a local Persistent Volume-backed backingstore"

For VMware deployments, skip to Section 15.3.3, "Creating an s3 compatible Multicloud Object Gateway backingstore" for further instructions.

15.3.2.1. Creating an AWS-backed backingstore

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

- From the command-line interface, run the following command:

```
odf noobaa backingstore create aws-s3 <backingstore_name> --access-key=<AWS
ACCESS KEY> --secret-key=<AWS SECRET ACCESS KEY> --target-bucket <bucket-
name> -n openshift-storage
```

<backingstore_name>

The name of the backingstore.

<AWS ACCESS KEY> and <AWS SECRET ACCESS KEY>

The AWS access key ID and secret access key you created for this purpose.

<bucket-name>

The existing AWS bucket name. This argument indicates to the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.
The output will be similar to the following:

```
INFO[0001]   Exists: NooBaa "noobaa"
INFO[0002]   Created: BackingStore "aws-resource"
INFO[0002]   Created: Secret "backing-store-secret-aws-resource"
```

1. Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <backingstore-secret-name>
  namespace: openshift-storage
type: Opaque
```

```
data:
  AWS_ACCESS_KEY_ID: <AWS ACCESS KEY ID ENCODED IN BASE64>
  AWS_SECRET_ACCESS_KEY: <AWS SECRET ACCESS KEY ENCODED IN
BASE64>
```

<AWS ACCESS KEY> and ***<AWS SECRET ACCESS KEY>***

Supply and encode your own AWS access key ID and secret access key using Base64, and use the results for **<AWS ACCESS KEY ID ENCODED IN BASE64>** and **<AWS SECRET ACCESS KEY ENCODED IN BASE64>**.

<backingstore-secret-name>

The name of the backingstore secret created in the previous step.

1. Apply the following YAML for a specific backing store:

```
apiVersion: noobaa.io/v1alpha1
kind: BackingStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: bs
  namespace: openshift-storage
spec:
  awsS3:
    secret:
      name: <backingstore-secret-name>
      namespace: openshift-storage
    targetBucket: <bucket-name>
  type: aws-s3
```

<bucket-name>

The existing AWS bucket name.

<backingstore-secret-name>

The name of the backingstore secret created in the previous step.

15.3.2.2. Creating an IBM COS-backed backingstore

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

1. From the command-line interface, run the following command:

```
odf noobaa backingstore create ibm-cos <backingstore_name> --access-key=<IBM
ACCESS KEY> --secret-key=<IBM SECRET ACCESS KEY> --endpoint=<IBM COS
ENDPOINT> --target-bucket <bucket-name> -n openshift-storage
```

<backingstore_name>

The name of the backingstore.

<IBM ACCESS KEY>*, *<IBM SECRET ACCESS KEY>*, and *<IBM COS ENDPOINT>

An IBM access key ID, secret access key and the appropriate regional endpoint that corresponds to the location of the existing IBM bucket.

To generate the above keys on IBM cloud, you must include HMAC credentials while creating the service credentials for your target bucket.

<bucket-name>

An existing IBM bucket name. This argument indicates MCG about the bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.

The output will be similar to the following:

```
INFO[0001]   Exists: NooBaa "noobaa"
INFO[0002]   Created: BackingStore "ibm-resource"
INFO[0002]   Created: Secret "backing-store-secret-ibm-resource"
```

2. Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <backingstore-secret-name>
  namespace: openshift-storage
type: Opaque
data:
  IBM_COS_ACCESS_KEY_ID: <IBM COS ACCESS KEY ID ENCODED IN BASE64>
  IBM_COS_SECRET_ACCESS_KEY: <IBM COS SECRET ACCESS KEY ENCODED IN
BASE64>
```

<IBM COS ACCESS KEY ID ENCODED IN BASE64> and <IBM COS SECRET ACCESS KEY ENCODED IN BASE64>

Provide and encode your own IBM COS access key ID and secret access key using Base64, and use the results in place of these attributes respectively.

<backingstore-secret-name>

The name of the backingstore secret.

3. Apply the following YAML for a specific backing store:

```
apiVersion: noobaa.io/v1alpha1
kind: BackingStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: bs
```

```
  namespace: openshift-storage
spec:
  ibmCos:
    endpoint: <endpoint>
    secret:
      name: <backingstore-secret-name>
      namespace: openshift-storage
    targetBucket: <bucket-name>
  type: ibm-cos
```

<bucket-name>

an existing IBM COS bucket name. This argument indicates to MCG about the bucket to use as a target bucket for its backingstore, and subsequently, data storage and administration.

<endpoint>

A regional endpoint that corresponds to the location of the existing IBM bucket name. This argument indicates to MCG about the endpoint to use for its backingstore, and subsequently, data storage and administration.

<backingstore-secret-name>

The name of the secret created in the previous step.

15.3.2.3. Creating an Azure-backed backingstore

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

- From the command-line interface, run the following command:

```
odf noobaa backingstore create azure-blob <backingstore_name> --account-key=<AZURE
ACCOUNT KEY> --account-name=<AZURE ACCOUNT NAME> --target-blob-container
<blob container name> -n openshift-storage
```

<backingstore_name>

The name of the backingstore.

<AZURE ACCOUNT KEY> and **<AZURE ACCOUNT NAME>**

An AZURE account key and account name you created for this purpose.

<blob container name>

An existing Azure blob container name. This argument indicates to MCG about the bucket to use as a target bucket for its backingstore, and subsequently, data storage and administration.

The output will be similar to the following:

```
INFO[0001]   Exists: NooBaa "noobaa"
INFO[0002]   Created: BackingStore "azure-resource"
INFO[0002]   Created: Secret "backing-store-secret-azure-resource"
```

1. Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <backingstore-secret-name>
type: Opaque
data:
  AccountName: <AZURE ACCOUNT NAME ENCODED IN BASE64>
  AccountKey: <AZURE ACCOUNT KEY ENCODED IN BASE64>
```

<AZURE ACCOUNT NAME ENCODED IN BASE64> and <AZURE ACCOUNT KEY ENCODED IN BASE64>

Supply and encode your own Azure Account Name and Account Key using Base64, and use the results in place of these attributes respectively.

<backingstore-secret-name>

A unique name of backingstore secret.

1. Apply the following YAML for a specific backing store:

```
apiVersion: noobaa.io/v1alpha1
kind: BackingStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: bs
  namespace: openshift-storage
spec:
  azureBlob:
    secret:
      name: <backingstore-secret-name>
      namespace: openshift-storage
    targetBlobContainer: <blob-container-name>
  type: azure-blob
```

<blob-container-name>

An existing Azure blob container name. This argument indicates to the MCG about the bucket to use as a target bucket for its backingstore, and subsequently, data storage and administration.

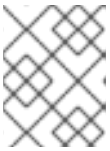
<backingstore-secret-name>

with the name of the secret created in the previous step.

15.3.2.4. Creating a GCP-backed backingstore

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

- From the command-line interface, run the following command:

```
odf noobaa backingstore create google-cloud-storage <backingstore_name> --private-key-json-file=<PATH TO GCP PRIVATE KEY JSON FILE> --target-bucket <GCP bucket name> -n openshift-storage
```

<backingstore_name>

Name of the backingstore.

<PATH TO GCP PRIVATE KEY JSON FILE>

A path to your GCP private key created for this purpose.

<GCP bucket name>

An existing GCP object storage bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.
The output will be similar to the following:

```
INFO[0001]   Exists: NooBaa "noobaa"
INFO[0002]   Created: BackingStore "google-gcp"
INFO[0002]   Created: Secret "backing-store-google-cloud-storage-gcp"
```

1. Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <backingstore-secret-name>
type: Opaque
data:
  GoogleServiceAccountPrivateKeyJson: <GCP PRIVATE KEY ENCODED IN BASE64>
```

<GCP PRIVATE KEY ENCODED IN BASE64>

Provide and encode your own GCP service account private key using Base64, and use the results for this attribute.

<backingstore-secret-name>

A unique name of the backingstore secret.

1. Apply the following YAML for a specific backing store:

```
apiVersion: noobaa.io/v1alpha1
kind: BackingStore
```

```
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: bs
  namespace: openshift-storage
spec:
  googleCloudStorage:
    secret:
      name: <backingstore-secret-name>
      namespace: openshift-storage
    targetBucket: <target bucket>
  type: google-cloud-storage
```

<target bucket>

An existing Google storage bucket. This argument indicates to the MCG about the bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.

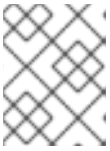
<backingstore-secret-name>

The name of the secret created in the previous step.

15.3.2.5. Creating a local Persistent Volume-backed backingstore

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.

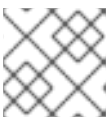


NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

- From the command-line interface, run the following command:



NOTE

This command must be run from within the **openshift-storage** namespace.

```
$ odf noobaa -n openshift-storage backingstore create pv-pool <backingstore_name> --num-volumes <NUMBER OF VOLUMES>  --pv-size-gb <VOLUME SIZE> --request-cpu <CPU REQUEST> --request-memory <MEMORY REQUEST> --limit-cpu <CPU LIMIT> --limit-memory <MEMORY LIMIT> --storage-class <LOCAL STORAGE CLASS>
```

- Apply the following YAML for a specific backing store:

```
apiVersion: noobaa.io/v1alpha1
kind: BackingStore
metadata:
```

```
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: <backingstore_name>
  namespace: openshift-storage
spec:
  pvPool:
    numVolumes: <NUMBER OF VOLUMES>
    resources:
      requests:
        storage: <VOLUME SIZE>
        cpu: <CPU REQUEST>
        memory: <MEMORY REQUEST>
      limits:
        cpu: <CPU LIMIT>
        memory: <MEMORY LIMIT>
    storageClass: <LOCAL STORAGE CLASS>
  type: pv-pool
```

<backingstore_name>

The name of the backingstore.

<NUMBER OF VOLUMES>

The number of volumes you would like to create. Note that increasing the number of volumes scales up the storage.

<VOLUME SIZE>

Required size in GB of each volume.

<CPU REQUEST>

Guaranteed amount of CPU requested in CPU unit **m**.

<MEMORY REQUEST>

Guaranteed amount of memory requested.

<CPU LIMIT>

Maximum amount of CPU that can be consumed in CPU unit **m**.

<MEMORY LIMIT>

Maximum amount of memory that can be consumed.

<LOCAL STORAGE CLASS>

The local storage class name, recommended to use **ocs-storagecluster-ceph-rbd**.

The output will be similar to the following:

```
INFO[0001]   Exists: NooBaa "noobaa"
INFO[0002]   Exists: BackingStore "local-mcg-storage"
```

15.3.3. Creating an s3 compatible Multicloud Object Gateway backingstore

The Multicloud Object Gateway (MCG) can use any S3 compatible object storage as a backing store, for example, Red Hat Ceph Storage's RADOS Object Gateway (RGW). The following procedure shows how to create an S3 compatible MCG backing store for Red Hat Ceph Storage's RGW. Note that when

the RGW is deployed, OpenShift Data Foundation operator creates an S3 compatible backingstore for MCG automatically.

Procedure

1. From the command-line interface, run the following command:



NOTE

This command must be run from within the **openshift-storage** namespace.

```
odf noobaa backingstore create s3-compatible rgw-resource --access-key=<RGW ACCESS KEY> --secret-key=<RGW SECRET KEY> --target-bucket=<bucket-name> --endpoint=<RGW endpoint> -n openshift-storage
```

   a. To get the **<RGW ACCESS KEY>** and **<RGW SECRET KEY>**, run the following command using your RGW user secret name:

```
oc get secret <RGW USER SECRET NAME> -o yaml -n openshift-storage
```

   b. Decode the access key ID and the access key from Base64 and keep them.

   c. Replace **<RGW USER ACCESS KEY>** and **<RGW USER SECRET ACCESS KEY>** with the appropriate, decoded data from the previous step.

   d. Replace **<bucket-name>** with an existing RGW bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.

   e. To get the **<RGW endpoint>**, see Accessing the RADOS Object Gateway S3 endpoint . The output will be similar to the following:

```
INFO[0001]   Exists: NooBaa "noobaa"
INFO[0002]   Created: BackingStore "rgw-resource"
INFO[0002]   Created: Secret "backing-store-secret-rgw-resource"
```

2. Alternatively, create the backingstore using a YAML. Create a **CephObjectStore** user. This also creates a secret containing the RGW credentials:

```
apiVersion: ceph.rook.io/v1
kind: CephObjectStoreUser
metadata:
  name: <RGW-Username>
  namespace: openshift-storage
spec:
  store: ocs-storagecluster-cephobjectstore
  displayName: "<Display-name>"
```

   a. Replace **<RGW-Username>** and **<Display-name>** with a unique username and display name.

3. Apply the following YAML for an S3-Compatible backing store:

```
apiVersion: noobaa.io/v1alpha1
kind: BackingStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: <backingstore-name>
  namespace: openshift-storage
spec:
  s3Compatible:
    endpoint: <RGW endpoint>
    secret:
      name: <backingstore-secret-name>
      namespace: openshift-storage
    signatureVersion: v4
    targetBucket: <RGW-bucket-name>
  type: s3-compatible
```

a. Replace **<backingstore-secret-name>** with the name of the secret that was created with **CephObjectStore** in the previous step.

b. Replace **<bucket-name>** with an existing RGW bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.

c. To get the **<RGW endpoint>**, see Accessing the RADOS Object Gateway S3 endpoint .

15.3.4. Adding storage resources for hybrid and Multicloud using the user interface

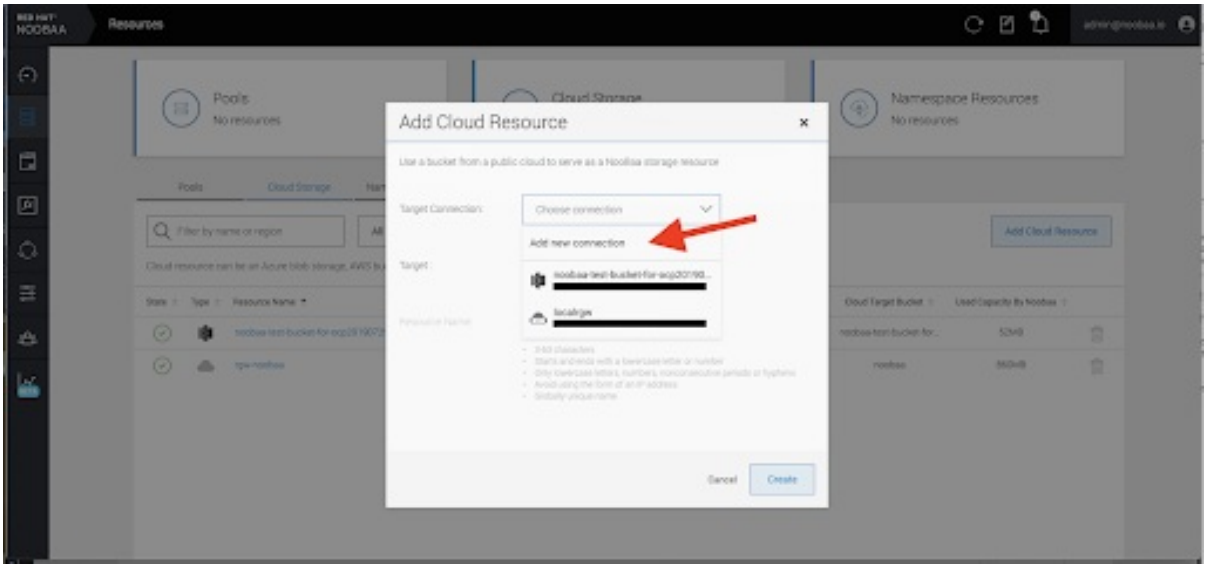
Use this procedure to add storage resources for hybrid and multicloud environments using the user interface.

Procedure

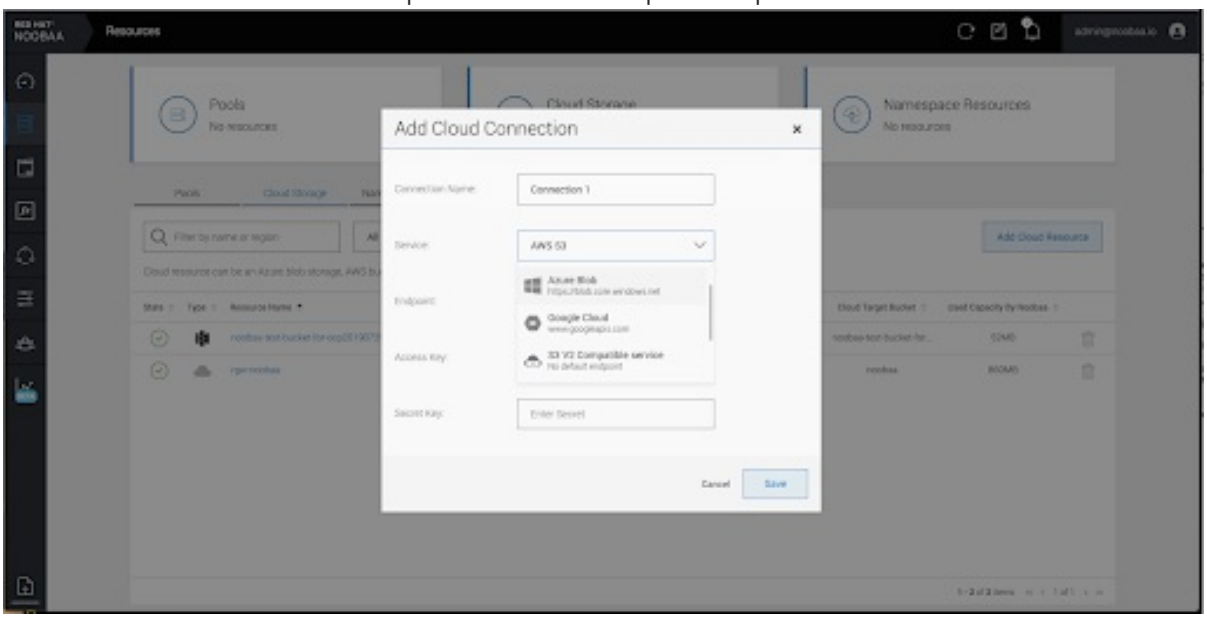
1. In the OpenShift Web Console, click **Storage → Data Foundation**.
2. In the **Storage Systems** tab, select the storage system and then click **Overview → Object** tab.
3. Select the **Multicloud Object Gateway** link.
1. Select the **Resources** tab in the left, highlighted below. From the list that populates, select **Add Cloud Resource**.



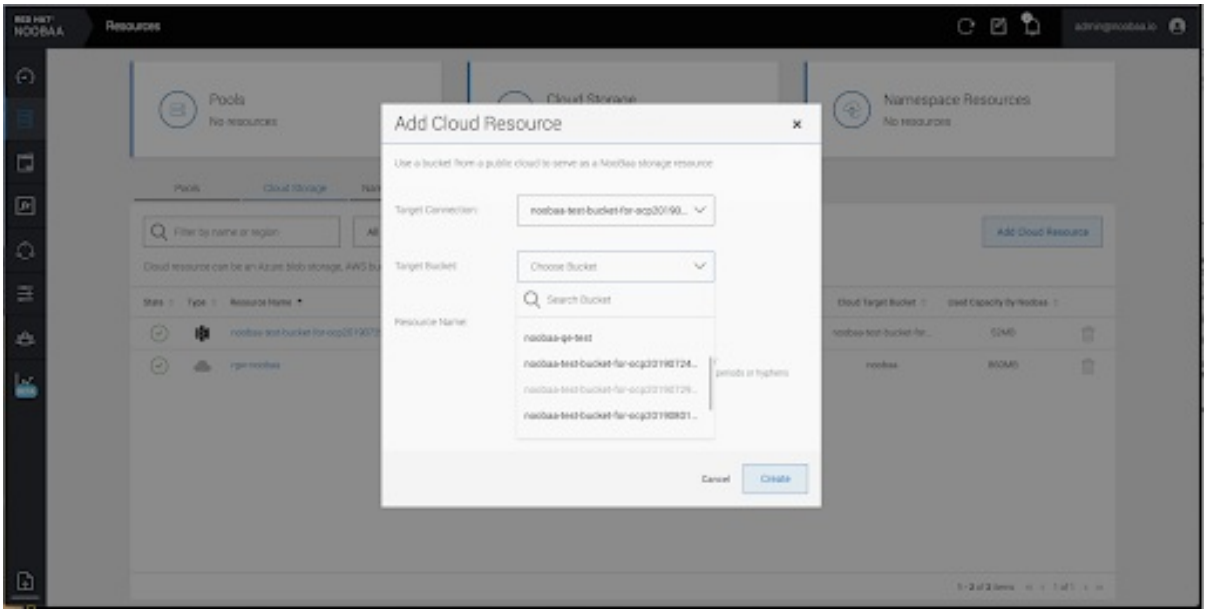
2. Select **Add new connection**.



3. Select the relevant native cloud provider or S3 compatible option and fill in the details.



4. Select the newly created connection and map it to the existing bucket.



5. Repeat these steps to create as many backing stores as needed.



NOTE

Resources created in NooBaa UI cannot be used by OpenShift UI or MCG CLI.

15.3.5. Creating a new bucket class

Bucket class is a CRD representing a class of buckets that defines tiering policies and data placements for an Object Bucket Class.

Use this procedure to create a bucket class in OpenShift Data Foundation.

Procedure

1. In the OpenShift Web Console, click **Storage → Object Storage**.
2. Click the **Bucket Class** tab.
3. Click **Create Bucket Class**.
4. On the **Create new Bucket Class** page, perform the following:
   a. Select the bucket class type and enter a bucket class name.
      i. Select the **BucketClass type**. Choose one of the following options:
         - **Standard**: data will be consumed by a Multicloud Object Gateway (MCG), deduped, compressed and encrypted.
         - **Namespace**: data is stored on the NamespaceStores without performing de-duplication, compression or encryption.
         - **Vector**: specialized bucket class for storing vector embeddings for machine learning and AI applications. Vector bucket classes require an NSFS NamespaceStore and support only the Lance vector database type.
           By default, **Standard** is selected.
      ii. Enter a **Bucket Class Name**.
      iii. Click **Next**.

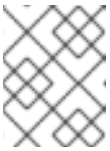


NOTE

If you select **Vector** as the bucket class type, the Placement Policy step is hidden, and you will be prompted to select an NSFS NamespaceStore resource. For more information about vector bucket classes, see Creating a vector BucketClass .

   b. In **Placement Policy**, select **Tier 1 - Policy Type** and click **Next**. You can choose either one of the options as per your requirements.
      - **Spread** allows spreading of the data across the chosen resources.
      - **Mirror** allows full duplication of the data across the chosen resources.
      - Click **Add Tier** to add another policy tier.

c. Select at least one **Backing Store** resource from the available list if you have selected **Tier 1 - Policy Type** as **Spread** and click **Next**. Alternatively, you can also create a new backing store.



NOTE

You need to select at least 2 backing stores when you select Policy Type as Mirror in previous step.

d. Review and confirm Bucket Class settings.

e. Click **Create Bucket Class**.

Verification

1. In the OpenShift Web Console, click **Storage → Object Storage**.
2. Click the **Bucket Class** tab and search the new Bucket Class.

15.3.6. Editing a bucket class

Use the following procedure to edit the bucket class components through the YAML file by clicking the **edit** button on the Openshift web console.

Prerequisites

- Administrator access to OpenShift Web Console.

Procedure

1. In the OpenShift Web Console, click **Storage → Object Storage**.
2. Click the **Bucket Class** tab.
3. Click the Action Menu **(⋮)** next to the Bucket class you want to edit.
4. Click **Edit Bucket Class**.
5. You are redirected to the **YAML** file, make the required changes in this file and click **Save**.

15.3.7. Editing backing stores for bucket class

Use the following procedure to edit an existing Multicloud Object Gateway (MCG) bucket class to change the underlying backing stores used in a bucket class.

Prerequisites

- Administrator access to OpenShift Web Console.
- A bucket class.
- Backing stores.

Procedure

1. In the OpenShift Web Console, click **Storage → Object Storage**.
2. Click the **Bucket Class** tab.
3. Click the Action Menu **(⋮)** next to the Bucket class you want to edit.
4. Click **Edit Bucket Class Resources**.
5. On the **Edit Bucket Class Resources** page, edit the bucket class resources either by adding a backing store to the bucket class or by removing a backing store from the bucket class. You can also edit bucket class resources created with one or two tiers and different placement policies.
   - To add a backing store to the bucket class, select the name of the backing store.
   - To remove a backing store from the bucket class, uncheck the name of the backing store.
6. Click **Save**.

15.4. MANAGING NAMESPACE BUCKETS

Namespace buckets unify object storage across multiple cloud providers, letting you read from several sources and write to your preferred provider through a single bucket interface.

Namespace buckets let you connect data repositories on different providers together, so you can interact with all of your data through a single unified view. Add the object bucket associated with each provider to the namespace bucket, and access your data through the namespace bucket to see all of your object buckets at once. This lets you write to your preferred storage provider while reading from multiple other storage providers, greatly reducing the cost of migrating to a new storage provider.

You can interact with objects in a namespace bucket using the S3 API. See S3 API endpoints for objects in namespace buckets for more information.



NOTE

A namespace bucket can only be used if its write target is available and functional.

15.4.1. Amazon S3 API endpoints for objects in namespace buckets

You can interact with objects in the namespace buckets using the Amazon Simple Storage Service (S3) API.

Ensure that the credentials provided for the Multicloud Object Gateway (MCG) enables you to perform the AWS S3 namespace bucket operations. You can use the AWS tool, **aws-cli** to verify that all the operations can be performed on the target bucket. Also, the list bucket which is using this MCG account shows the target bucket.

Red Hat OpenShift Data Foundation supports the following namespace bucket operations:

- ListBuckets
- ListObjects
- ListMultipartUploads
- ListObjectVersions

- GetObject
- HeadObject
- CopyObject
- PutObject
- CreateMultipartUpload
- UploadPartCopy
- UploadPart
- ListParts
- AbortMultipartUpload
- PubObjectTagging
- DeleteObjectTagging
- GetObjectTagging
- GetObjectAcl
- PutObjectAcl
- DeleteObject
- DeleteObjects

See the Amazon S3 API reference documentation for the most up-to-date information about these operations and how to use them.

Additional resources

- Amazon S3 REST API Reference
- Amazon S3 CLI Reference

15.4.2. Adding a namespace bucket using the Multicloud Object Gateway CLI and YAML

Namespace buckets aggregate object storage from multiple providers into a unified view. Use the Multicloud Object Gateway CLI or YAML to add AWS S3 or IBM COS namespace buckets.

For more information about namespace buckets, see Managing namespace buckets.

Depending on the type of your deployment and whether you want to use YAML or the Multicloud Object Gateway CLI, choose one of the following procedures to add a namespace bucket:

- Adding an AWS S3 namespace bucket using YAML
- Adding an IBM COS namespace bucket using YAML
- Adding an AWS S3 namespace bucket using the Multicloud Object Gateway CLI

- Adding an IBM COS namespace bucket using the Multicloud Object Gateway CLI

15.4.2.1. Adding an AWS S3 namespace bucket using YAML

Prerequisites

- Openshift Container Platform with OpenShift Data Foundation operator installed.
- Access to the Multicloud Object Gateway (MCG).
  For information, see Chapter 2, Accessing the Multicloud Object Gateway with your applications.

Procedure

1. Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <namespacestore-secret-name>
  type: Opaque
data:
  AWS_ACCESS_KEY_ID: <AWS ACCESS KEY ID ENCODED IN BASE64>
  AWS_SECRET_ACCESS_KEY: <AWS SECRET ACCESS KEY ENCODED IN BASE64>
```

   where **<namespacestore-secret-name>** is a unique NamespaceStore name.

   You must provide and encode your own AWS access key ID and secret access key using **Base64**, and use the results in place of **<AWS ACCESS KEY ID ENCODED IN BASE64>** and **<AWS SECRET ACCESS KEY ENCODED IN BASE64>**.

2. Create a NamespaceStore resource using OpenShift custom resource definitions (CRDs).
   A NamespaceStore represents underlying storage to be used as a **read** or **write** target for the data in the MCG namespace buckets.

   To create a NamespaceStore resource, apply the following YAML:

```
apiVersion: noobaa.io/v1alpha1
kind: NamespaceStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: <resource-name>
  namespace: openshift-storage
spec:
  awsS3:
    secret:
      name: <namespacestore-secret-name>
      namespace: <namespace-secret>
    targetBucket: <target-bucket>
  type: aws-s3
```

<resource-name>

The name you want to give to the resource.

<namespacestore-secret-name>

The secret created in the previous step.

<namespace-secret>

The namespace where the secret can be found.

<target-bucket>

The target bucket you created for the NamespaceStore.

3. Create a namespace bucket class that defines a namespace policy for the namespace buckets. The namespace policy requires a type of either **single** or **multi**.

- A namespace policy of type **single** requires the following configuration:

```
apiVersion: noobaa.io/v1alpha1
kind: BucketClass
metadata:
  labels:
    app: noobaa
  name: <my-bucket-class>
  namespace: openshift-storage
spec:
  namespacePolicy:
    type: Single
    single:
      resource: <resource>
```

<my-bucket-class>

The unique namespace bucket class name.

<resource>

The name of a single NamespaceStore that defines the read and write target of the namespace bucket.

- A namespace policy of type **multi** requires the following configuration:

```
apiVersion: noobaa.io/v1alpha1

kind: BucketClass
metadata:
  labels:
    app: noobaa
  name: <my-bucket-class>
  namespace: openshift-storage
spec:
  namespacePolicy:
    type: Multi
    multi:
      writeResource: <write-resource>
      readResources:
      - <read-resources>
      - <read-resources>
```

<my-bucket-class>

A unique bucket class name.

<write-resource>

The name of a single NamespaceStore that defines the **write** target of the namespace bucket.

<read-resources>

A list of the names of the NamespaceStores that defines the **read** targets of the namespace bucket.

4. Create a bucket using an Object Bucket Class (OBC) resource that uses the bucket class defined in the earlier step using the following YAML:

```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  name: <resource-name>
  namespace: openshift-storage
spec:
  generateBucketName: <my-bucket>
  storageClassName: openshift-storage.noobaa.io
  additionalConfig:
    bucketclass: <my-bucket-class>
```

<resource-name>

The name you want to give to the resource.

<my-bucket>

The name you want to give to the bucket.

<my-bucket-class>

The bucket class created in the previous step.
After the OBC is provisioned by the operator, a bucket is created in the MCG, and the operator creates a **Secret** and **ConfigMap** with the same name and in the same namespace as that of the OBC.

15.4.2.2. Adding an IBM COS namespace bucket using YAML

Prerequisites

- Openshift Container Platform with OpenShift Data Foundation operator installed.
- Access to the Multicloud Object Gateway (MCG), see Chapter 2, Accessing the Multicloud Object Gateway with your applications.

Procedure

1. Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <namespacestore-secret-name>
```

```
  type: Opaque
data:
  IBM_COS_ACCESS_KEY_ID: <IBM COS ACCESS KEY ID ENCODED IN BASE64>
  IBM_COS_SECRET_ACCESS_KEY: <IBM COS SECRET ACCESS KEY ENCODED IN
BASE64>
```

<namespacestore-secret-name>

A unique NamespaceStore name.
You must provide and encode your own IBM COS access key ID and secret access key using **Base64**, and use the results in place of **<IBM COS ACCESS KEY ID ENCODED IN BASE64>** and **<IBM COS SECRET ACCESS KEY ENCODED IN BASE64>**.

2. Create a NamespaceStore resource using OpenShift custom resource definitions (CRDs).
A NamespaceStore represents underlying storage to be used as a **read** or **write** target for the data in the MCG namespace buckets.

   To create a NamespaceStore resource, apply the following YAML:

```
apiVersion: noobaa.io/v1alpha1
kind: NamespaceStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: bs
  namespace: openshift-storage
spec:
  s3Compatible:
    endpoint: <IBM COS ENDPOINT>
    secret:
      name: <namespacestore-secret-name>
      namespace: <namespace-secret>
    signatureVersion: v2
    targetBucket: <target-bucket>
  type: ibm-cos
```

<IBM COS ENDPOINT>

The appropriate IBM COS endpoint.

<namespacestore-secret-name>

The secret created in the previous step.

<namespace-secret>

The namespace where the secret can be found.

<target-bucket>

The target bucket you created for the NamespaceStore.

3. Create a namespace bucket class that defines a namespace policy for the namespace buckets. The namespace policy requires a type of either **single** or **multi**.

   - The namespace policy of type **single** requires the following configuration:

```
apiVersion: noobaa.io/v1alpha1
kind: BucketClass
metadata:
  labels:
    app: noobaa
  name: <my-bucket-class>
  namespace: openshift-storage
spec:
  namespacePolicy:
    type: Single
    single:
      resource: <resource>
```

<my-bucket-class>

The unique namespace bucket class name.

<resource>

The name of a single NamespaceStore that defines the **read** and **write** target of the namespace bucket.

- The namespace policy of type **multi** requires the following configuration:

```
apiVersion: noobaa.io/v1alpha1
kind: BucketClass
metadata:
  labels:
    app: noobaa
  name: <my-bucket-class>
  namespace: openshift-storage
spec:
  namespacePolicy:
    type: Multi
    multi:
      writeResource: <write-resource>
      readResources:
      - <read-resources>
      - <read-resources>
```

<my-bucket-class>

The unique bucket class name.

<write-resource>

The name of a single NamespaceStore that defines the write target of the namespace bucket.

<read-resources>

A list of the NamespaceStores names that defines the **read** targets of the namespace bucket.

4. To create a bucket using an Object Bucket Class (OBC) resource that uses the bucket class defined in the previous step, apply the following YAML:

```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
```

```
  name: <resource-name>
  namespace: openshift-storage
spec:
  generateBucketName: <my-bucket>
  storageClassName: openshift-storage.noobaa.io
  additionalConfig:
    bucketclass: <my-bucket-class>
```

<resource-name>

The name you want to give to the resource.

<my-bucket>

The name you want to give to the bucket.

<my-bucket-class>

The bucket class created in the previous step.
After the OBC is provisioned by the operator, a bucket is created in the MCG, and the operator creates a **Secret** and **ConfigMap** with the same name and in the same namespace as that of the OBC.

15.4.2.3. Adding an AWS S3 namespace bucket using the Multicloud Object Gateway CLI

Prerequisites

- Openshift Container Platform with OpenShift Data Foundation operator installed.
- Access to the Multicloud Object Gateway (MCG), see Chapter 2, Accessing the Multicloud Object Gateway with your applications.
- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS

Procedure

1. In the command-line interface, create a NamespaceStore resource.
   A NamespaceStore represents an underlying storage to be used as a **read** or **write** target for the data in MCG namespace buckets.

```
$ odf noobaa namespacestore create aws-s3 <namespacestore> --access-key <AWS ACCESS KEY> --secret-key <AWS SECRET ACCESS KEY> --target-bucket <bucket-name> -n openshift-storage
```

<namespacestore>

The name of the NamespaceStore.

<AWS ACCESS KEY> and <AWS SECRET ACCESS KEY>

The AWS access key ID and secret access key you created for this purpose.

<bucket-name>

The existing AWS bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.

2. Create a namespace bucket class that defines a namespace policy for the namespace buckets. The namespace policy can be either **single** or **multi**.

   - To create a namespace bucket class with a namespace policy of type **single**:

     ```
     $ odf noobaa bucketclass create namespace-bucketclass single <my-bucket-class> --resource <resource> -n openshift-storage
     ```

     <resource-name>

     The name you want to give the resource.

     <my-bucket-class>

     A unique bucket class name.

     <resource>

     A single namespace-store that defines the **read** and **write** target of the namespace bucket.

   - To create a namespace bucket class with a namespace policy of type **multi**:

     ```
     $ odf noobaa bucketclass create namespace-bucketclass multi <my-bucket-class> --write-resource <write-resource> --read-resources <read-resources> -n openshift-storage
     ```

     <resource-name>

     The name you want to give the resource.

     <my-bucket-class>

     A unique bucket class name.

     <write-resource>

     A single namespace-store that defines the **write** target of the namespace bucket.

     <read-resources>s

     A list of namespace-stores separated by commas that defines the **read** targets of the namespace bucket.

3. Create a bucket using an Object Bucket Class (OBC) resource that uses the bucket class defined in the previous step.

   ```
   $ odf noobaa obc create my-bucket-claim -n openshift-storage --app-namespace my-app --bucketclass <custom-bucket-class>
   ```

   <bucket-name>

   A bucket name of your choice.

   <custom-bucket-class>

   The name of the bucket class created in the previous step.

   After the OBC is provisioned by the operator, a bucket is created in the MCG, and the operator creates a **Secret** and a **ConfigMap** with the same name and in the same namespace as that of the OBC.

15.4.2.4. Adding an IBM COS namespace bucket using the Multicloud Object Gateway CLI

Prerequisites

- Openshift Container Platform with OpenShift Data Foundation operator installed.
- Access to the Multicloud Object Gateway (MCG), see Chapter 2, Accessing the Multicloud Object Gateway with your applications.
- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose either Linux(x86_64), Windows, or Mac OS.

Procedure

1. In the command-line interface, create a NamespaceStore resource.
   A NamespaceStore represents an underlying storage to be used as a **read** or **write** target for the data in the MCG namespace buckets.

   ```
   $ odf noobaa namespacestore create ibm-cos <namespacestore> --endpoint <IBM COS ENDPOINT> --access-key <IBM ACCESS KEY> --secret-key <IBM SECRET ACCESS KEY> --target-bucket <bucket-name> -n openshift-storage
   ```

   <namespacestore>

   The name of the NamespaceStore.

   <IBM ACCESS KEY>, <IBM SECRET ACCESS KEY>, <IBM COS ENDPOINT>

   An IBM access key ID, secret access key, and the appropriate regional endpoint that corresponds to the location of the existing IBM bucket.

   <bucket-name>

   An existing IBM bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.

2. Create a namespace bucket class that defines a namespace policy for the namespace buckets. The namespace policy requires a type of either **single** or **multi**.

   - To create a namespace bucket class with a namespace policy of type **single**:

     ```
     $ odf noobaa bucketclass create namespace-bucketclass single <my-bucket-class> --resource <resource> -n openshift-storage
     ```

     <resource-name>

     The name you want to give the resource.

     <my-bucket-class>

     A unique bucket class name.

     <resource>

     A single NamespaceStore that defines the **read** and **write** target of the namespace bucket.

- To create a namespace bucket class with a namespace policy of type **multi**:

```
$ odf noobaa bucketclass create namespace-bucketclass multi <my-bucket-class> --write-resource <write-resource> --read-resources <read-resources> -n openshift-storage
```

<resource-name>

The name you want to give the resource.

<my-bucket-class>

A unique bucket class name.

<write-resource>

A single NamespaceStore that defines the **write** target of the namespace bucket.

<read-resources>

A comma-separated list of NamespaceStores that defines the **read** targets of the namespace bucket.

3. Create a bucket using an Object Bucket Class (OBC) resource that uses the bucket class defined in the earlier step.

```
$ odf noobaa obc create my-bucket-claim -n openshift-storage --app-namespace my-app --bucketclass <custom-bucket-class>
```

<bucket-name>

A bucket name of your choice.

<custom-bucket-class>

The name of the bucket class created in the previous step.
After the OBC is provisioned by the operator, a bucket is created in the MCG, and the operator creates a **Secret** and **ConfigMap** with the same name and in the same namespace as that of the OBC.

15.4.3. Adding a namespace bucket using the OpenShift Container Platform user interface

You can add namespace buckets using the OpenShift Container Platform user interface. For information about namespace buckets, see Managing namespace buckets.

Prerequisites

- Ensure that Openshift Container Platform with OpenShift Data Foundation operator is already installed.
- Access to the Multicloud Object Gateway (MCG).

Procedure

1. On the OpenShift Web Console, navigate to **Storage → Object Storage → Namespace Store** tab.
2. Click **Create namespace store** to create a **namespacestore** resources to be used in the namespace bucket.

a. Enter a **namespacestore** name.

b. Choose a provider and region.

c. Either select an existing secret, or click **Switch to credentials** to create a secret by entering a secret key and secret access key.

d. Enter a target bucket.

e. Click **Create**.

3. On the Namespace Store tab, verify that the newly created **namespacestore** is in the **Ready** state.

4. Repeat steps 2 and 3 until you have created all the desired amount of resources.

5. Navigate to **Bucket Class** tab and click **Create Bucket Class**.

a. Choose **Namespace BucketClass type** radio button.

b. Enter a **BucketClass name** and click **Next**.

c. Choose a **Namespace Policy Type** for your namespace bucket, and then click **Next**.

- If your namespace policy type is **Single**, you need to choose a read resource.
- If your namespace policy type is **Multi**, you need to choose read resources and a write resource.
- If your namespace policy type is **Cache**, you need to choose a Hub namespace store that defines the read and write target of the namespace bucket.

d. Select one **Read and Write NamespaceStore** which defines the read and write targets of the namespace bucket and click **Next**.

e. Review your new bucket class details, and then click **Create Bucket Class**.

6. Navigate to **Bucket Class** tab and verify that your newly created resource is in the **Ready** phase.

7. Navigate to **Object Bucket Claims** tab and click **Create Object Bucket Claim**.

a. Enter **ObjectBucketClaim Name** for the namespace bucket.

b. Select **StorageClass** as **openshift-storage.noobaa.io**.

c. Select the **BucketClass** that you created earlier for your **namespacestore** from the list. By default, **noobaa-default-bucket-class** gets selected.

d. Click **Create**. The namespace bucket is created along with Object Bucket Claim for your namespace.

8. Navigate to **Object Bucket Claims** tab and verify that the Object Bucket Claim created is in **Bound** state.

9. Navigate to **Object Buckets** tab and verify that the your namespace bucket is present in the list and is in **Bound** state.

15.5. MIRRORING DATA FOR HYBRID AND MULTICLOUD BUCKETS

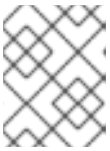
You can use the simplified process of the Multicloud Object Gateway (MCG) to span data across cloud providers and clusters. Before you create a bucket class that reflects the data management policy and mirroring, you must add a backing storage that can be used by the MCG.

You can set up mirroring data by using the OpenShift UI, YAML or command-line interface.

15.5.1. Creating bucket classes to mirror data using the command-line interface

Prerequisites

- Download the OpenShift Data Foundation command-line interface binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

1. From the command-line interface, run the following command to create a bucket class with a mirroring policy:

```
$ odf noobaa bucketclass create placement-bucketclass mirror-to-aws --
backingstores=azure-resource,aws-resource --placement Mirror
```

2. Set the newly created bucket class to a new bucket claim to generate a new bucket that will be mirrored between two locations:

```
$ odf noobaa obc create  mirrored-bucket --bucketclass=mirror-to-aws
```

15.5.2. Creating bucket classes to mirror data using a YAML

Use this procedure to create bucket classes for mirroring data using a YAML configuration file.

Procedure

1. Apply the following YAML. This YAML is a hybrid example that mirrors data between local Ceph storage and AWS:

```
apiVersion: noobaa.io/v1alpha1
kind: BucketClass
metadata:
  labels:
    app: noobaa
  name: <bucket-class-name>
  namespace: openshift-storage
spec:
  placementPolicy:
    tiers:
    - backingStores:
```

```
    - <backing-store-1>
    - <backing-store-2>
    placement: Mirror
```

2. Add the following lines to your standard Object Bucket Claim (OBC):

```
additionalConfig:
  bucketclass: mirror-to-aws
```

For more information about OBCs, see Section 15.7, “Object Bucket Claim” .

15.6. BUCKET POLICIES IN THE MULTICLOUD OBJECT GATEWAY

Bucket policies in the Multicloud Object Gateway use AWS S3-compatible syntax to control user access permissions to buckets and objects stored in OpenShift Data Foundation.

OpenShift Data Foundation supports AWS S3 bucket policies. Bucket policies allow you to grant users access permissions for buckets and the objects in them.

15.6.1. Introduction to bucket policies

Bucket policies are an access policy option available for you to grant permission to your AWS S3 buckets and objects. Bucket policies use JSON-based access policy language.

Additional resources

- AWS Access Policy Language Overview

15.6.2. Using bucket policies in Multicloud Object Gateway

Prerequisites

- A running OpenShift Data Foundation Platform.
- Access to the Multicloud Object Gateway (MCG), see Section 15.2, “Accessing the Multicloud Object Gateway with your applications”
- A valid Multicloud Object Gateway user account. See Creating a user in the Multicloud Object Gatewayfor instructions to create a user account.

Procedure

1. Create the bucket policy in JSON format.
   For example:

```
{
    "Version": "NewVersion",
    "Statement": [
        {
            "Sid": "Example",
            "Effect": "Allow",
            "Principal": {
                "AWS":  "john.doe@example.com"
            },
```

```
            "Action": [
                "s3:GetObject"
            ],
            "Resource": [
                "arn:aws:s3:::john_bucket"
            ]
        }
    ]
}
```

Replace **john.doe@example.com** with a valid Multicloud Object Gateway user account.

2. Using AWS S3 client, use the **put-bucket-policy** command to apply the bucket policy to your S3 bucket:

```
# aws --endpoint ENDPOINT --no-verify-ssl s3api put-bucket-policy --bucket MyBucket --policy file://BucketPolicy
```

   a. Replace ***ENDPOINT*** with the S3 endpoint.

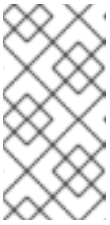
   b. Replace ***MyBucket*** with the bucket to set the policy on.

   c. Replace ***BucketPolicy*** with the bucket policy JSON file.

   d. Add **--no-verify-ssl** if you are using the default self signed certificates.
   For example:

```
# aws --endpoint https://s3-openshift-storage.apps.gogo44.noobaa.org --no-verify-ssl s3api put-bucket-policy -bucket MyBucket --policy file://BucketPolicy
```

   For more information on the **put-bucket-policy** command, see the AWS CLI Command Reference for put-bucket-policy.



NOTE

The principal element specifies the user that is allowed or denied access to a resource, such as a bucket. Currently, Only NooBaa accounts can be used as principals. In the case of object bucket claims, NooBaa automatically create an account **obc-account.<generated bucket name>@noobaa.io**.



NOTE

Bucket policy conditions are not supported.

Additional resources

- AWS Access Policy Language Overview
- AWS Bucket Policy Examples
- IAM JSON policy elements reference

15.6.3. Creating a user in the Multicloud Object Gateway

Prerequisites

- A running OpenShift Data Foundation Platform.
- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

- Execute the following command to create an MCG user account:

```
$ odf noobaa account create <noobaa-account-name> [--allow_bucket_create=true] [--default_resource=''] [--force_md5_etag=false] [--gid=-1] [--new_buckets_path='/'] [--nsfs_account_config=false] [--nsfs_only=true] [--uid=-1]
```

<allow_bucket_create>

Specify if this account be allowed to create new buckets.

<default_resource>

Specify the default resource on which new buckets are created.

<force_md5_etag>

Specify if **md5 etag** calculation be enabled for the account.

<gid_number>

Specify the NSFS GID.

<new_buckets_path>

Specify the path where the new buckets are created.

<nsfs_account_config>

Specify if NSFS account needs to be created.

<nsfs_only>

Set if this account is used only for NSFS.

<uid_number>

Specify the NSFS UID.

15.7. OBJECT BUCKET CLAIM

Object Bucket Claims (OBCs) provide applications with access to S3-compatible object storage by automatically provisioning buckets and credentials in OpenShift Data Foundation.

15.7.1. Object Bucket Claim

An Object Bucket Claim can be used to request an S3 compatible bucket backend for your workloads.

You can create an Object Bucket Claim in three ways. An object bucket claim creates a new bucket and an application account in NooBaa with permissions to the bucket, including a new access key and secret access key. The application account is allowed to access only a single bucket and can't create new

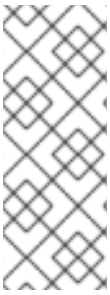
buckets by default.

Additional resources

- Section 15.7.2, "Dynamic Object Bucket Claim"
- Section 15.7.3, "Creating an Object Bucket Claim using the command-line interface"
- Section 15.7.4, "Creating an Object Bucket Claim using the OpenShift Web Console"

15.7.2. Dynamic Object Bucket Claim

Similar to Persistent Volumes, you can add the details of the Object Bucket claim (OBC) to your application's YAML, and get the object service endpoint, access key, and secret access key available in a configuration map and secret. It is easy to read this information dynamically into environment variables of your application.



NOTE

The Multicloud Object Gateway endpoints uses self-signed certificates only if OpenShift uses self-signed certificates. Using signed certificates in OpenShift automatically replaces the Multicloud Object Gateway endpoints certificates with signed certificates. Get the certificate currently used by Multicloud Object Gateway by accessing the endpoint via the browser. See Accessing the Multicloud Object Gateway with your applications for more information.

Procedure

1. Add the following lines to your application YAML:

```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  name: <obc-name>
spec:
  generateBucketName: <obc-bucket-name>
  storageClassName: openshift-storage.noobaa.io
```

   These lines are the OBC itself.

   a. Replace **<obc-name>** with the a unique OBC name.

   b. Replace **<obc-bucket-name>** with a unique bucket name for your OBC.

2. To automate the use of the OBC add more lines to the YAML file.
   For example:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: testjob
spec:
  template:
    spec:
      restartPolicy: OnFailure
```

```
containers:
  - image: <your application image>
    name: test
    env:
      - name: BUCKET_NAME
        valueFrom:
          configMapKeyRef:
            name: <obc-name>
            key: BUCKET_NAME
      - name: BUCKET_HOST
        valueFrom:
          configMapKeyRef:
            name: <obc-name>
            key: BUCKET_HOST
      - name: BUCKET_PORT
        valueFrom:
          configMapKeyRef:
            name: <obc-name>
            key: BUCKET_PORT
      - name: AWS_ACCESS_KEY_ID
        valueFrom:
          secretKeyRef:
            name: <obc-name>
            key: AWS_ACCESS_KEY_ID
      - name: AWS_SECRET_ACCESS_KEY
        valueFrom:
          secretKeyRef:
            name: <obc-name>
            key: AWS_SECRET_ACCESS_KEY
```

The example is the mapping between the bucket claim result, which is a configuration map with data and a secret with the credentials. This specific job claims the Object Bucket from NooBaa, which creates a bucket and an account.

a. Replace all instances of **<obc-name>** with your OBC name.

b. Replace **<your application image>** with your application image.

3. Apply the updated YAML file:

```
# oc apply -f <yaml.file>
```

Replace **<yaml.file>** with the name of your YAML file.

4. To view the new configuration map, run the following:

```
# oc get cm <obc-name> -o yaml
```

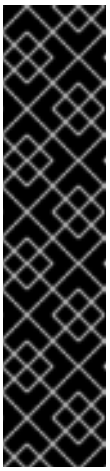
Replace **obc-name** with the name of your OBC.

You can expect the following environment variables in the output:

- **BUCKET_HOST** - Endpoint to use in the application.
- **BUCKET_PORT** - The port available for the application.

- The port is related to the **BUCKET_HOST**. For example, if the **BUCKET_HOST** is https://my.example.com, and the **BUCKET_PORT** is 443, the endpoint for the object service would be https://my.example.com:443.

- **BUCKET_NAME** - Requested or generated bucket name.
- **AWS_ACCESS_KEY_ID** - Access key that is part of the credentials.
- **AWS_SECRET_ACCESS_KEY** - Secret access key that is part of the credentials.



IMPORTANT

Retrieve the **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY**. The names are used so that it is compatible with the AWS S3 API. You need to specify the keys while performing S3 operations, especially when you read, write or list from the Multicloud Object Gateway (MCG) bucket. The keys are encoded in Base64. Decode the keys before using them.

```
# oc get secret <obc_name> -o yaml
```

<obc_name>

Specify the name of the object bucket claim.

15.7.3. Creating an Object Bucket Claim using the command-line interface

When creating an Object Bucket Claim (OBC) using the command-line interface, you get a configuration map and a Secret that together contain all the information your application needs to use the object storage service.

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

1. Use the command-line interface to generate the details of a new bucket and credentials. Run the following command:

```
# odf noobaa obc create <obc-name> -n openshift-storage
```

Replace **<obc-name>** with a unique OBC name, for example, **myappobc**.

Additionally, you can use the **--app-namespace** option to specify the namespace where the OBC configuration map and secret will be created, for example, **myapp-namespace**.

For example:

```
INFO[0001]   Created: ObjectBucketClaim "test21obc"
```

The command-line interface has created the necessary configuration and has informed OpenShift about the new OBC.

2. Run the following command to view the OBC:

```
# oc get obc -n openshift-storage
```

For example:

```
NAME       STORAGE-CLASS               PHASE   AGE
test21obc  openshift-storage.noobaa.io  Bound   38s
```

3. Run the following command to view the YAML file for the new OBC:

```
# oc get obc test21obc -o yaml -n openshift-storage
```

For example:

```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  creationTimestamp: "2019-10-24T13:30:07Z"
  finalizers:
  - objectbucket.io/finalizer
  generation: 2
  labels:
    app: noobaa
    bucket-provisioner: openshift-storage.noobaa.io-obc
    noobaa-domain: openshift-storage.noobaa.io
  name: test21obc
  namespace: openshift-storage
  resourceVersion: "40756"
  selfLink: /apis/objectbucket.io/v1alpha1/namespaces/openshift-
storage/objectbucketclaims/test21obc
  uid: 64f04cba-f662-11e9-bc3c-0295250841af
spec:
  ObjectBucketName: obc-openshift-storage-test21obc
  bucketName: test21obc-933348a6-e267-4f82-82f1-e59bf4fe3bb4
  generateBucketName: test21obc
  storageClassName: openshift-storage.noobaa.io
status:
  phase: Bound
```

4. Inside of your **openshift-storage** namespace, you can find the configuration map and the secret to use this OBC. The CM and the secret have the same name as the OBC.
Run the following command to view the secret:

```
# oc get -n openshift-storage secret test21obc -o yaml
```

For example:

```
apiVersion: v1
data:
  AWS_ACCESS_KEY_ID: c0M0R2xVanF3ODR3bHBkVW94cmY=
  AWS_SECRET_ACCESS_KEY:
Wi9kcFluSWxHRzlWaFlzNk1hc0xma2JXcjM1MVhqa051SlBleXpmOQ==
kind: Secret
metadata:
  creationTimestamp: "2019-10-24T13:30:07Z"
  finalizers:
  - objectbucket.io/finalizer
  labels:
    app: noobaa
    bucket-provisioner: openshift-storage.noobaa.io-obc
    noobaa-domain: openshift-storage.noobaa.io
  name: test21obc
  namespace: openshift-storage
  ownerReferences:
  - apiVersion: objectbucket.io/v1alpha1
    blockOwnerDeletion: true
    controller: true
    kind: ObjectBucketClaim
    name: test21obc
    uid: 64f04cba-f662-11e9-bc3c-0295250841af
  resourceVersion: "40751"
  selfLink: /api/v1/namespaces/openshift-storage/secrets/test21obc
  uid: 65117c1c-f662-11e9-9094-0a5305de57bb
type: Opaque
```

The secret gives you the S3 access credentials.

5. Run the following command to view the configuration map:

```
# oc get -n openshift-storage cm test21obc -o yaml
```

For example:

```
apiVersion: v1
data:
  BUCKET_HOST: 10.0.171.35
  BUCKET_NAME: test21obc-933348a6-e267-4f82-82f1-e59bf4fe3bb4
  BUCKET_PORT: "31242"
  BUCKET_REGION: ""
  BUCKET_SUBREGION: ""
kind: ConfigMap
metadata:
  creationTimestamp: "2019-10-24T13:30:07Z"
  finalizers:
  - objectbucket.io/finalizer
  labels:
    app: noobaa
    bucket-provisioner: openshift-storage.noobaa.io-obc
    noobaa-domain: openshift-storage.noobaa.io
  name: test21obc
  namespace: openshift-storage
  ownerReferences:
```

```
- apiVersion: objectbucket.io/v1alpha1
  blockOwnerDeletion: true
  controller: true
  kind: ObjectBucketClaim
  name: test21obc
  uid: 64f04cba-f662-11e9-bc3c-0295250841af
resourceVersion: "40752"
selfLink: /api/v1/namespaces/openshift-storage/configmaps/test21obc
uid: 651c6501-f662-11e9-9094-0a5305de57bb
```

The configuration map contains the S3 endpoint information for your application.

15.7.4. Creating an Object Bucket Claim using the OpenShift Web Console

You can create an Object Bucket Claim (OBC) using the OpenShift Web Console.

Prerequisites

- Administrative access to the OpenShift Web Console.
- In order for your applications to communicate with the OBC, you need to use the configmap and secret. For more information about this, see Section 15.7.2, “Dynamic Object Bucket Claim” .

Procedure

1. Log into the OpenShift Web Console.
2. On the left navigation bar, click **Storage → Object Storage → Object Bucket Claims**.
3. Choose the bucket type:
   - Click the **General** card to create a standard object bucket claim.
   - Click the **S3 Vector** card to create a vector bucket claim for storing vector embeddings. For more information, see Creating vector buckets using ObjectBucketClaim .
4. Click **Create Object Bucket Claim**.
   a. Enter a name for your object bucket claim and select the appropriate storage class based on your deployment, internal or external, from the dropdown menu:

      Internal mode

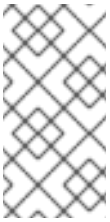
      The following storage classes, which were created after deployment, are available for use:

      - **ocs-storagecluster-ceph-rgw** uses the Ceph Object Gateway (RGW)
      - **openshift-storage.noobaa.io** uses the Multicloud Object Gateway (MCG)

      External mode

      The following storage classes, which were created after deployment, are available for use:

      - **ocs-external-storagecluster-ceph-rgw** uses the RGW
      - **openshift-storage.noobaa.io** uses the MCG



NOTE

The RGW OBC storage class is only available with fresh installations of OpenShift Data Foundation version 4.5. It does not apply to clusters upgraded from previous OpenShift Data Foundation releases.

b. Click **Create**.
Once you create the OBC, you are redirected to its detail page.



NOTE

For RGW OBCs, the OBC-specific credentials are required to access the newly created bucket. RGW administrator credentials cannot be used to access this bucket.

15.7.5. Attaching an Object Bucket Claim to a deployment

Once created, Object Bucket Claims (OBCs) can be attached to specific deployments.

Prerequisites

- Administrative access to the OpenShift Web Console.

Procedure

1. On the left navigation bar, click **Storage → Object Storage → Object Bucket Claims**.
2. Click the Action menu **(⋮)** next to the OBC you created.
   a. From the drop-down menu, select **Attach to Deployment**
   b. Select the desired deployment from the Deployment Name list, then click **Attach**.

15.7.6. Viewing object buckets using the OpenShift Web Console

You can view the details of object buckets created for Object Bucket Claims (OBCs) using the OpenShift Web Console.

Prerequisites

- Administrative access to the OpenShift Web Console.

Procedure

1. Log into the OpenShift Web Console.
2. On the left navigation bar, click **Storage → Object Storage → Object Buckets**.
   Optonal: You can also navigate to the details page of a specific OBC, and click the **Resource** link to view the object buckets for that OBC.
3. Select the object bucket of which you want to see the details. Once selected you are navigated to the **Object Bucket Details** page.

15.7.7. Deleting Object Bucket Claims

Prerequisites

- Administrative access to the OpenShift Web Console.

Procedure

1. On the left navigation bar, click **Storage → Object Storage → Object Bucket Claims**.
2. Click the Action menu **(⋮)** next to the Object Bucket Claim (OBC) you want to delete.
   a. Select **Delete Object Bucket Claim**.
   b. Click **Delete**.

15.8. CACHING POLICY FOR OBJECT BUCKETS

Cache buckets connect a local Multicloud Object Gateway bucket to a large S3-compatible hub target such as AWS or IBM COS, reducing latency for frequently accessed objects.

A cache bucket is a namespace bucket with a hub target and a cache target. The hub target is an S3 compatible large object storage bucket. The cache bucket is the local Multicloud Object Gateway bucket. You can create a cache bucket that caches an AWS bucket or an IBM COS bucket.



IMPORTANT

Cache buckets are a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

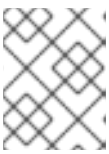
For more information, see Technology Preview Features Support Scope .

- AWS S3
- IBM COS

15.8.1. Creating an AWS cache bucket

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

1. Create a NamespaceStore resource. A NamespaceStore represents an underlying storage to be used as a read or write target for the data in the MCG namespace buckets. From the command-line interface, run the following command:

```
odf noobaa namespacestore create aws-s3 <namespacestore> --access-key <AWS ACCESS KEY> --secret-key <AWS SECRET ACCESS KEY> --target-bucket <bucket-name>
```

   a. Replace **<namespacestore>** with the name of the namespacestore.

   b. Replace **<AWS ACCESS KEY>** and **<AWS SECRET ACCESS KEY>** with an AWS access key ID and secret access key you created for this purpose.

   c. Replace **<bucket-name>** with an existing AWS bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.
   You can also add storage resources by applying a YAML. First create a secret with credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <namespacestore-secret-name>
type: Opaque
data:
  AWS_ACCESS_KEY_ID: <AWS ACCESS KEY ID ENCODED IN BASE64>
  AWS_SECRET_ACCESS_KEY: <AWS SECRET ACCESS KEY ENCODED IN BASE64>
```

   You must supply and encode your own AWS access key ID and secret access key using Base64, and use the results in place of **<AWS ACCESS KEY ID ENCODED IN BASE64>** and **<AWS SECRET ACCESS KEY ENCODED IN BASE64>**.

   Replace **<namespacestore-secret-name>** with a unique name.

   Then apply the following YAML:

```
apiVersion: noobaa.io/v1alpha1
kind: NamespaceStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: <namespacestore>
  namespace: openshift-storage
spec:
  awsS3:
    secret:
      name: <namespacestore-secret-name>
      namespace: <namespace-secret>
    targetBucket: <target-bucket>
  type: aws-s3
```

   d. Replace **<namespacestore>** with a unique name.

e. Replace **<namespacestore-secret-name>** with the secret created in the previous step.

f. Replace **<namespace-secret>** with the namespace used to create the secret in the previous step.

g. Replace **<target-bucket>** with the AWS S3 bucket you created for the namespacestore.

2. Run the following command to create a bucket class:

```
odf noobaa bucketclass create namespace-bucketclass cache <my-cache-bucket-class> --backingstores <backing-store> --hub-resource <namespacestore>
```

a. Replace **<my-cache-bucket-class>** with a unique bucket class name.

b. Replace **<backing-store>** with the relevant backing store. You can list one or more backingstores separated by commas in this field.

c. Replace **<namespacestore>** with the namespacestore created in the previous step.

3. Run the following command to create a bucket using an Object Bucket Claim (OBC) resource that uses the bucket class defined in step 2.

```
$ odf noobaa obc create <my-bucket-claim> my-app --bucketclass <custom-bucket-class>
```

a. Replace **<my-bucket-claim>** with a unique name.

b. Replace **<custom-bucket-class>** with the name of the bucket class created in step 2.

15.8.2. Creating an IBM COS cache bucket

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

1. Create a NamespaceStore resource. A NamespaceStore represents an underlying storage to be used as a read or write target for the data in the MCG namespace buckets. From the command-line interface, run the following command:

```
odf noobaa namespacestore create ibm-cos <namespacestore> --endpoint <IBM COS ENDPOINT> --access-key <IBM ACCESS KEY> --secret-key <IBM SECRET ACCESS KEY> --target-bucket <bucket-name>
```

a. Replace **<namespacestore>** with the name of the NamespaceStore.

b. Replace **<IBM ACCESS KEY>**, **<IBM SECRET ACCESS KEY>**, **<IBM COS ENDPOINT>** with an IBM access key ID, secret access key and the appropriate regional endpoint that corresponds to the location of the existing IBM bucket.

c. Replace **<bucket-name>** with an existing IBM bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.
You can also add storage resources by applying a YAML. First, Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <namespacestore-secret-name>
type: Opaque
data:
  IBM_COS_ACCESS_KEY_ID: <IBM COS ACCESS KEY ID ENCODED IN BASE64>
  IBM_COS_SECRET_ACCESS_KEY: <IBM COS SECRET ACCESS KEY ENCODED
IN BASE64>
```

You must supply and encode your own IBM COS access key ID and secret access key using Base64, and use the results in place of **<IBM COS ACCESS KEY ID ENCODED IN BASE64>** and **<IBM COS SECRET ACCESS KEY ENCODED IN BASE64>**.

Replace **<namespacestore-secret-name>** with a unique name.

Then apply the following YAML:

```
apiVersion: noobaa.io/v1alpha1
kind: NamespaceStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: <namespacestore>
  namespace: openshift-storage
spec:
  s3Compatible:
    endpoint: <IBM COS ENDPOINT>
    secret:
      name: <backingstore-secret-name>
      namespace: <namespace-secret>
    signatureVersion: v2
    targetBucket: <target-bucket>
  type: ibm-cos
```

d. Replace **<namespacestore>** with a unique name.

e. Replace **<IBM COS ENDPOINT>** with the appropriate IBM COS endpoint.

f. Replace **<backingstore-secret-name>** with the secret created in the previous step.

g. Replace **<namespace-secret>** with the namespace used to create the secret in the previous step.

h. Replace **<target-bucket>** with the AWS S3 bucket you created for the namespacestore.

2. Run the following command to create a bucket class:

```
odf noobaa bucketclass create namespace-bucketclass cache <my-bucket-class> --
backingstores <backing-store> --hubResource <namespacestore>
```

a. Replace **<my-bucket-class>** with a unique bucket class name.

b. Replace **<backing-store>** with the relevant backing store. You can list one or more backingstores separated by commas in this field.

c. Replace **<namespacestore>** with the namespacestore created in the previous step.

3. Run the following command to create a bucket using an Object Bucket Claim resource that uses the bucket class defined in step 2.

```
$ odf noobaa obc create <my-bucket-claim> my-app --bucketclass <custom-bucket-class>
```

a. Replace **<my-bucket-claim>** with a unique name.

b. Replace **<custom-bucket-class>** with the name of the bucket class created in step 2.

15.9. SCALING MULTICLOUD OBJECT GATEWAY PERFORMANCE BY ADDING ENDPOINTS

Increase Multicloud Object Gateway throughput for S3-intensive workloads by scaling the number of endpoints to match application demand.

The Multicloud Object Gateway performance may vary from one environment to another. In some cases, specific applications require faster performance which can be easily addressed by scaling S3 endpoints.

The Multicloud Object Gateway resource pool is a group of NooBaa daemon containers that provide two types of services enabled by default:

- Storage service
- S3 endpoint service

15.9.1. S3 endpoints in the Multicloud Object Gateway

The S3 endpoint is a service that every Multicloud Object Gateway (MCG) provides by default that handles the heavy lifting data digestion in the MCG. The endpoint service handles the inline data chunking, deduplication, compression, and encryption, and it accepts data placement instructions from the MCG.

15.9.2. Scaling the Multicloud Object Gateway with storage nodes

Prerequisites

- A running OpenShift Data Foundation cluster on OpenShift Container Platform with access to the Multicloud Object Gateway (MCG).

A storage node in the MCG is a NooBaa daemon container attached to one or more Persistent Volumes (PVs) and used for local object service data storage. NooBaa daemons can be deployed on Kubernetes nodes. This can be done by creating a Kubernetes pool consisting of StatefulSet pods.

Procedure

1. Log in to **OpenShift Web Console**.
2. From the MCG user interface, click **Overview → Add Storage Resources**.
3. In the window, click **Deploy Kubernetes Pool**
4. In the **Create Pool** step create the target pool for the future installed nodes.
5. In the **Configure** step, configure the number of requested pods and the size of each PV. For each new pod, one PV is to be created.
6. In the **Review** step, you can find the details of the new pool and select the deployment method you wish to use: local or external deployment. If local deployment is selected, the Kubernetes nodes will deploy within the cluster. If external deployment is selected, you will be provided with a YAML file to run externally.
7. All nodes will be assigned to the pool you chose in the first step, and can be found under **Resources → Storage resources → Resource name**.

15.10. AUTOMATIC SCALING OF MULTICLOUD OBJECT GATEWAY ENDPOINTS

The number of MultiCloud Object Gateway (MCG) endpoints scale automatically when the load on the MCG S3 service increases or decreases. OpenShift Data Foundation clusters are deployed with one active MCG endpoint. Each MCG endpoint pod is configured by default with 1 CPU and 2Gi memory request, with limits matching the request. When the CPU load on the endpoint crosses over an 80% usage threshold for a consistent period of time, a second endpoint is deployed lowering the load on the first endpoint. When the average CPU load on both endpoints falls below the 80% threshold for a consistent period of time, one of the endpoints is deleted. This feature improves performance and serviceability of the MCG.

You can scale the Horizontal Pod Autoscaler (HPA) for **noobaa-endpoint** using the following **oc patch** command, for example:

```
# oc patch -n openshift-storage storagecluster ocs-storagecluster \
   --type merge \
   --patch '{"spec": {"multiCloudGateway": {"endpoints": {"minCount": 3,"maxCount": 10}}}}'
```

The example above sets the **minCount** to **3** and the **maxCount to `10**.



WARNING

Changing **maxCount** in the **StorageCluster** CR triggers a PostgreSQL restart, which causes a temporary S3 outage of approximately 30–60 seconds. This occurs because the noobaa-operator dynamically calculates the PostgreSQL **max_connections** value based on the configured maximum endpoint count, and PostgreSQL requires a restart to apply the new value.

- Changing only **minCount** does not trigger a PostgreSQL restart.
- HPA-driven autoscaling within the already-configured **minCount** and **maxCount** range does not trigger a PostgreSQL restart.

If S3 availability is critical to your workload, perform **maxCount** changes during a scheduled maintenance window.

CHAPTER 16. MANAGING PERSISTENT VOLUME CLAIMS

Persistent volume claims (PVCs) in OpenShift Data Foundation allow applications to request and use durable block, file, or object storage. Learn how to configure, monitor, and automate PVC management.



IMPORTANT

Expanding PVCs is not supported for PVCs backed by OpenShift Data Foundation.

16.1. CONFIGURING APPLICATION PODS TO USE OPENSHIFT DATA FOUNDATION

Follow the instructions in this section to configure OpenShift Data Foundation as storage for an application pod.

Prerequisites

- Administrative access to OpenShift Web Console.
- OpenShift Data Foundation Operator is installed and running in the **openshift-storage** namespace. In OpenShift Web Console, click **Ecosystem → Installed Operators** to view installed operators.
- The default storage classes provided by OpenShift Data Foundation are available. In OpenShift Web Console, click **Storage → StorageClasses** to view default storage classes.

Procedure

1. **Create a Persistent Volume Claim (PVC) for the application to use.**
   a. In OpenShift Web Console, click **Storage → Persistent Volume Claims**
   b. Set the **Project** for the application pod.
   c. Click **Create Persistent Volume Claim**
      i. Specify a **Storage Class** provided by OpenShift Data Foundation.
      ii. Specify the PVC **Name**, for example, **myclaim**.
      iii. Select the required **Access Mode**.



NOTE

The **Access Mode**, **Shared access (RWX)** is not supported in IBM FlashSystem.

      iv. For Rados Block Device (RBD), if the **Access mode** is ReadWriteOnce (**RWO**), select the required **Volume mode**. The default volume mode is **Filesystem**.
      v. Specify a **Size** as per application requirement.
      vi. Click **Create** and wait until the PVC is in **Bound** status.

2. **Configure a new or existing application pod to use the new PVC.**

   - For a new application pod, perform the following steps:

     i. Click **Workloads →Pods**.

     ii. Create a new application pod.

     iii. Under the **spec:** section, add **volumes:** section to add the new PVC as a volume for the application pod.

     ```
     volumes:
       - name: <volume_name>
         persistentVolumeClaim:
           claimName: <pvc_name>
     ```

     For example:

     ```
     volumes:
       - name: mypd
         persistentVolumeClaim:
           claimName: myclaim
     ```

   - For an existing application pod, perform the following steps:

     i. Click **Workloads →Deployment Configs**.

     ii. Search for the required deployment config associated with the application pod.

     iii. Click on its **Action menu (⋮) → Edit Deployment Config**.

     iv. Under the **spec:** section, add **volumes:** section to add the new PVC as a volume for the application pod and click **Save**.

     ```
     volumes:
       - name: <volume_name>
         persistentVolumeClaim:
           claimName: <pvc_name>
     ```

     For example:

     ```
     volumes:
       - name: mypd
         persistentVolumeClaim:
           claimName: myclaim
     ```

3. **Verify that the new configuration is being used.**

   a. Click **Workloads → Pods**.

   b. Set the **Project** for the application pod.

   c. Verify that the application pod appears with a status of **Running**.

   d. Click the application pod name to view pod details.

e. Scroll down to **Volumes** section and verify that the volume has a **Type** that matches your new Persistent Volume Claim, for example, **myclaim**.

16.2. VIEWING PERSISTENT VOLUME CLAIM REQUEST STATUS

Use this procedure to view the status of a PVC request.

Prerequisites

- Administrator access to OpenShift Data Foundation.

Procedure

1. Log in to OpenShift Web Console.
2. Click **Storage → Persistent Volume Claims**
3. Search for the required PVC name by using the **Filter** textbox. You can also filter the list of PVCs by Name or Label to narrow down the list
4. Check the **Status** column corresponding to the required PVC.
5. Click the required **Name** to view the PVC details.

16.3. REVIEWING PERSISTENT VOLUME CLAIM REQUEST EVENTS

Use this procedure to review and address Persistent Volume Claim (PVC) request events.

Prerequisites

- Administrator access to OpenShift Web Console.

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation**
2. In the **Storage systems** tab, select the storage system and then click **Overview → Block and File**.
3. Locate the **Inventory** card to see the number of PVCs with errors.
4. Click **Storage → Persistent Volume Claims**
5. Search for the required PVC using the **Filter** textbox.
6. Click on the PVC name and navigate to **Events**
7. Address the events as required or as directed.

16.4. DYNAMIC PROVISIONING

Dynamic provisioning in OpenShift Data Foundation lets applications automatically request and bind persistent storage without manual administrator intervention.

16.4.1. About dynamic provisioning

The StorageClass resource object describes and classifies storage that can be requested, as well as provides a means for passing parameters for dynamically provisioned storage on demand. StorageClass objects can also serve as a management mechanism for controlling different levels of storage and access to the storage. Cluster Administrators (**cluster-admin**) or Storage Administrators (**storage-admin**) define and create the StorageClass objects that users can request without needing any intimate knowledge about the underlying storage volume sources.

The OpenShift Data Foundation persistent volume framework enables this functionality and allows administrators to provision a cluster with persistent storage. The framework also gives users a way to request those resources without having any knowledge of the underlying infrastructure.

Many storage types are available for use as persistent volumes in OpenShift Data Foundation. Storage plug-ins might support static provisioning, dynamic provisioning or both provisioning types.

16.4.2. Dynamic provisioning in OpenShift Data Foundation

Red Hat OpenShift Data Foundation is software-defined storage that is optimised for container environments. It runs as an operator on OpenShift Container Platform to provide highly integrated and simplified persistent storage management for containers.

OpenShift Data Foundation supports a variety of storage types, including:

- Block storage for databases
- Shared file storage for continuous integration, messaging, and data aggregation
- Object storage for archival, backup, and media storage

Version 4 uses Red Hat Ceph Storage to provide the file, block, and object storage that backs persistent volumes, and Rook.io to manage and orchestrate provisioning of persistent volumes and claims. NooBaa provides object storage, and its Multicloud Gateway allows object federation across multiple cloud environments (available as a Technology Preview).

In OpenShift Data Foundation 4, the Red Hat Ceph Storage Container Storage Interface (CSI) driver for RADOS Block Device (RBD) and Ceph File System (CephFS) handles the dynamic provisioning requests. When a PVC request comes in dynamically, the CSI driver has the following options:

- Create a PVC with ReadWriteOnce (RWO) and ReadWriteMany (RWX) access that is based on Ceph RBDs with volume mode **Block**.
- Create a PVC with ReadWriteOnce (RWO) access that is based on Ceph RBDs with volume mode **Filesystem**. The supported **fstype** for **Filesystem** mode is **ext4**. This is the default.
- Create a PVC with ReadWriteOnce (RWO) and ReadWriteMany (RWX) access that is based on CephFS for volume mode **Filesystem**.
- Create a PVC with ReadWriteOncePod (RWOP) access that is based on CephFS,NFS and RBD. With RWOP access mode, you mount the volume as read-write by a single pod on a single node.

The judgment of which driver (RBD or CephFS) to use is based on the entry in the **storageclass.yaml** file.

16.4.3. Available dynamic provisioning plug-ins

OpenShift Data Foundation provides the following provisioner plug-ins, which have generic implementations for dynamic provisioning that use the cluster's configured provider's API to create new storage resources:

Storage type	Provisioner plug-in name	Notes
OpenStack Cinder	**kubernetes.io/cinder**	
AWS Elastic Block Store (EBS)	**kubernetes.io/aws-ebs**	For dynamic provisioning when using multiple clusters in different zones, tag each node with **Key=kubernetes.io/cluster/<cluster_name>,Value=<cluster_id>** where **<cluster_name>** and **<cluster_id>** are unique per cluster.
AWS Elastic File System (EFS)		Dynamic provisioning is accomplished through the EFS provisioner pod and not through a provisioner plug-in.
Azure Disk	**kubernetes.io/azure-disk**	
Azure File	**kubernetes.io/azure-file**	The **persistent-volume-binder** ServiceAccount requires permissions to create and get Secrets to store the Azure storage account and keys.
GCE Persistent Disk (gcePD)	**kubernetes.io/gce-pd**	In multi-zone configurations, it is advisable to run one OpenShift Data Foundation cluster per GCE project to avoid PVs from being created in zones where no node in the current cluster exists.
VMware vSphere	**kubernetes.io/vsphere-volume**	
Red Hat Virtualization	**csi.ovirt.org**	



IMPORTANT

Any chosen provisioner plug-in also requires configuration for the relevant cloud, host, or third-party provider as per the relevant documentation.

CHAPTER 17. VOLUME SNAPSHOTS

Volume snapshots capture the state of a persistent volume at a point in time, enabling data protection, backup, and cloning workflows for OpenShift Data Foundation block and file storage.

A volume snapshot is the state of the storage volume in a cluster at a particular point in time. These snapshots help to use storage more efficiently by not having to make a full copy each time and can be used as building blocks for developing an application.

You can create multiple snapshots of the same persistent volume claim (PVC). For CephFS, you can create up to 100 snapshots per PVC. For RADOS Block Device (RBD), you can create up to 512 snapshots per PVC.



NOTE

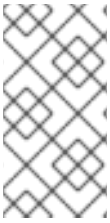
You cannot schedule periodic creation of snapshots.

17.1. CREATING VOLUME SNAPSHOTS

You can create a volume snapshot either from the Persistent Volume Claim (PVC) page or the Volume Snapshots page.

Prerequisites

- For a consistent snapshot, the PVC should be in **Bound** state and not be in use. Ensure to stop all IO before taking the snapshot.



NOTE

OpenShift Data Foundation only provides crash consistency for a volume snapshot of a PVC if a pod is using it. For application consistency, be sure to first tear down a running pod to ensure consistent snapshots or use any quiesce mechanism provided by the application to ensure it.

Procedure

1. From the Persistent Volume Claims page:
   a. Click **Storage → Persistent Volume Claims** from the OpenShift Web Console.
2. To create a volume snapshot, do one of the following:
   - Beside the desired PVC, click Action menu **(⋮) → Create Snapshot**.
   - Click on the PVC for which you want to create the snapshot and click **Actions → Create Snapshot**.
3. Enter a **Name** for the volume snapshot.
4. Choose the **Snapshot Class** from the drop-down list.
5. Click **Create**. You will be redirected to the Details page of the volume snapshot that is created.

   From the Volume Snapshots page

6. Click **Storage → Volume Snapshots** from the OpenShift Web Console.
7. In the **Volume Snapshots** page, click **Create Volume Snapshot**
8. Choose the required **Project** from the drop-down list.
9. Choose the **Persistent Volume Claim** from the drop-down list.
10. Enter a **Name** for the snapshot.
11. Choose the **Snapshot Class** from the drop-down list.
12. Click **Create**. You will be redirected to the Details page of the volume snapshot that is created.

Verification

- Go to the **Details** page of the PVC and click the **Volume Snapshots** tab to see the list of volume snapshots. Verify that the new volume snapshot is listed.
- Click **Storage → Volume Snapshots** from the OpenShift Web Console. Verify that the new volume snapshot is listed.
- Wait for the volume snapshot to be in **Ready** state.

17.2. RESTORING VOLUME SNAPSHOTS

When you restore a volume snapshot, a new Persistent Volume Claim (PVC) gets created. The restored PVC is independent of the volume snapshot and the parent PVC.

You can restore a volume snapshot from either the Persistent Volume Claim page or the Volume Snapshots page.

Procedure

1. From the Persistent Volume Claims page:
   You can restore volume snapshot from this page only if the parent PVC is present.
   a. Click **Storage → Persistent Volume Claims** from the OpenShift Web Console.
2. Click on the PVC name with the volume snapshot to restore a volume snapshot as a new PVC.
3. In the **Volume Snapshots** tab, click the Action menu (⋮) next to the volume snapshot you want to restore.
4. Click **Restore as new PVC**.
5. Enter a name for the new PVC.
6. Select the **Storage Class** name.
7. Select the **Access Mode** of your choice.
8. Optional: For RBD, select **Volume mode**.
9. Click **Restore**. You are redirected to the new PVC details page.

From the Volume Snapshots page

10. Click **Storage → Volume Snapshots** from the OpenShift Web Console.
11. In the **Volume Snapshots** tab, click the Action menu (⋮) next to the volume snapshot you want to restore.
12. Click **Restore as new PVC**.
13. Enter a name for the new PVC.
14. Select the **Storage Class** name.
15. Select the **Access Mode** of your choice.
16. Optional: For RBD, select **Volume mode**.
17. Click **Restore**. You are redirected to the new PVC details page.

Verification

- Click **Storage → Persistent Volume Claims** from the OpenShift Web Console and confirm that the new PVC is listed in the **Persistent Volume Claims** page.
- Wait for the new PVC to reach **Bound** state.

17.3. DELETING VOLUME SNAPSHOTS

Prerequisites

- For deleting a volume snapshot, the volume snapshot class which is used in that particular volume snapshot should be present.

Procedure

1. From the Persistent Volume Claims page:
   a. Click **Storage → Persistent Volume Claims** from the OpenShift Web Console.
2. Click on the PVC name which has the volume snapshot that needs to be deleted.
3. In the **Volume Snapshots** tab, beside the desired volume snapshot, click Action menu **(⋮) → Delete Volume Snapshot**

   From Volume Snapshots page

4. Click **Storage → Volume Snapshots** from the OpenShift Web Console.
5. In the **Volume Snapshots** page, beside the desired volume snapshot click Action menu **(⋮) → Delete Volume Snapshot**

Verification

- Ensure that the deleted volume snapshot is not present in the **Volume Snapshots** tab of the PVC details page.

- Click **Storage → Volume Snapshots** and ensure that the deleted volume snapshot is not listed.

CHAPTER 18. VOLUME CLONING

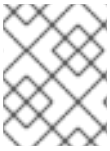
Volume clones create independent, point-in-time duplicates of persistent volume claims for testing, development, or data migration without impacting the source volume.

A clone is a duplicate of an existing storage volume that is used as any standard volume. You create a clone of a volume to make a point in time copy of the data. A persistent volume claim (PVC) cannot be cloned with a different size. You can create up to 512 clones per PVC for both CephFS and RADOS Block Device (RBD).

18.1. CREATING A CLONE

Prerequisites

- Source PVC must be in **Bound** state and must not be in use.



NOTE

Do not create a clone of a PVC if a Pod is using it. Doing so might cause data corruption because the PVC is not quiesced (paused).

Procedure

1. Click **Storage → Persistent Volume Claims** from the OpenShift Web Console.
2. To create a clone, do one of the following:
   - Beside the desired PVC, click Action menu **(⋮) → Clone PVC**.
   - Click on the PVC that you want to clone and click **Actions → Clone PVC**.
3. Enter a **Name** for the clone.
4. Select the access mode of your choice.
5. Enter the required size of the clone.
6. Select the storage class in which you want to create the clone.
   The storage class can be any RBD storage class and it need not necessarily be the same as the parent PVC.
7. Click **Clone**. You are redirected to the new PVC details page.
8. Wait for the cloned PVC status to become **Bound**.
   The cloned PVC is now available to be consumed by the pods. This cloned PVC is independent of its dataSource PVC.

CHAPTER 19. REPLACING STORAGE NODES

You can choose one of the following procedures to replace storage nodes:

- Replacing operational nodes on Red Hat OpenStack Platform installer-provisioned infrastructure
- Replacing failed nodes on Red Hat OpenStack Platform installer-provisioned infrastructure

CHAPTER 20. REPLACING OPERATIONAL NODES ON RED HAT OPENSTACK PLATFORM INSTALLER-PROVISIONED INFRASTRUCTURE

Procedure

1. Log in to the OpenShift Web Console, and click **Compute → Nodes**.
2. Identify the node that you need to replace. Take a note of its **Machine Name**.
3. Mark the node as unschedulable:

```
$ oc adm cordon <node_name>
```

<node_name>

Specify the name of node that you need to replace.

4. Drain the node:

```
$ oc adm drain <node_name> --force --delete-emptydir-data=true --ignore-daemonsets
```



IMPORTANT

This activity might take at least 5 - 10 minutes or more. Ceph errors generated during this period are temporary and are automatically resolved when you label the new node, and it is functional.

5. Click **Compute → Machines**. Search for the required machine.
6. Besides the required machine, click **Action menu (⋮) → Delete Machine**.
7. Click **Delete** to confirm that the machine is deleted. A new machine is automatically created.
8. Wait for the new machine to start and transition into **Running** state.



IMPORTANT

This activity might take at least 5 - 10 minutes or more.

9. Click **Compute → Nodes**. Confirm that the new node is in **Ready** state.
10. Apply the OpenShift Data Foundation label to the new node:

From the user interface

a. For the new node, click **Action Menu (⋮) → Edit Labels**.

b. Add **cluster.ocs.openshift.io/openshift-storage**, and click **Save**.

From the command-line interface

- Apply the OpenShift Data Foundation label to the new node:

```
$ oc label node <new_node_name> cluster.ocs.openshift.io/openshift-storage=""
```

<new_node_name>

Specify the name of the new node.

Verification

1. Verify that the new node is present in the output:

```
$ oc get nodes --show-labels | grep cluster.ocs.openshift.io/openshift-storage= |cut -d' ' -f1
```

2. Click **Workloads → Pods**. Confirm that at least the following pods on the new node are in **Running** state:
   - **openshift-storage.cephfs.csi.ceph.com-***
   - **openshift-storage.rbd.csi.ceph.com-***
3. Verify that all the other required OpenShift Data Foundation pods are in **Running** state.
4. Verify that the new Object Storage Device (OSD) pods are running on the replacement node:

```
$ oc get pods -o wide -n openshift-storage| egrep -i <new_node_name> | egrep osd
```

5. Optional: If cluster-wide encryption is enabled on the cluster, verify that the new OSD devices are encrypted.
   For each of the new nodes identified in the previous step, do the following:
   a. Create a debug pod and open a chroot environment for the one or more selected hosts:

```
$ oc debug node/<node_name>
```

```
$ chroot /host
```

   b. Display the list of available block devices:

```
$ lsblk
```

   Check for the **crypt** keyword beside the one or more **ocs-deviceset** names.
6. If the verification steps fail, contact Red Hat Support .

CHAPTER 21. REPLACING FAILED NODES ON RED HAT OPENSTACK PLATFORM INSTALLER-PROVISIONED INFRASTRUCTURE

Procedure

1. Log in to the OpenShift Web Console, and click **Compute → Nodes**.
2. Identify the faulty node, and click on its **Machine Name**.
3. Click **Actions → Edit Annotations**, and click **Add More**.
4. Add **machine.openshift.io/exclude-node-draining**, and click **Save**.
5. Click **Actions → Delete Machine**, and click **Delete**.
6. A new machine is automatically created, wait for new machine to start.



IMPORTANT

This activity might take at least 5 - 10 minutes or more. Ceph errors generated during this period are temporary and are automatically resolved when you label the new node, and it is functional.

7. Click **Compute → Nodes**. Confirm that the new node is in **Ready** state.
8. Apply the OpenShift Data Foundation label to the new node using any one of the following:

   From the user interface

   a. For the new node, click **Action Menu (⋮) → Edit Labels**.

   b. Add **cluster.ocs.openshift.io/openshift-storage**, and click **Save**.

   From the command-line interface

   - Apply the OpenShift Data Foundation label to the new node:

   ```
   $ oc label node <new_node_name> cluster.ocs.openshift.io/openshift-storage=""
   ```

   <new_node_name>

   Specify the name of the new node.

9. Optional: If the failed Red Hat OpenStack Platform instance is not removed automatically, terminate the instance from Red Hat OpenStack Platform console.

Verification

1. Verify that the new node is present in the output:

   ```
   $ oc get nodes --show-labels | grep cluster.ocs.openshift.io/openshift-storage= |cut -d' ' -f1
   ```

2. Click **Workloads → Pods**. Confirm that at least the following pods on the new node are in **Running** state:
   - **openshift-storage.cephfs.csi.ceph.com-***
   - **openshift-storage.rbd.csi.ceph.com-***
3. Verify that all the other required OpenShift Data Foundation pods are in **Running** state.
4. Verify that the new Object Storage Device (OSD) pods are running on the replacement node:

```
$ oc get pods -o wide -n openshift-storage| egrep -i <new_node_name> | egrep osd
```

5. Optional: If cluster-wide encryption is enabled on the cluster, verify that the new OSD devices are encrypted.
   For each of the new nodes identified in the previous step, do the following:
   a. Create a debug pod and open a chroot environment for the one or more selected hosts:

```
$ oc debug node/<node_name>
```

```
$ chroot /host
```

   b. Display the list of available block devices:

```
$ lsblk
```

   Check for the **crypt** keyword beside the one or more **ocs-deviceset** names.
6. If the verification steps fail, contact Red Hat Support .

CHAPTER 22. REPLACING STORAGE DEVICES

CHAPTER 23. REPLACING OPERATIONAL OR FAILED STORAGE DEVICES ON RED HAT OPENSTACK PLATFORM INSTALLER-PROVISIONED INFRASTRUCTURE

Use this procedure to replace storage device in OpenShift Data Foundation which is deployed on Red Hat OpenStack Platform. This procedure helps to create a new Persistent Volume Claim (PVC) on a new volume and remove the old object storage device (OSD).

Procedure

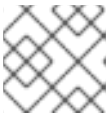
1. Identify the OSD that needs to be replaced and the OpenShift Container Platform node that has the OSD scheduled on it.

```
$ oc get -n openshift-storage pods -l app=rook-ceph-osd -o wide
```

Example output:

```
rook-ceph-osd-0-6d77d6c7c6-m8xj6    0/1    CrashLoopBackOff    0    24h    10.129.0.16
compute-2   <none>          <none>
rook-ceph-osd-1-85d99fb95f-2svc7    1/1    Running             0    24h    10.128.2.24    compute-
0   <none>          <none>
rook-ceph-osd-2-6c66cdb977-jp542    1/1    Running             0    24h    10.130.0.18    compute-
1   <none>          <none>
```

In this example, **rook-ceph-osd-0-6d77d6c7c6-m8xj6** needs to be replaced and **compute-2** is the OpenShift Container platform node on which the OSD is scheduled.



NOTE

If the OSD to be replaced is healthy, the status of the pod will be **Running**.

2. Scale down the **rook-ceph-operator** and **ocs-operator** deployments.

```
$ oc -n openshift-storage scale deployment rook-ceph-operator --replicas=0
```

```
$ oc -n openshift-storage scale deployment ocs-operator --replicas=0
```

3. Scale down the OSD deployment for the OSD to be replaced.

```
$ osd_id_to_remove=0
$ oc scale -n openshift-storage deployment rook-ceph-osd-${osd_id_to_remove} --replicas=0
```

where, **osd_id_to_remove** is the integer in the pod name immediately after the **rook-ceph-osd** prefix. In this example, the deployment name is **rook-ceph-osd-0**.

Example output:

```
deployment.extensions/rook-ceph-osd-0 scaled
```

4. Verify that the **rook-ceph-osd** pod is terminated.

```
$ oc get -n openshift-storage pods -l ceph-osd-id=${osd_id_to_remove}
```

Example output:

```
No resources found.
```



NOTE

If the **rook-ceph-osd** pod is in **terminating** state, use the **force** option to delete the pod.

```
$ oc delete pod rook-ceph-osd-0-6d77d6c7c6-m8xj6 --force --grace-period=0
```

Example output:

```
warning: Immediate deletion does not wait for confirmation that the running resource has been terminated. The resource may continue to run on the cluster indefinitely.
  pod "rook-ceph-osd-0-6d77d6c7c6-m8xj6" force deleted
```

5. Incase, the persistent volume associated with the failed OSD fails, get the failed persistent volumes details and delete them using the following commands:

```
$ oc get pv
$ oc delete pv <failed-pv-name>
```

6. Remove the old OSD from the cluster so that a new OSD can be added.

   a. Delete any old **ocs-osd-removal** jobs.

   ```
   $ oc delete -n openshift-storage job ocs-osd-removal-${osd_id_to_remove}
   ```

   Example output:

   ```
   job.batch "ocs-osd-removal-0" deleted
   ```

   b. Change to the **openshift-storage** project.

   ```
   $ oc project openshift-storage
   ```

   c. Remove the old OSD from the cluster.

   ```
   $ oc process -n openshift-storage ocs-osd-removal -p
   FAILED_OSD_IDS=${osd_id_to_remove} FORCE_OSD_REMOVAL=false |oc create -n
   openshift-storage -f -
   ```

   You can add comma separated OSD IDs in the command to remove more than one OSD. (For example, FAILED_OSD_IDS=0,1,2).

The FORCE_OSD_REMOVAL value must be changed to “true” in clusters that only have three OSDs, or clusters with insufficient space to restore all three replicas of the data after the OSD is removed.



WARNING

Proceeding will permanently remove the OSD from the cluster and destroy its data. Ensure **osd_id_to_remove** is correct. Do not process multiple IDs from the same failure domain at once. Contact Red Hat Technical Support if you need assistance.

7. Verify that the OSD was removed successfully by checking the status of the **ocs-osd-removal-job** pod.
   A status of **Completed** confirms that the OSD removal job succeeded.

```
# oc get pod -l job-name=ocs-osd-removal-job -n openshift-storage
```

8. Ensure that the OSD removal is completed.

```
$ oc logs -l job-name=ocs-osd-removal-job -n openshift-storage --tail=-1 | egrep -i 'completed removal'
```

   Example output:

```
2022-05-10 06:50:04.501511 I | cephosd: completed removal of OSD 0
```



IMPORTANT

If the **ocs-osd-removal-job** fails and the pod is not in the expected **Completed** state, check the pod logs for further debugging.

For example:

```
# oc logs -l job-name=ocs-osd-removal-job -n openshift-storage --tail=-1
```

9. If encryption was enabled at the time of install, remove **dm-crypt** managed **device-mapper** mapping from the OSD devices that are removed from the respective OpenShift Data Foundation nodes.

   a. Get PVC name(s) of the replaced OSD(s) from the logs of **ocs-osd-removal-job** pod :

```
$ oc logs -l job-name=ocs-osd-removal-job -n openshift-storage --tail=-1  |egrep -i ‘pvc|deviceset’
```

   For example:

```
2021-05-12 14:31:34.666000 I | cephosd: removing the OSD PVC "ocs-deviceset-xxxx-xxx-xxx-xxx"
```

b. For each of the nodes identified in step #1, do the following:

i. Create a **debug** pod and **chroot** to the host on the storage node.

```
$ oc debug node/<node name>
$ chroot /host
```

ii. Find relevant device name based on the PVC names identified in the previous step

```
sh-4.4# dmsetup ls| grep <pvc name>
ocs-deviceset-xxx-xxx-xxx-xxx-block-dmcrypt (253:0)
```

iii. Remove the mapped device.

```
$ cryptsetup luksClose --debug --verbose ocs-deviceset-xxx-xxx-xxx-xxx-block-dmcrypt
```



NOTE

If the above command gets stuck due to insufficient privileges, run the following commands:

- Press **CTRL+Z** to exit the above command.
- Find PID of the process which was stuck.

```
$ ps -ef | grep crypt
```

- Terminate the process using **kill** command.

```
$ kill -9 <PID>
```

- Verify that the device name is removed.

```
$ dmsetup ls
```

10. Delete the **ocs-osd-removal** job.

```
$ oc delete -n openshift-storage job ocs-osd-removal-${osd_id_to_remove}
```

Example output:

```
job.batch "ocs-osd-removal-0" deleted
```

11. Scale up the **rook-ceph-operator** and **ocs-operator** deployments.

```
$ oc -n openshift-storage scale deployment rook-ceph-operator --replicas=1
```

```
$ oc -n openshift-storage scale deployment ocs-operator --replicas=1
```

Verification

1. Verify that there is a new OSD running.

```
$ oc get -n openshift-storage pods -l app=rook-ceph-osd
```

Example output:

```
rook-ceph-osd-0-5f7f4747d4-snshw                 1/1     Running     0          4m47s
rook-ceph-osd-1-85d99fb95f-2svc7                 1/1     Running     0          1d20h
rook-ceph-osd-2-6c66cdb977-jp542                 1/1     Running     0          1d20h
```

2. Verify that there is a new PVC created which is in **Bound** state.

```
$ oc get -n openshift-storage pvc
```

Example output:

```
NAME                      STATUS   VOLUME                                     CAPACITY   ACCESS
MODES   STORAGECLASS                  AGE
noobaa-db-pg-cluster-1    Bound    pvc-dfd99eb2-8616-4663-be7e-2fcabe8130f8   50Gi
RWO           ocs-storagecluster-ceph-rbd   <unset>                 26h
noobaa-db-pg-cluster-2    Bound    pvc-cd757f56-1864-423e-a08e-fe60b6a448dc   50Gi
RWO           ocs-storagecluster-ceph-rbd   <unset>                 26h
ocs-deviceset-0-data-0-gwb5l   Bound    pvc-bea680cd-7278-463d-a4f6-3eb5d3d0defe
512Gi      RWO           standard                  94s
ocs-deviceset-1-data-0-w9pjm   Bound    pvc-01aded83-6ef1-42d1-a32e-6ca0964b96d4
512Gi      RWO           standard                  6d
ocs-deviceset-2-data-0-7bxcq   Bound    pvc-5d07cd6c-23cb-468c-89c1-72d07040e308
512Gi      RWO           standard                  6d
```

3. Optional: If cluster-wide encryption is enabled on the cluster, verify that the new OSD devices are encrypted.

   a. Identify the nodes where the new OSD pods are running.

```
$ oc get -n openshift-storage -o=custom-columns=NODE:.spec.nodeName pod/_<OSD-
pod-name>_
```

<OSD-pod-name>

Is the name of the OSD pod.
For example:

```
$ oc get -n openshift-storage -o=custom-columns=NODE:.spec.nodeName pod/rook-
ceph-osd-0-544db49d7f-qrgqm
```

Example output:

```
NODE
compute-1
```

b. For each of the nodes identified in previous step, do the following:

i. Create a debug pod and open a chroot environment for the selected host(s).

```
$ oc debug node/<node name>
$ chroot /host
```

ii. Run “lsblk” and check for the “crypt” keyword beside the **ocs-deviceset** name(s)

```
$ lsblk
```

4. Log in to OpenShift Web Console and view the storage dashboard.
OSD status in OpenShift Container Platform storage dashboard after device replacement
image::images/ocs_storage_dashboard_healthy_osd.png[RHOCP storage dashboard showing the healthy OSD.]

CHAPTER 24. UPGRADING TO OPENSHIFT DATA FOUNDATION

Upgrade from OpenShift Container Storage to OpenShift Data Foundation using the Operator Lifecycle Manager update channel and follow post-upgrade verification steps.

24.1. OVERVIEW OF THE OPENSHIFT DATA FOUNDATION UPDATE PROCESS

This chapter helps you to upgrade between the minor releases and z-streams for all Red Hat OpenShift Data Foundation deployments (Internal, Internal-Attached and External). The upgrade process remains the same for all deployments.

You can upgrade OpenShift Data Foundation and its components, either between minor releases like 4.21 and 4.22, or between z-stream updates like 4.22.0 and 4.22.1 by enabling automatic updates (if not done so during operator installation) or performing manual updates. When a new z-stream release becomes available, the upgrade process triggers automatically if the update strategy was set to Automatic. It is recommended to take snapshots of all the volumes in use before running any maintenance or upgrade procedures.

Extended Update Support (EUS)

EUS to EUS upgrade in OpenShift Data Foundation is sequential and it is aligned with OpenShift upgrade. For more information, see *Performing a Control Plane Only update* and *Control Plane Only updates for layered products and Operators installed through Operator Lifecycle Manager* in the OpenShift Container Platform documentation.

For EUS upgrade of OpenShift Container Platform and OpenShift Data Foundation, make sure that OpenShift Data Foundation is upgraded along with OpenShift Container Platform and compatibility between OpenShift Data Foundation and OpenShift Container Platform is always maintained.

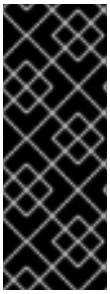
Example workflow of EUS upgrade:

1. Pause the worker machine pools.
2. Update OpenShift <4.y> → OpenShift <4.y+1>.
3. Update OpenShift Data Foundation <4.y> → OpenShift Data Foundation <4.y+1>.
4. Update OpenShift <4.y+1> → OpenShift <4.y+2>.
5. Update to OpenShift Data Foundation <4.y+2>.
6. Unpause the worker machine pools.



NOTE

You can update to ODF <4.y+2> either before or after worker machine pools are unpaused.



IMPORTANT

When you update OpenShift Data Foundation in external mode, make sure that the Red Had Ceph Storage and OpenShift Data Foundation versions are compatible. For more information about supported Red Had Ceph Storage version in external mode, refer to the *Red Hat OpenShift Data Foundation Supportability and Interoperability Checker* . Provide the required OpenShift Data Foundation version in the checker to see the supported Red Had Ceph version corresponding to the version in use.

You also need to upgrade the different parts of Red Hat OpenShift Data Foundation in the following order for both internal and external mode deployments:

1. **Update OpenShift Container Platform** according to the *Updating clusters* documentation for OpenShift Container Platform.
2. **Update Red Hat OpenShift Data Foundation.**
    a. **To prepare a disconnected environment for updates**, see *Using Operator Lifecycle Manager in disconnected environments* to be able to update OpenShift Data Foundation as well as Local Storage Operator when in use. For the list of packages to be mirrored for upgrading OpenShift Data Foundation in disconnected environments, see the Disconnected environment section in the *Planning your deployment* guide.
    b. **For updating between minor releases**, see *Updating Red Hat OpenShift Data Foundation 4.20 to 4.22*.
    c. **For updating between z-stream releases**, see *Updating Red Hat OpenShift Data Foundation 4.22.x to 4.22.y*.
    d. **For updating external mode deployments**, you must also perform the steps from the *Updating the Red Hat OpenShift Data Foundation external secret* section.
    e. **If you use local storage, then update the Local Storage operator** See *Checking for Local Storage Operator deployments* if you are unsure.



IMPORTANT

If you have an existing setup of OpenShift Data Foundation 4.21 with disaster recovery (DR) enabled, ensure to update all your clusters in the environment at the same time and avoid updating a single cluster. This is to avoid any potential issues and maintain best compatibility. It is also important to maintain consistency across all OpenShift Data Foundation DR instances.

Update considerations

Review the following important considerations before you begin.

- The Red Hat OpenShift Container Platform version is the same as Red Hat OpenShift Data Foundation. See the *Interoperability Matrix* for more information about supported combinations of OpenShift Container Platform and Red Hat OpenShift Data Foundation.
- To know whether your cluster was deployed in internal or external mode, see *How to determine if ODF cluster has storage in internal or external mode*.
- The Local Storage Operator is fully supported only when the Local Storage Operator version matches the Red Hat OpenShift Container Platform version.

- In OpenShift Data Foundation clusters with disaster recovery (DR) enabled, during upgrade to version 4.22, previous **bluestore-rdr** OSDs are migrated to **bluestore** OSDs. **bluestore** backed OSDs now provide the same improved performance of **bluestore-rdr** based OSDs, which is important when the cluster is required to be used for Regional Disaster Recovery.
  During upgrade you can view the status of the OSD migration. In the OpenShift Web Console, navigate to **Storage → Data Foundation → Storage System**. In the **Activity** card of the **Block and File** tab you can view ongoing activities. **Migrating cluster OSDs** shows the status of the migration from **bluestore-rdr** to **bluestore**.



IMPORTANT

To avoid data loss, it is recommended to take a backup of NooBaa DB PVC regularly. If NooBaa DB fails and cannot be recovered, then you can revert to the latest backed-up version. For instructions on backing up NooBaa DB, follow the steps in *Perform a One-Time Backup of the Database for the Multicloud Object Gateway*.

Additional resources

- Performing a Control Plane Only update
- Control Plane Only updates for layered products and Operators installed through Operator Lifecycle Manager
- Red Hat OpenShift Data Foundation Supportability and Interoperability Checker
- Updating clusters
- Using Operator Lifecycle Manager in disconnected environments
- Disconnected environment
- Updating Red Hat OpenShift Data Foundation 4.20 to 4.22
- Updating Red Hat OpenShift Data Foundation 4.22.x to 4.22.y
- Updating the Red Hat OpenShift Data Foundation external secret
- Checking for Local Storage Operator deployments
- Interoperability Matrix
- How to determine if ODF cluster has storage in internal or external mode
- Perform a One-Time Backup of the Database for the Multicloud Object Gateway

24.2. UPDATING RED HAT OPENSHIFT DATA FOUNDATION 4.21 TO 4.22

This chapter helps you to upgrade between the minor releases for all Red Hat OpenShift Data Foundation deployments (Internal, Internal-Attached and External). The upgrade process remains the same for all deployments. The Only difference is what gets upgraded and what's not.

- For Internal and Internal-attached deployments, upgrading OpenShift Data Foundation upgrades all OpenShift Data Foundation services including the backend Red Hat Ceph Storage (RHCS) cluster.

- For External mode deployments, upgrading OpenShift Data Foundation only upgrades the OpenShift Data Foundation service while the backend Ceph storage cluster remains untouched and needs to be upgraded separately.
  You must upgrade Red Hat Ceph Storage along with OpenShift Data Foundation to get new feature support, security fixes, and other bug fixes. As there is no dependency on RHCS upgrade, you can upgrade the OpenShift Data Foundation operator first followed by RHCS upgrade or vice-versa. For more information about RHCS releases, see the knowledgebase solution, Red Hat Ceph Storage releases and corresponding Ceph package versions .



IMPORTANT

Upgrading to 4.22 directly from any version older than 4.21 is not supported.

When upgrading from OpenShift Data Foundation 4.21.z to 4.22, you must upgrade to OpenShift Data Foundation 4.22.2 or later versions. Upgrades to OpenShift Data Foundation 4.22.0 or 4.22.1 are not supported and will fail.

Prerequisites

- Ensure that the **OpenShift Container Platform** cluster has been updated to the latest stable release of version 4.22.x, see Updating Clusters.
- Ensure that the **OpenShift Data Foundation** cluster is healthy and data is resilient.
  - Navigate to **Storage → Data Foundation → Storage Systems** tab and then click on the storage system name.
  - Check for the green tick on the status card of both **Overview - Block and File** and **Object** tabs. Green tick indicates that the *storage cluster*, *object service* and *data resiliency* are all healthy.
- Ensure that all **OpenShift Data Foundation** Pods, including the operator pods, are in **Running** state in the **openshift-storage** namespace.
  To view the state of the pods, on the OpenShift Web Console, click **Workloads → Pods**. Select **openshift-storage** from the **Project** drop-down list.



NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

- Ensure that you have sufficient time to complete the OpenShift Data Foundation update process, as the update time varies depending on the number of OSDs that run in the cluster.
- Optional: To reduce the upgrade time for large clusters, it is encouraged to increase the **maxUnavailable** for **NodePlugin**. Use the following command:

```
VAL=$(oc get nodes -o name | awk 'END {printf "%.0f\n", NR * 0.2}') && oc patch driver openshift-storage.rbd.csi.ceph.com -n openshift-storage --type='merge' -p "{\"spec\":{\"nodePlugin\":{\"updateStrategy\":{\"rollingUpdate\":{\"maxUnavailable\":$VAL}}}}}" && oc patch driver openshift-storage.cephfs.csi.ceph.com -n openshift-storage --type='merge' -p "{\"spec\":{\"nodePlugin\":{\"updateStrategy\":{\"rollingUpdate\":{\"maxUnavailable\":$VAL}}}}}"
```

This affects CephFS and CephRBD daemonsets and allows the pods to restart simultaneously or be unavailable and reduce the upgrade time. For an optimal value, you can set the parameter values to

20%. However, if the value is too high, disruption for new volumes might be observed during the upgrade.

- **Prerequisite relevant only for OpenShift Data Foundation deployments on AWS using AWS Security Token Service (STS)**
  Add another entry in the trust policy for **noobaa-core** account as follows:

  1. Log into AWS web console where the AWS role resides using http://console.aws.amazon.com/.
  2. Enter the IAM management tool and click **Roles**.
  3. Find the name of the role created for AWS STS to support Multicloud Object Gateway (MCG) authentication using the following command in OpenShift CLI:

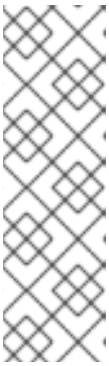
     ```
     $ oc get deployment noobaa-operator -o yaml -n openshift-storage | grep ROLEARN -A1
           value: arn:aws:iam::123456789101:role/your-role-name-here
     ```

  4. Search for the role name that you obtained from the previous step in the tool and click on the role name.
  5. Under the role summary, click **Trust relationships**.
  6. In the **Trusted entities** tab, click **Edit trust policy** on the right.
  7. Under the **"Action": "sts:AssumeRoleWithWebIdentity"** field, there are two fields to enable access for two NooBaa service accounts **noobaa** and **noobaa-endpoint**. Add another entry for the core pod's new service account name, **system:serviceaccount:openshift-storage:noobaa-core**.
  8. Click **Update policy** at the bottom right of the page.
     The update might take about 5 minutes to get in place.

Procedure

1. On the OpenShift Web Console, navigate to **Ecosystem → Installed Operators**.
2. Select **openshift-storage** project.
3. Click the OpenShift Data Foundation operator name.
4. Click the **Subscription** tab and click the link under **Update Channel**.
5. Select the **stable-4.22** update channel and **Save** it.
6. If the **Upgrade status** shows **requires approval**, click on **requires approval**.
   a. On the Install Plan Details page, click **Preview Install Plan**.
   b. Review the install plan and click **Approve**.
      Wait for the **Status** to change from **Unknown** to **Created**.
7. Navigate to **Ecosystem → Installed Operators**.
8. Select the **openshift-storage** project.
   Wait for the OpenShift Data Foundation Operator **Status** to change to **Up to date**.

9. After the operator is successfully upgraded, the web console automatically reloads to apply the changes. During this process, a temporary error message might appear on the page and this is expected and disappears after the refresh completes.



NOTE

After upgrading, if your cluster has five or more nodes, racks, or rooms, and when there are five or more number of failure domains present in the deployment, you can configure Ceph monitor counts based on the number of racks or zones. An alert is displayed in the notification panel or Alert Center of the OpenShift Web Console to indicate the option to increase the number of Ceph monitor counts. You can use the **Configure** option in the alert to configure the Ceph monitor counts. For more information, see *Resolving low Ceph monitor count alert* in the Troubleshooting guide.

Verification

- Check the **Version** after the OpenShift Data Foundation name and check the operator status.
  - Navigate to **Ecosystem → Installed Operators** and select the **openshift-storage** project.
  - When the upgrade completes, the version updates to a new version number for OpenShift Data Foundation and status changes to **Succeeded** with a green tick.
- Verify that the **OpenShift Data Foundation** cluster is healthy and data is resilient.
  - Navigate to **Storage → Data Foundation → Storage Systems** tab and then click on the storage system name.
  - Check for the green tick on the status card of **Overview- Block and File** and **Object** tabs. Green tick indicates that the storage cluster, object service and data resiliency is healthy.
- If verification steps fail, contact Red Hat Support .



IMPORTANT

After updating external mode deployments, you must also update the external secret. For instructions, see *Updating the OpenShift Data Foundation external secret* .

Additional resources

- Updating the OpenShift Data Foundation external secret
- Commonly required logs for troubleshooting
- Resolving low Ceph monitor count alert

24.3. UPDATING RED HAT OPENSHIFT DATA FOUNDATION 4.22.X TO 4.22.Y

Apply a OpenShift Data Foundation z-stream update within the 4.22 release to receive bug fixes and security patches without changing the minor version, using either the automatic approval strategy or by manually approving the generated InstallPlan.

- For Internal and Internal-attached deployments, upgrading OpenShift Data Foundation upgrades all OpenShift Data Foundation services including the backend Red Hat Ceph Storage (RHCS) cluster.

- For External mode deployments, upgrading OpenShift Data Foundation only upgrades the OpenShift Data Foundation service while the backend Ceph storage cluster remains untouched and needs to be upgraded separately.
  Hence, we recommend upgrading RHCS along with OpenShift Data Foundation in order to get new feature support, security fixes, and other bug fixes. Since we do not have a strong dependency on RHCS upgrade, you can upgrade the OpenShift Data Foundation operator first followed by RHCS upgrade or vice-versa. See solution to know more about RHCS releases.

When a new z-stream release becomes available, the upgrade process triggers automatically if the update strategy was set to **Automatic**. If the update strategy is set to **Manual** then use the following procedure.

Prerequisites

- Ensure that the **OpenShift Container Platform** cluster has been updated to the latest stable release of version 4.22.x, see Updating Clusters.

- Ensure that the **OpenShift Data Foundation** cluster is healthy and data is resilient.

  - Navigate to **Storage → Data Foundation → Storage Systems** tab and then click on the storage system name.

  - Check for the green tick on the status card of **Overview - Block and File** and **Object** tabs. Green tick indicates that the storage cluster, object service and data resiliency is healthy.

- Ensure that all **OpenShift Data Foundation** Pods, including the operator pods, are in **Running** state in the **openshift-storage** namespace.
  To view the state of the pods, on the OpenShift Web Console, click **Workloads → Pods**. Select **openshift-storage** from the **Project** drop-down list.



NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

- Ensure that you have sufficient time to complete the OpenShift Data Foundation update process, as the update time varies depending on the number of OSDs that run in the cluster.

Procedure

1. On the OpenShift Web Console, navigate to **Ecosystem → Installed Operators**.
2. Select **openshift-storage** project.



NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

3. Click the **OpenShift Data Foundation** operator name.

4. Click the **Subscription** tab.
5. If the **Upgrade Status** shows **require approval**, click on **requires approval** link.
6. On the **InstallPlan Details** page, click **Preview Install Plan**.
7. Review the install plan and click **Approve**.
8. Wait for the Status to change from **Unknown** to **Created**.
9. After the operator is successfully installed, the web console automatically reloads to apply the changes. During this process, a temporary error message might appear on the page and this is expected and disappears after the refresh completes.

Verification

- Check the **Version** after the OpenShift Data Foundation name and check the operator status.
  - Navigate to **Ecosystem → Installed Operators** and select the **openshift-storage** project.
  - When the upgrade completes, the version updates to a new version number for OpenShift Data Foundation and status changes to **Succeeded** with a green tick.
- Verify that the OpenShift Data Foundation cluster is healthy and data is resilient.
  - Navigate to **Storage → Data Foundation → Storage Systems** tab and then click on the storage system name.
  - Check for the green tick on the status card of **Overview - Block and File** and **Object** tabs. Green tick indicates that the storage cluster, object service and data resiliency is healthy
- If verification steps fail, contact Red Hat Support.

24.4. CHANGING THE UPDATE APPROVAL STRATEGY

To ensure that the storage system gets updated automatically when a new update is available in the same channel, we recommend keeping the update approval strategy to **Automatic**. Changing the update approval strategy to **Manual** will need manual approval for each upgrade.

Procedure

1. Navigate to **Ecosystem → Installed Operators**.
2. Select **openshift-storage** from the **Project** drop-down list.



NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

3. Click on **OpenShift Data Foundation** operator name
4. Go to the **Subscription** tab.
5. Click on the **pencil** icon for changing the **Update approval**.

6. Select the update approval strategy and click **Save**.

Verification

- Verify that the Update approval shows the newly selected approval strategy after it.